

Managing O-RAN Networks: xApp Development from Zero to Hero

Joao F. Santos[✉], *Member, IEEE*, Alexandre Huff[✉], Daniel Campos[✉], Kleber V. Cardoso[✉], Cristiano B. Both[✉], and Luiz A. DaSilva[✉], *Fellow, IEEE*

Abstract—The Open Radio Access Network (O-RAN) Alliance proposes an open architecture that disaggregates the RAN and supports executing custom control logic in near-real time from third-party applications, the xApps. Despite O-RAN's efforts, the creation of xApps remains a complex and time-consuming endeavor, aggravated by the sometimes fragmented, outdated, or deprecated documentation from the O-RAN Software Community (OSC). These challenges hinder academia and industry from developing and validating solutions and algorithms on O-RAN networks. This tutorial addresses this gap by providing the first comprehensive guide for developing xApps to manage the O-RAN ecosystem from theory to practice. We provide a thorough theoretical foundation of the O-RAN architecture and detail the functionality offered by Near Real-Time RAN Intelligent Controller (Near-RT RIC) components. We examine the xApp design and configuration. We explore the xApp lifecycle and demonstrate how to deploy and manage xApps on a Near-RT RIC. We address the xApps' interfaces and capabilities, accompanied by practical examples. We provide comprehensive details on how xApps can control the RAN. We discuss debugging strategies and good practices to aid the xApp developers in testing their xApps. Finally, we review the current landscape and open challenges for creating xApps.

Index Terms—O-RAN, Disaggregated Networks, xApp, RAN Management, Near-RT RIC

I. INTRODUCTION

Radio Access Networks (RANs) are transitioning from monolithic implementations using specialized hardware in favor of more agile, innovative, and customizable solutions based on disaggregation [1], open interfaces [2], and softwareization [3]. An important manifestation of this transition is embodied in the Open Radio Access Network (O-RAN) vision, which has gained substantial traction through the establishment of a worldwide consortium [4] with broad industry participation and has also attracted regulatory interest [5]. The O-RAN Alliance proposes an open architecture that disaggregates the RAN into different functional components, connected under a common control and management overlay

that can execute custom control logic via third-party applications supplied by, e.g., RAN solutions and consulting companies, Mobile Network Operators (MNOs), the open-source community, and new entrants in the market [6]–[8]. In addition, O-RAN provides specifications that complement and build on top of 3GPP standards, establishing well-defined open interfaces for connecting the disaggregated RAN components to ensure interoperability across different vendors [9].

The O-RAN Alliance aims to promote competition and innovation, empowering MNOs to build more flexible and cost-effective networks, encouraging new entrants and startups, and facilitating collaboration among industry and academia stakeholders, with potential benefits to MNOs and end-users alike [10]. Through the standardization of third-party applications to manage the RAN, the O-RAN vision (i) fosters innovation in the telecom market, allowing MNOs to deploy tailored applications to customize their network operations [11]; (ii) creates conditions for cost saving through competition between app providers, but also between hardware manufacturers; and (iii) ensures the RAN equipment is future-proof, as MNOs can test and validate new solutions and algorithms on their existing physical network infrastructure [12].

The O-RAN Alliance partnered with the Linux Foundation to create the O-RAN Software Community (OSC) [13], an open-source project responsible for creating reference implementations of O-RAN components following the O-RAN specifications, serving as a starting point for prototyping O-RAN solutions [12]–[14]. The OSC supports and distributes a number of first-party xApps, modular applications designed to manage and optimize various aspects and parameters of the RAN. The xApps act as plugin-like extensions, enhancing the capabilities of the RAN and providing MNOs with different functionality, e.g., managing the admission control of UEs, monitoring the Key Performance Measurements (KPMs) of base stations, detecting traffic anomalies, and performing traffic steering [15]. In addition, both academia and industry have developed several third-party xApps to test and demonstrate their solutions and algorithms on real O-RAN networks (detailed further in Section III).

There is a vast literature on O-RAN, covering aspects from the basic understanding and new concepts [16], to how the O-RAN principles are influencing the evolution of mobile networks towards 6G [17], the benefits for mobile operators to adopt O-RAN in their networks [18], and the security vulnerabilities and threat surface introduced by the open, cloud-based O-RAN architecture [19]. However, there remains a significant gap in the literature regarding the theoretical

Manuscript received August 4th, 2024; revised December 19th, 2024; accepted February 4th, 2025. The research leading to this paper received support from the Commonwealth Cyber Initiative, an investment in the advancement of cyber R&D, innovation, and workforce development. This work also received support from the OpenRAN@Brasil Program.

Joao F. Santos and Luiz A. DaSilva are with the Commonwealth Cyber Initiative (CCI) and Virginia Tech, USA. Emails: {joaosantos, ldsilva}@vt.edu

Alexandre Huff is with the Universidade Tecnológica Federal do Paraná, Brazil. Email: alexandrehuff@utfpr.edu.br

Daniel Campos and Kleber V. Cardoso are with the Universidade Federal de Goiás, Brazil. Emails: dante_campos@discente.ufg.br, kleber@ufg.br

Cristiano B. Both is with the Universidade do Vale do Rio dos Sinos, Brazil. Email: cbboth@unisinos.br

foundation and technical background for developing xApps. Despite O-RAN's standardization and development efforts, their creation of an SDK to create xApps with support for different programming languages, and a growing community of developers from academia and industry, the process of creating xApps is still far from a straightforward endeavor.

From an implementation point of view, xApps are highly complex microservices that interact with multiple components of the Near Real-Time RAN Intelligent Controller (Near-RT RIC) through widely different APIs and protocols [6], [12], making it challenging for newcomers to start prototyping their xApps. The few existing works on the development of xApps address specific considerations, e.g., data flows between O-RAN entities [20], interactions with base stations [21], or designing Deep Reinforcement Learning (DRL) agents [6], without providing context about all the features and capabilities available for xApp developers. In addition, some aspects of xApps are still undergoing active standardization, e.g., the AI/ML workflow [22], while others have been left for further studies, e.g., security inside the Near-RT RIC [23], or had a complete revamp in recent O-RAN releases, e.g., subscriptions to information from the RAN (detailed further in Section VI).

The documentation for creating xApps is outdated and fragmented across the OSC's Wiki and Gerrit webpages, with numerous tutorials becoming deprecated as the project evolved. The lack of consolidated and up-to-date tutorials is a well-known issue in the community, which prompted responses from different stakeholders, including practitioners creating educative video series [24], professional organizations developing interactive online resources [25], and private initiatives offering training courses to address the need for easy and accessible documentation regarding O-RAN and xApp development [26]–[28]. Ultimately, the lack of consolidated documentation and comprehensive guidelines available to the community impose barriers for new players in the telecom market and increase the costs for industry and academia to develop, test, and validate their xApps.

The purpose of this tutorial paper is to provide a thorough guide on how to develop xApps, from theory to practice. In particular, the contributions of this paper are as follows:

- We create the first comprehensive guide with instructions for developing, managing, and evaluating xApps, supporting xApp developers from theory to practice.
- We present a theoretical foundation on O-RAN and practical knowledge related to the realization of O-RAN entities and xApps, following the OSC's design choices.
- We provide context and detail the functionality offered by Near-RT RIC components to xApps, accompanied by practical examples to demonstrate their utilization.
- We highlight the current open challenges for developing xApps and testing them in end-to-end scenarios.

This paper accompanies a public online repository containing the supporting material used throughout the tutorial, namely, the xApp descriptor and schema files, example source codes, and Python representations of ASN.1 documents. For additional information, we refer the reader to [29].

Paper Structure: The remainder of this paper is organized as shown in Fig. 1. In Section II, we provide a theoretical

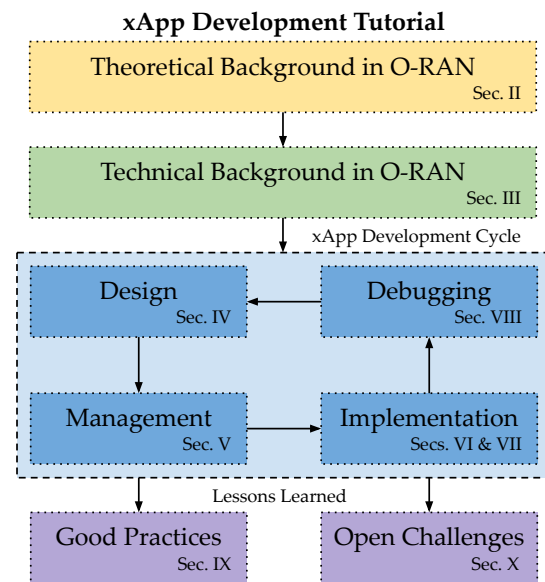


Fig. 1: The xApp development cycle and the tutorial paper organization. Sections II–III provide developers with theoretical and technical background for getting started in O-RAN, Sections IV–VIII detail the xApp development cycle accompanied of practical examples, and Sections IX–X contain lessons learned, with good practices and current open challenges.

background on O-RAN, its architecture, and the components of the Near-RT RIC. In Section III, we discuss O-RAN implementation details, review fundamentals on containers for developing xApps, and provide an overview of current first- and third-party xApps. In Section IV, we detail the xApp architecture and interfaces, and describe how to design and define xApps using configuration and schema files. In Section V, we detail the xApp lifecycle and demonstrate how to interact with the Near-RT RIC to manage xApps. In Section VI, we describe the different xApp interfaces and functionality, providing examples of how to develop xApps capable of communicating with one another, using persistent storage, and reacting to user input. In Section VII, we detail how xApps can subscribe to information from base stations and manage their operation. In Section VIII, we discuss debugging strategies and methods to validate the operation of xApps and test their interfaces. In Section IX, we discuss good practices to facilitate the development of xApps. In Section X, we outline ongoing xApp standardization efforts and discuss open challenges for developing xApps. Finally, in Section XI, we pose our concluding remarks. For ease of reference, we list the acronyms used throughout this paper at the end.

II. THEORETICAL BACKGROUND IN O-RAN

In this section, we review the O-RAN principles and architecture, as illustrated in Fig. 2. We zoom into the Near-RT RIC, describe its internal components, and detail how they provide the functionality to support the operation of xApps.

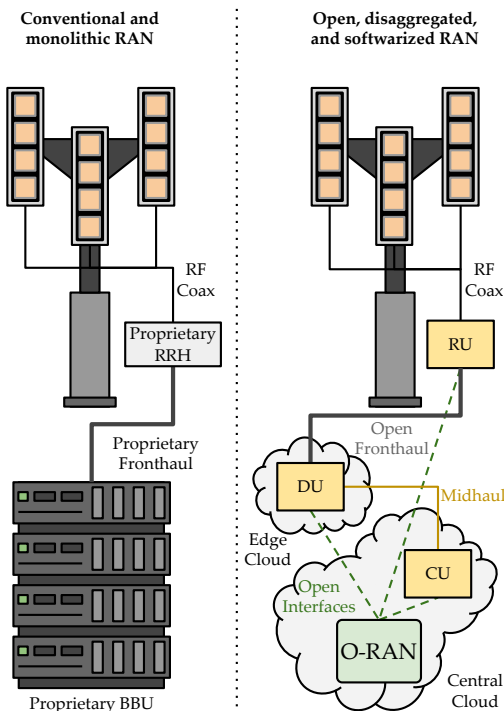


Fig. 2: Comparison between the conventional, monolithic RAN and the O-RAN paradigm, showing the latter's decomposition of the RAN into functional components running in software, interconnected using open interfaces, and orchestrated by a common control and management overlay.

A. Principles and Architecture

The layers of the 5G protocol stack operate at different timescales, and their computational requirements grow at different rates based on the number of users and their demand [30]. As a result, the 3GPP introduced functional splits for 5G, breaking down monolithic RAN deployments with one-size-fits-all Remote Radio Units (RRUs) and Base Band Units (BBUs) into a series of discrete RAN functions that can be placed and scaled on-demand [31]. The O-RAN Alliance examined the split options and selected Split 7.2x to underpin their architecture, due to its balance between the simplicity of the functional components and the throughput and latency requirements for their interfaces. The Split 7.2x decomposes 5G base stations, known as gNodeBs, into CUs, DUs, and RUs that implement different functions of the 5G RAN protocol stack. Specifically, (i) the CUs implement functionalities at the higher layers that operate over larger timescales, e.g., packet processing operations; (ii) the DUs handle time-critical operations at the lower layers, e.g., signal processing operations; and (iii) the RUs manage and interface with Radio Frequency (RF) components, e.g., Fast Fourier Transform (FFT)/Inverse FFT (IFFT) [10]. In addition, O-RAN goes one step further in disaggregation and splits the CU into two logical components: (i) the CU-User Plane (UP), responsible for the Packet Data Convergence Protocol (PDCP) layer's UP and the Service Data Adaptation Protocol (SDAP) layer and (ii) the CU-Control Plane (CP), responsible for the PDCP layer's CP and the Radio Resource Control (RRC) layer.

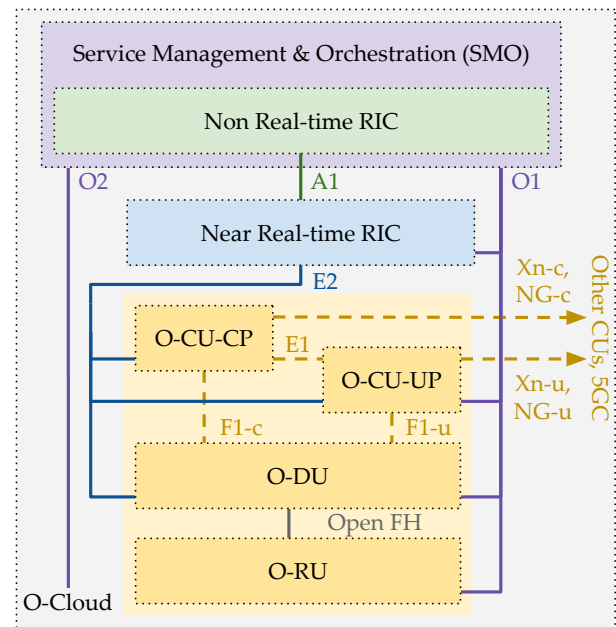


Fig. 3: Logical architecture of O-RAN, showing the RICs and their interfaces for managing the RAN and its E2 Nodes, i.e., O-CU, O-DU, and O-RU. The dashed lines indicate interfaces standardized by the 3GPP, whereas the solid lines indicate new interfaces introduced by the O-RAN Alliance.

This split option allows different components to be deployed at distinct network locations and leverage different hardware accelerators, e.g., Digital Signal Processors (DSPs) and Field Programmable Gate Arrays (FPGAs) [12].

The O-RAN functional split promotes the decoupling between RAN hardware and software components similar to the Cloud RAN (C-RAN) paradigm [32], as DU, CU-UP, and CU-CP can run as software instances in a hierarchy of cloud platforms and be autonomously deployed or scaled on demand [9]. It is important to note that, unlike C-RAN, O-RAN is not oblivious to the underlying computing and virtualization infrastructure, but instead, O-RAN incorporates it as a part of the ecosystem known as Open Cloud (O-Cloud) [12]. This component is an abstraction that combines (i) physical nodes, e.g., servers and data centers; (ii) software components, e.g., containers and virtual machine hypervisors; and (iii) management and orchestration functionalities, e.g., Fault, Configuration, Accounting, Performance, and Security (FCAPS). O-RAN interacts with the O-Cloud through the Service Management and Orchestration (SMO), the central component responsible for orchestration, management, and automation in the MNO's infrastructure, e.g., the Open Source MANO (OSM) [33] or the Open Networking Automation Platform (ONAP) [34]. A new O2 interface connects the SMO and the O-Cloud, enabling the programmatic management and deployment of network functions, the definition of an inventory of the facilities under the O-Cloud, as well as monitoring, fault tolerance, and update strategies [10], [12].

As part of the functional split, the 3GPP standardized interfaces for the communication between CUs, DUs, and RUs. However, these standards allowed vendors to introduce

proprietary extensions between their components, which resulted in vendor lock-in [10]. To mitigate this issue, the O-RAN Alliance has created more restrictive specifications (built on top of 3GPP standards) for open interfaces between RAN components that ensure complete vendor interoperability [12]. In addition, O-RAN introduces new standardized APIs for controlling and managing each E2 Node, including gNodeBs and eNodeBs (4G base stations) and their functional splits, i.e., CUs, DUs, and RUs. These new interfaces contain: the (i) E2 interface for controlling different RAN functions exposed by each node, i.e., the control knobs supported by each node, e.g., handover thresholds, scheduling directives, or power-saving parameters; and the (ii) O1 interface, for operations and maintenance of each node, establishing heartbeats, setting up alarms, and reporting KPMs. Due to these new open interfaces and APIs, the RAN nodes in O-RAN are dubbed as Open (O-) components, e.g., O-CU(-CP/UP), O-DU, and O-RU.

With the introduction of open and programmable interfaces across all E2 Nodes, O-RAN can orchestrate their operation under a common control and management overlay to optimize their performance using data-driven closed-control loops [31]. As mentioned earlier, the O-RAN Alliance envisions two different RICs for running closed-control loops in different locations and timescales: the (i) Near-RT RIC, deployed closer to the edge and RAN nodes to perform near-real-time control loops with a periodicity between 10 ms and 1000 ms, supporting xApps with custom control logic to perform radio resource management; and the (ii) Non Real-Time RAN Intelligent Controller (Non-RT RIC), deployed as a component of the MNO's SMO framework to perform non-real-time control loops longer than 1 s, managing Machine Learning (ML) models and supporting rApps with custom control logic to dictate the long-term behavior of the network [12]. The RICs communicate via their A1 interface, which the Non-RT RIC uses to deploy policies that guide the Near-RT RIC optimization goals.

Fig. 3 depicts the O-RAN architecture specified by the O-RAN Alliance and shows the interplay between the entities discussed in this section. Throughout the rest of this tutorial, we have several diagrams showing logical and practical components of the O-RAN ecosystem, as well as their interfaces and interactions. To make it easier for the reader to understand the interactions between O-RAN entities and their locations within the O-RAN ecosystem, we adopt a color code where: yellow refers to E2 Nodes and the RAN, blue refers to the Near-RT RIC and its components, green refers to the Non-RT RIC, purple refers to the SMO, pink refers to Docker containers, and gray refers to the underlying software or hardware infrastructure supporting the O-RAN components.

B. Near-RT RIC Purpose and Interactions

The Near-RT RIC is the O-RAN entity responsible for providing near-real-time RAN orchestration and network automation [35]. Its primary purpose is to host and facilitate the operation of external applications, the xApps, for running near-real-time closed-control loops to monitor, analyze, and optimize network parameters for achieving desired network behavior

and performance. Arguably, the Near-RT RIC operates in a similar fashion to ONOS [36]. In the context of Software-Defined Networking (SDN) for transport networks. The Near-RT RIC has received much attention and contributions from the members of the O-RAN Alliance, being one of the most complete and mature software components provided by the OSC. The Near-RT RIC interacts with other O-RAN entities through southbound and northbound interfaces to leverage their information and capabilities for managing the RAN, as shown in Fig. 3. In the following, we detail these interactions.

- **E2 Nodes:** The Near-RT RIC interacts with RAN components, e.g., the disaggregated O-CU-CP, O-CU-UP, O-DU and O-RU, or the monolithic O-gNodeB and O-eNodeB. It collects real-time measurements and data from these nodes to monitor network performance, traffic load, interference levels, and other relevant metrics. The Near-RT RIC also communicates with E2 Nodes to configure and adjust different RAN parameters, e.g., transmit power, antenna settings, modulation schemes, and scheduling algorithms, based on the decisions made by the xApps.
- **SMO:** The Near-RT RIC interacts with the SMO responsible for the overall management of the network and services. The SMO provides high-level control and coordination functions, and the Near-RT RIC acts as an extension to this system by offering near-real-time optimization and intelligence capabilities within the RAN.
- **Non-RT RIC:** The Near-RT RIC interacts with the Non-RT RIC that defines and enforces policies, quality of service requirements, and regulatory constraints. The Near-RT RIC exchanges information and aligns its decision-making process with the policies described by this entity to ensure that network optimizations and resource allocations are in compliance with the established rules.

The interactions between the Near-RT RIC and other O-RAN entities to exchange network KPMs, control information, and system settings create a collaborative, distributed ecosystem that enables near-real-time programmability, automation, and intelligence within the RAN.

C. Different O-RAN Flavors and Near-RT RICs

The O-RAN specifications contain the requirements, capabilities, and interfaces for O-RAN entities, leaving the implementation details to the discretion of vendors or system integrators. In addition, the O-RAN Alliance partnered with the Linux Foundation to create the OSC [13] and provide a fully operational, open-source reference implementation of O-RAN entities (discussed in the following subsection) to demonstrate their capabilities and serve as the starting point for commercial products. These approaches fostered the creation of different O-RAN flavors, such as the FlexRIC from OAI [37], the SD-RAN from ONF [38], and the dRAX from Accelleran [39]. While possessing interoperable external interfaces toward E2 Nodes, the different O-RAN flavors adopt different design choices for their RICs, internal components, xApps, and Service Models (SMs), making them not interoperable. For example, the OSC and SD-RAN adopt a microservice philosophy, where each capability of the Near-RT RIC is

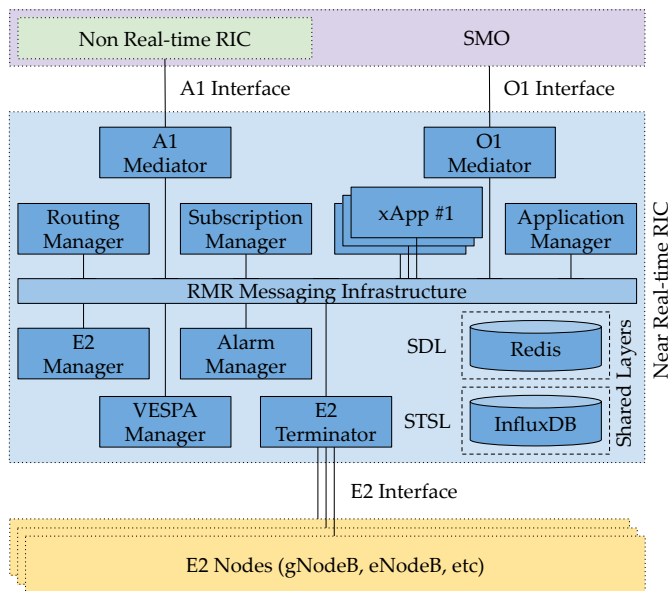


Fig. 4: The internal components of the Near-RT RIC and how they communicate with other O-RAN entities using different interfaces. The xApps reside inside the Near-RT RIC and interact with its components to leverage their capabilities to perform actions, e.g., subscribe to information from the RAN.

implemented through a discrete component. In contrast, the FlexRIC and dRAX tend to be more monolithic and/or provide different communication interfaces not standardized by the O-RAN specifications, such as the OAI E42 interface for direct communication between xApps and E2 Nodes [40]. For a comprehensive comparison between O-RAN flavors and their RICs, we refer the reader to other works that evaluated their performance and features [41] [42].

Without loss of generality, we will focus on the O-RAN flavor from the OSC throughout the rest of this tutorial. This choice is motivated by its status as the first O-RAN implementation available in the literature, its widespread adoption in academia and industry, and the extensive body of research and solutions developed based on it. By leveraging the OSC implementation, we aim to benefit a broader audience and maximize the impact of this tutorial. While other O-RAN flavors may have different design choices and APIs, the concepts and lessons introduced through this tutorial remain valuable and applicable across other O-RAN implementations.

D. Near-RT RIC Components

In a similar fashion to how the O-RAN ecosystem is composed of several entities, the Near-RT RIC from the OSC is implemented as a collection of microservices that work in unison to allow MNOs to manage their RANs in near-real time. Fig. 4 illustrates the components of the Near-RT RIC, each of which provides specific functionality to xApps or supports their operation inside the Near-RT RIC cluster. We describe each component of the Near-RT RIC below.

RIC Message Router (RMR): implements the internal messaging infrastructure for communication between all Near-

RT RIC components, including management components, interface terminators, and xApps. The RMR library allows applications to send/receive messages to/from other applications without information about their IP, location, or the underlying transport mechanism. We detail RMR later in Section VI-A.

Routing Manager (RtMgr): manages RMR routes in the Near-RT RIC. It is responsible for creating and distributing RMR routing policies to Near-RT RIC components and xApps.

Application Manager (AppMgr): manages the xApp deployment and lifecycle in the Near-RT RIC. It is responsible for (un)installing xApps and notifying other Near-RT RIC components about the current set of xApps. We detail how the xApp developer can interact with it later in Section V-C.

Subscription Manager (SubMgr): manages subscriptions from xApps to E2 Nodes. It is responsible for creating routes and abstracting the interaction with the E2 Nodes. We detail how xApps can subscribe to E2 Nodes later in Section VII-C.

E2 Manager (E2Mgr): manages the E2 Nodes registered with the Near-RT RIC. It sets up E2 Nodes with the Near-RT RIC, monitors their health, and informs any issues to xApps.

E2 Terminator (E2Term): intermediates the communication between xApps (or other Near-RT RIC components) and E2 Nodes, acting as a translation layer between the internal RMR messaging infrastructure used inside the Near-RT RIC and the external SCTP protocol used by E2 Nodes.

Shared Layers: provide a lightweight, high-speed interface for managing data storage in the Near-RT RIC. Shared Data Layer (SDL) and Shared Time Series Layer (STSL) offer stateless storage, abstracting the underlying database technology from the business. SDL stores relational data, while STSL stores time series data. We detail how xApps can leverage the Shared Layers for persistent storage in Section VI-C.

VESPA Manager (VesMgr): starts, configures, and uses the Virtual Event Streaming (VES) Agent to adapt the collection of internal statistics using Prometheus to scrape metrics from Near-RT RIC components and xApps, and forward them to an SMO, e.g., ONAP, or another VES Collector.

Alarm Manager: manages alarms from xApps and Near-RT RIC components, interfacing with the Prometheus Alert Manager to post the alarms as alerts. It also de-duplicates, silences, inhibits alerts, and routing them to the VES Agent.

A1 Mediator: receives policies from the Non-RT RIC and forwards them to xApps via RMR. A policy contains high-level directive that serves to steer the behavior of xApps. We detail how xApps can consume policies in Section VI-B.

O1 Mediator: exposes metrics and information about the status of the Near-RT RIC components, registered E2 Nodes, and xApps to the management entities in the SMO.

This section discussed the principles behind O-RAN, its specifications, and the concepts and motivations therein. However, to develop xApps, we must go one step further into the O-RAN ecosystem, exploring some implementation details and supporting technologies underpinning the Near-RT RIC and the xApp themselves. The deployment and installation of the Near-RT RIC, as well as the system requirements for running it, will differ depending on the O-RAN flavor and the scale of operation. We followed the system requirements from the

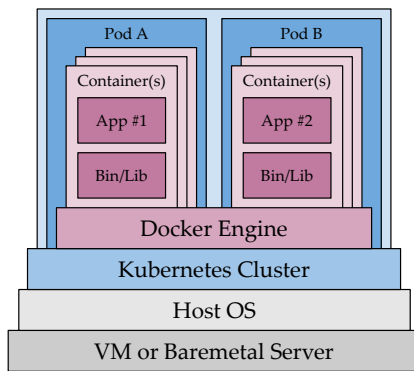


Fig. 5: The layers of abstraction for running xApps, showing their Kubernetes pods, which include one or more Docker containers with the complete environment to run the xApp source code, and the underlying Near-RT RIC cluster, running on the host OS of a virtual machine of baremetal server.

official OSC documentation [43] to create a development environment and the new Near-RT RIC installation guidelines [44]. We refer the reader to these references for information on system specifications and installation instructions.

III. TECHNICAL BACKGROUND IN O-RAN

In this section, we review technical matters related to the realization of O-RAN entities and xApps. First, we discuss the O-RAN implementation and design choices taken by the OSC that influence the development of xApps. Then, we present the cloud technologies supporting xApps in a Near-RT RIC. Next, we introduce the OSC's resources for facilitating xApp development. Finally, we overview existing first- and third-party xApps to demonstrate their capabilities and inspire readers to use them as a starting point to develop their xApps.

A. O-RAN Implementation

Each O-RAN flavor can take distinct design choices and technical approaches, which leads to a lack of interoperability across O-RAN stacks [41]. For example, the OSC uses the RMR messaging protocol for communication between Near-RT RIC components and xApps [12], whereas the SD-RAN employs the gRPC protocol [38]. In theory, both protocols perform the same task, but their implementations are widely different and, hence, incompatible in practice. These implementation differences affect xApps, as xApps developed for a given O-RAN flavor will not necessarily be compatible with others, e.g., xApps for OSC deployments are incompatible with SD-RAN and vice-versa. Consequently, the xApp developer must decide which O-RAN flavor they will cater to. In the remainder of this tutorial, we will instruct the reader on creating xApps for the O-RAN flavor from the OSC, providing technical background and practical examples.

The OSC is an open-source community, academia, industry, and open-source developers contribute to developing and improving an open-source O-RAN implementation [12]–[14]. It provides an open-source implementation of the Near-RT RIC, following a microservice architecture based on cloud technologies [14]. The Near-RT RIC is a specialized Kubernetes

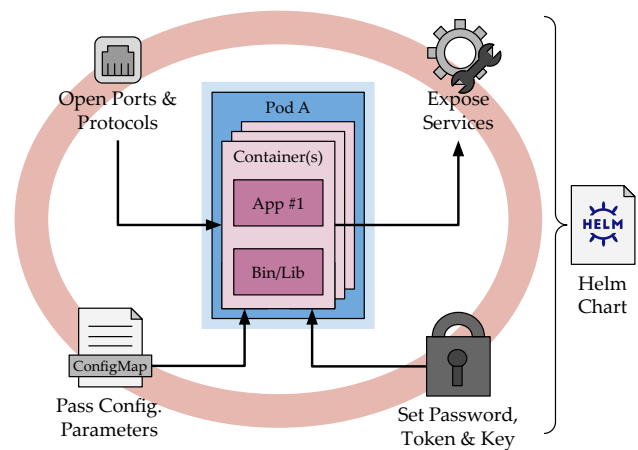


Fig. 6: Examples of the different aspects that are automated through the use of Helm Charts, facilitating the deployment of multiple pods and their containers in a Near-RT RIC.

cluster that adopts the Docker container engine [12]. In this way, each Near-RT RIC component is an isolated Kubernetes pod running one or more Docker containers, whose resources, ports, and interfaces are described using Helm Charts (we detail these cloud technologies later in this section). In the OSC, the xApps are also cloud-native microservices, i.e., Kubernetes pods based on Docker containers, running inside the Near-RT RIC cluster, as illustrated previously in Fig. 4. Thus, it is essential for the xApp developer to understand the fundamentals behind these cloud technologies and use them accordingly. Moreover, the OSC created an xApp SDK [45] to facilitate the development of xApps in different languages and interfacing with the Near-RT RIC components, which we will leverage throughout this tutorial.

B. Cloud Technologies Supporting xApps

A common challenge in software distribution is ensuring that your application will run in the environment of an interested party, such as MNOs, who may employ widely different software and hardware platforms. To this end, Docker containers are lightweight virtual environments that include everything required to run a given application, i.e., source code, dependencies, libraries, and settings [46]. These applications and their virtual environments can be conveniently packaged into a single file, a Docker container image [12], making them portable and facilitating their distribution. While sharing the same kernel with the host OS of the underlying server or virtual machine (a common practice in cloud environments), the Docker containers are isolated from one another, allowing them to have different OSs, libraries, and versions, improving security for running applications from third-parties and mitigating potential library versioning conflicts. We detail the process for creating Docker container images supporting xApps later in Section V-B.

A second challenge arises when managing the deployment of several containers on a cluster of servers, such as the components of the Near-RT RIC and the xApps from various developers, which can have very different requirements

in terms of computing resources, ports, and interfaces. To this end, Kubernetes serves as an orchestration platform for managing the lifecycle, health, communication, and storage of container deployments [47]. It groups one or more containers working together to provide a service or run an application into an independent and isolated pod. Fig. 5 illustrates an abstract example of an xApp pod composed of multiple containers. Kubernetes can automatically restart pods if there are errors, scale their computational resources according to demand, or replicate them entirely for load balancing across servers. The Near-RT RIC Kubernetes cluster separates its pods into different namespaces: (i) *ricinfra*, containing pods that provide the supporting infrastructure for the operation of the Kubernetes cluster; (ii) *ricplt*, containing pods of the O-RAN components in the Near-RT RIC platform, e.g., *AppMgr*, *RtMgr*, *SubMgr*, etc.; and (iii) *ricxapp*, containing all the installed xApp pods. We detail the mechanism for deploying xApps pods in a Near-RT RIC cluster later in Section V-C.

As the size of your Kubernetes deployment grows, so does the complexity of managing it. Since applications based on a microservice architecture can span many containers or pods working together, manually (i) downloading multiple Docker container images to create pods; (ii) passing configuration parameters to each container, (iii) defining their open ports and protocols to describe how their services are exposed, and (iv) managing their authentication credentials through passwords and access tokens can quickly become intractable. To this end, Helm automates the creation, description, configuration, and deployment of Kubernetes pods [48]. It combines multiple configuration files that define different properties and requirements of Kubernetes pods and their containers into a single reusable package, a Helm Chart. Fig. 6 illustrates the Helm Chart for an abstract example of an xApp pod leveraging different features from Kubernetes. The Helm Charts also contain a "value.yaml" file that allows the user to customize the configuration parameters of pods before their deployment on a Kubernetes cluster. While the OSC provides Helm Charts for automating the deployment of the Near-RT RIC components, the xApp developer must provide their own. The creation of Helm Charts for xApps is partially automated using the *dms_cli* tool provided by the OSC, detailed later in Section V-C.

C. OSC's xApp Development Resources

The OSC provides open-source reference implementations of (i) O-RAN entities and their internal components, e.g., the SMO, the Non- and Near-RT RICs; (ii) endpoints for O-RAN interfaces, e.g., A1, O1, and E2; and (iii) simulators for modeling and testing the behavior of nodes using the interfaces mentioned above, e.g., *AlSim*, *OlSim*, and the *E2Sim*. In addition, to facilitate the development of xApps, the OSC provides an xApp SDK [45], which contains libraries to facilitate and abstract the communication with Near-RT RIC components for leveraging their capabilities as part of the xApp's business logic to manage RANs and tools to help the xApp development cycle. These tools include (i) the ASN.1 Compiler, which automatically generates C++ bindings for

E2 Nodes based on SMs (detailed in Section VII); (ii) the *dms_cli*, for managing the lifecycle of xApps in a Near-RT RIC cluster (detailed in Section V); and (iii) the xApp Frameworks, which contain the set of libraries listed above, as well as their bindings in different programming languages.

The xApp Frameworks streamline the xApp development process, allowing the xApp developers to use the same libraries, APIs, and design philosophy to create xApps in Python, C++, Go, or Rust. In addition, they abstract a series of tasks related to the (de-)registration of xApp with the Near-RT RIC components (detailed in Section VI) and the subscription to E2 Nodes (detailed in Section VII), which considerably simplifies the xApp development process. Throughout the remainder of this tutorial, we will use the Python xApp Framework [63] to demonstrate the development of xApps due to Python's widespread adoption and its smooth learning curve for new software developers. We introduce the libraries available to xApps in Section VI and explain how to use the *E2Sim* to test the interactions between xApps and E2 Nodes in Section VII.

D. Existing First- and Third-party xApps

The number of first- and third-party xApps has increased considerably since the first OSC release in 2019, and it is expected to grow even more as the interest and investments in O-RAN continue to rise [64]. In this context, first-party refers to xApps provided and supported by OSC, FlexRIC, and SD-RAN initiatives. For example, the OSC provides a few xApps out of the box, e.g., the Admission Control (AC) for controlling the maximum number of UEs admitted on the RAN, the KPM Monitoring, for obtaining period reports on UEs and gNodeB metrics, the Anomaly Detection (AD), for inspecting stored performance metrics and identifying anomalous UEs, and the Traffic Steering (TS), for managing the handover of UEs between different gNodeBs to optimize network performance. Meanwhile, the SD-RAN provides their version of a KPM Monitoring xApp, as well as the Mobile Handover (MHO), also for managing handovers, the Mobility Load Balancing (MLB), for controlling the gNodeB's cell individual offset according to its load, and the RAN Slice Manager (RSM), for creating RAN slices and configuring their resource allocation. For more detailed information about the specific use cases supported by O-RAN flavors and the minimal running setups for the first-party xApps mentioned above, we refer the reader to the official OSC [65] and the SD-RAN documentation [38]. Conversely, third-party refers to xApps designed by the growing community of open-source developers from industry and academia. To understand the capabilities of existing xApps, we conducted a literature review to identify examples of third-party xApps for the OSC. We classified them according to their category, objective, evaluation approach, support for ML, and whether they provide implementation details to help xApp developers use or recreate them. Table I summarizes the results of our review.

We grouped these works into distinct categories related to their operation: RAN Slicing, addressing aspects related to the control and optimization of RAN slices [49]–[51]; Security,

TABLE I: Qualitative classification of the current third-party xApps for the OSC available in the literature.

Works	Category	Objective	Evaluation	ML Support	Development Details
Johnson, et al. [49]	RAN Slicing	Policy driven RAN slice control	Emulation: OSC & srsLTE	–	–
Bonati, et al. [50]		RAN slice scheduling optimization	Emulation: Colosseum	DRL	–
Zhang, et al. [51]		RAN slice power and resource allocation	Simulation: Matlab 5G Toolbox	Federated DRL	–
Huff, et al. [52]	Security and Fault-tolerance	Fault-tolerance	Emulation: OSC	–	–
Wen, et al. [53]		Telemetry for security analysis	Theoretical	–	–
Lee, et al. [54]	ML	Online training environment	Emulation: OSC & srsLTE	RL	–
Iturria-Rivera, et al. [55]	Resource Allocation	Power and radio resource allocation	Theoretical	Multi-agent DRL	–
Mungari [56]		Radio resource management	Emulation: OSC & OAI	RL	–
Rego, et al. [57]	Spectrum Sensing		Emulation: OSC	–	✓
Orhan, et al. [58]	Data Traffic Management	Connection management optimization	Theoretical	DRL/GNN	–
Kouchaki, et al. [21]		QoE maximization	Emulation: OSC	RL	✓
Huang, et al. [59]		Throughput maximization	Theoretical	DRL	–
Agarwal, et al. [60]		QoE enhancement function	Simulation	–	–
Lacava, et al. [61]		Traffic Steering intelligent handover	Simulation: ns-O-RAN	DRL	✓
Alvirad, et al. [62]		Admission control of UEs	Simulation: ns3 LTE	DRL	–

considering fault-tolerance [52] and the streaming telemetry information [53]; ML with the creation of an online training reference workflow [54]; Resource allocation, considering radio [55], and power resources [56]; Spectral Sensing based on ML [57]; and Data Traffic Management, with several works managing the data traffic of for UEs [21], [58]–[62]. Regarding their evaluation strategies, of the 15 works we identified in our literature review, only four provided robust analytical models. However, these theoretical works provided limited numerical results without an empirical evaluation to demonstrate their solutions. Four other works have developed and evaluated prototypes in simulated environments, such as ns-3 and Matlab 5G Toolbox. The remaining seven works presented prototypes and evaluated their proposals using emulated environments, leveraging testbeds, such as Colosseum, and open-source radio stacks, e.g., srsLTE and OpenAirInterface (OAI). While these works effectively demonstrated their contributions experimentally, the vast majority did not make their xApps available or provide any development guidelines, inhibiting the reproduction of their results by the broader research community.

The official support of ML on xApps was only recently introduced by the OSC in December 2022, with the initial release of the AI/ML Framework (AIMLFW) [66], discussed later in Section X. However, there is a vast number of works in the literature that predate the release of the AIMLFW and present xApps with support for ML. We can observe a wide range of ML solutions in Table I, from DRL, to multi-agent DRL, Federated DRL, and Graph Neural Network (GNN). These works accomplish this feat by proposing a myriad of custom solutions using homebrewed software that incorporates ML into their xApps. While effective in their specific use cases, it is very challenging to support, distribute, and extend these non-standard solutions, which limits their applicability. Finally, it is worth mentioning that only three works provided development details about their solutions, giving the reader some understanding of the inner workings of their solutions. The lack of papers with implementation details related to the development of xApps hinders the reproduction of results, the extension of existing prototypes, and the creation of new solutions. This tutorial paper addresses this gap by providing comprehensive guidelines, from theory to practice, aiding

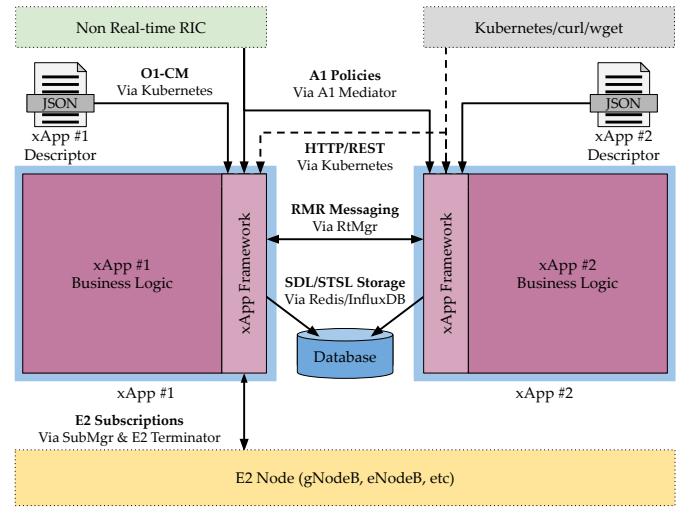


Fig. 7: The xApps can leverage capabilities from and interact with several entities of the O-RAN ecosystem and components of the Near-RT RIC through different APIs and interfaces.

xApp developers to design, create, and evaluate their xApps in realistic end-to-end environments.

IV. xAPP DESIGN: DEFINING YOUR APPLICATION

In this section, we overview the prerequisites for developing xApps, and delve on their architecture and interfaces. We examine the xApp descriptor and schema files, providing examples of how to define xApps pods and ports, pass control parameters, and configure its different interfaces.

A. Prerequisites

Developing xApps differs from the traditional implementation of programs for general-purpose operational systems, such as Linux and Windows. Unlike traditional programs, which often run standalone interacting with a single kernel via system calls, the xApps reside within the Near-RT RIC and operate as part of a distributed system composed of Near-RT RIC components and other xApps. This unique execution environment makes an xApp inseparable from the

Near-RT RIC and requires it to interact with Near-RT RIC components for leveraging their capabilities to perform actions through well-defined APIs and protocols, e.g., subscribe to information from the RAN via the E2 interface. Consequently, the xApp developer not only needs to be knowledgeable in a programming language (preferably one with an xApp Framework library, e.g., Python, C++, Go, or Rust) but also in microservices and cloud technologies, e.g., Docker, Kubernetes, and Helm. Familiarity with the Near-RT RIC as the execution environment for the xApps is also required, which demands an understanding of the O-RAN architecture and the capabilities, APIs, and interfaces of Near-RT RIC components.

In addition to the technical background and implementation skills, the xApp developer must have a solid understanding of mobile networking concepts and the intended use case for their xApps, e.g., network optimization or resource management. This initial conceptualization is essential for determining the xApp's objectives, defining the scope of its operation, and identifying the interfaces and APIs it will leverage ahead of the implementation of its business logic. Furthermore, it is strongly recommended that xApp developers create or utilize an O-RAN development environment to test how their xApp will operate in conjunction with other O-RAN entities. For more information about interactive online resources and remote access testbeds, we refer the reader to [25].

B. Architecture

From a functional perspective, xApps are discrete microservices that implement well-defined business logic to manage RANs [67]. This logic can involve collecting and processing data from E2 Nodes, calculating metrics to generate reports or trigger alarms, and controlling different aspects of the E2 Nodes according to a given algorithm [12]. From an implementation perspective, xApps are Kubernetes pods running inside the Near-RT RIC cluster, each of which may contain one or more Docker containers, as discussed in Section III-A. The application running in the Docker containers implements the business logic of the xApp, leveraging an xApp Framework library available in Python, C++, Go, or Rust programming languages, which provides xApps with a number of APIs to exploit the capabilities offered by the different components of the Near-RT RIC, as discussed in Section II-D.

Fig. 7 illustrates the xApp architecture and its interactions with other xApps and entities of the O-RAN ecosystem, intermediated through different components of the Near-RT RIC. In the following, we briefly introduce the different APIs available to the xApps to give the reader a high-level overview of the xApps' functionality. We will discuss each API in-depth later in Section VI, where we detail the purpose and concepts behind their operation and exemplify their utilization.

O1-CM Configuration: During startup, the xApp descriptor provides xApps with initial configuration parameters to interact with the Near-RT RIC and optional parameters that can be used to parameterize their operation [67]. The content of the xApp descriptor is loaded in the Kubernetes pod as a ConfigMap, allowing Near-RT RIC users, i.e., system administrators and network operators, to modify the optional parameters during runtime.

RMR Messaging: It allows xApps to communicate with one another and with components of the Near-RT RIC through a low-latency messaging library. RMR uses a publish-subscribe paradigm, enabling xApps to be oblivious to the IP addresses of other Kubernetes pods and exchange messages based on message types [68]. It also serves as the API in which the xApps receive AI policies from the Non-RT RIC through the AI Mediator.

SDL/STSL Storage: It provides xApps access to the shared database within the Near-RT RIC, facilitating read/write operations to persistent storage while abstracting the specific implementation details of the underlying database solution. It also handles the authentication and authorization processes to access the database, ensuring that xApps remain portable and stateless [69].

HTTP/REST: It provides xApps with REST callbacks for handling HTTP requests, allowing xApp developers to customize the response to Kubernetes' liveness and readiness probes according to their xApp's requirements [67]. One can also create REST callbacks to expose internal information about the xApp's business logic and respond to external commands and parameters, allowing users to interact with xApps directly via HTTP.

E2 Subscription: It allows xApps to obtain information from the RAN and control its operation. The xApps can subscribe to metrics and updates from a given set of E2 Nodes for post-processing or data analytics and control of the E2 Nodes according to their business logic by triggering or passing parameters to the supported RAN functions exposed via their SMs [70].

An xApp only requires a valid descriptor provided via the O1-CM to operate, as it contains the required initial configuration to enable the xApp's deployment and interaction with additional Near-RT RIC components, if demanded. In addition, the xApp descriptor instructs the AppMgr on how to install the xApp, which involves fetching the Docker images from an accessible Docker registry, configuring the Kubernetes pod, and notifying other Near-RT RIC components of the creation of a new xApp, e.g., RtMgr and SubMgr, to allow the new xApp to leverage their capabilities [67]. We will detail the xApp lifecycle and how the AppMgr uses the xApp descriptor further in Section V-A. The other APIs listed previously in this section are optional, e.g., SDL/STSL, RMR, E2 Subscriptions, etc., meaning that xApp developers can focus on learning and implementing only the interfaces needed to accomplish their intended business logic. In the following, we detail how to design and define xApps, specifying container images, opening ports, and configuring the APIs mentioned above.

C. Configuration

As part of the xApp development cycle, the xApp developers must design their applications according to the intended business logic, and define them through the creation of xApp descriptor and schema files. The former is a JSON file that instructs the Near-RT RIC to deploy the given xApp, specifying (i) what is the name and version of the xApp, (ii) what Docker container images it requires, and the locations of

```

1 {
2   "name": "example_xapp",
3   "version": "1.0.0",
4   "vendor": "example_vendor",
5   "containers": [
6     // Configures Containers and Images.
7     // Detailed in Sec. IV-C1.
8   ],
9   "rmr": {
10    // Configures RMR Messages.
11    // Detailed in Sec. IV-C2.
12  },
13  "messaging": {
14    "ports": [
15      // Configures Ports per Container.
16      // Detailed in Sec. IV-C3.
17    ]
18  },
19  "controls": {
20    // Optional Control Parameters.
21    // Detailed in Sec. IV-C4.
22  }
23 }

```

Listing 1: xApp Descriptor Template.

their Docker Registries, (iii) which ports must be open in each container, (iv) what RMR messages the xApp will publish and subscribe, (v) what A1 policies it will consume, and (vi) what optional parameters the user can control. The latter is a JSON schema file that the Near-RT RIC uses to verify and validate the content of the xApp descriptor before triggering the xApp deployment process, a process which we will detail further in Section V-A. Assuming all the required Docker Registries and images are reachable, an xApp developer only needs to share their xApp descriptor and schema files to distribute their application [67]. However, there are current discussions and research efforts toward developing a store or marketplace to distribute xApps [7]. Therefore, the xApp distribution process might change in the future.

The xApp descriptor follows the structure shown in Listing 1, which contains the name, version, and vendor of the xApp to be deployed in the Near-RT RIC. It is worth mentioning that the name and version are required parameters that serve to identify the xApp inside the Near-RT RIC and generate a unique name for the xApp's Kubernetes pod, detailed further in Section V-C. In the following, we first detail the subsequent sections of the xApp descriptor, shown in Listing 1, and then examine the xApp schema.

1) *Containers and Images*: This mandatory xApp descriptor section defines the containers that compose the xApp Kubernetes pod, as shown in Listing 2. Each xApp contains at least one Docker container, which can come from different images in different Docker Registries. For each container, the xApp developer must specify (i) its name, which will serve as a unique identifier used internally for opening ports and routing RMR messages to containers, and (ii) the location of its Docker image, i.e., the URL of the Docker Registry, the image name and its tag, which the AppMgr will use to pull the image locally and instantiate the container. In addition to

```

1 ...
2   "containers": [
3     {
4       "name": "example_container_1",
5       "image": {
6         "registry": "example.registry.com",
7         "name": "example_image_1",
8         "tag": "1.0.0"
9       }
10    },
11    {
12      "name": "example_container_2",
13      "image": {
14        "registry": "example.registry.com",
15        "name": "example_image_2",
16        "tag": "1.0.0"
17      },
18      "resources": {
19        "requests": {
20          "cpu": "1",
21          "memory": "64Mi"
22        },
23        "limits": {
24          "cpu": "2",
25          "memory": "128Mi"
26        }
27      }
28    }
29  ],
30  ...

```

Listing 2: Section for configuring containers and images.

the aforementioned required parameters, the xApp developer can specify the minimum computing and memory resources each container requires to run and limit the maximum resource utilization. These optional parameters ensure the xApp has access to the resources it requires to run and cap the resource utilization on the Near-RT RIC cluster.

2) *RMR Routing and Configuration*: This optional xApp descriptor section defines the RMR messages that the xApp Kubernetes pod will transmit and receive, and the A1 policies it will consume (all of which are optional), as shown in Listing 3. The RMR messaging operates in a publish-subscribe paradigm. If leveraging the RMR interface or consuming A1 policies, the xApp developer must specify the message types that their xApps will consume and the message types they will produce to use RMR. The RtMgr will use this information for creating routing tables and propagating them to other Near-RT RIC components and xApps. Some message types are required to avail from certain functionality from the Near-RT RIC components, e.g., the RIC_HEALTH_CHECK_REQ and RIC_HEALTH_CHECK_RESP are required for reacting to RMR health checks from the RtMgr. We will discuss in-depth the RMR functionality and explain the essential message types for the operation of xApps later in Section VI-A. If required for a particular use case or deployment, the xApp developer can customize the transport protocol and port for RMR's operation, the maximum message buffer size, and the number of threads listening to incoming messages. Furthermore, the xApp developer can specify a list of policy IDs their xApp

```

1  ...
2  "rmr": {
3    "txMessages": [
4      "A1_POLICY_RESP",
5      "A1_POLICY_QUERY",
6      "RIC_HEALTH_CHECK_RESP"
7    ],
8    "rxMessages": [
9      "RIC_INDICATION",
10     "A1_POLICY_REQ",
11     "RIC_HEALTH_CHECK_REQ"
12   ],
13   "protPort": "tcp:4560",
14   "maxSize": 2072,
15   "numWorkers": 1
16   "policies": [1]
17 },
18 ...

```

Listing 3: Section for configuring RMR message routing.

will consume. We will discuss the A1 policies and how xApps can avail from them later in Section VI.

3) *Ports and Services*: This optional xApp descriptor section defines the open ports and messages routed to each container that composes the xApp Kubernetes pods, as shown in Listing 4. Each container comprising the xApp Kubernetes pod can have different communication interfaces and avail of distinct functionality from Near-RT RIC components, which the xApp developer can specify by creating port definitions that must contain an identifying name, the target container's name, the port number, and a brief description. The AppMgr uses this information to trigger Kubernetes for creating a service port mapped to the corresponding container. For containers using HTTP/REST, port 8080 must be open to support reacting to external input from users of the Near-RT RIC and Kubernetes' liveness and readiness probes. For a Near-RT RIC cluster using the default configuration, each container leveraging RMR or consuming A1 policies must open (i) port 4061 to receive dynamic routing tables from the RtMgr and learn where to route messages, and (ii) port 4060 to receive messages from Near-RT RIC components and other xApps. The xApp developer has fine-grained control over the RMR message routing inside their xApp pod and must specify which messages will be produced and consumed per container.

4) *Optional Control Parameters*: This optional xApp descriptor section defines additional control parameters to customize the operation of the xApp, as shown in Listing 5. The xApp developer can include an arbitrary number of xApp-specific parameters, ranging from boolean values, strings, integer (or float) numbers, and arrays to more complex JSON objects comprised of a combination of the data types listed above. The control parameters listed in Listing 5 serve to parameterize the subscription to E2 Nodes, which we will explain in detail further in Section VII. The xApp descriptor file is loaded into the Kubernetes pod as a ConfigMap, which mounts the xApp descriptor as a JSON file inside the containers' directory tree. The location of the xApp descriptor is defined on the XAPP_DESCRIPTOR_PATH environment

```

1  ...
2  "ports": [
3    {
4      "name": "http",
5      "container": "example_container_1",
6      "port": 8080,
7      "description": "HTTP service port"
8    },
9    {
10     "name": "rmrroute",
11     "container": "example_container_2",
12     "port": 4561,
13     "description": "RMR route port"
14   },
15   {
16     "name": "rmrdata",
17     "container": "example_container_2",
18     "port": 4560,
19     "rxMessages": [
20       "RIC_INDICATION",
21       "A1_POLICY_REQ"
22     ],
23     "txMessages": [
24       "A1_POLICY_RESP",
25       "A1_POLICY_QUERY"
26     ],
27     "policies": [1],
28     "description": "RMR data port"
29   }
30 ]
31 ...

```

Listing 4: Section for configuring ports and services.

```

1  ...
2  "controls": {
3    "rmr_routing_needed": false,
4    "meid": "gnb123456",
5    "ran_function_id": 1231,
6    "action_definition": [
7      11, 12, 13, 14, 15
8    ],
9    "action_id": 1,
10   "action_type": "policy",
11   "subsequent_action": {
12     "subsequent_action_type": "continue",
13     "time_to_wait": "w10ms"
14   }
15 },
16 ...

```

Listing 5: Section for configuring optional control parameters.

variable, which the xApp can use to locate and load its content accordingly. However, the xApp Frameworks automate these tasks and make the content of the xApp descriptor readily available for the xApp developer to use as part of their business logic. The parameters specified in this xApp descriptor section can be updated by the user during runtime by editing the ConfigMap of the xApp Kubernetes pod (detailed in the next section), which the xApp developer can use to customize certain aspects of their xApp.

5) *xApp Schema File*: It is a JSON schema that annotates and validates the xApp descriptor JSON file. Before triggering

```

1 {
2   "$schema": "http://
   ↪ json-schema.org/draft-07/schema#",
3   "$id": "#/controls",
4   "type": "object",
5   "title": "Controls Section Schema",
6   "required": [
7     // List of required control parameters
8   ],
9   "properties": {
10    // Properties of the required parameters
11  }
12 }

```

Listing 6: xApp Schema Template.

Kubernetes to instantiate the xApp pod, the AppMgr verifies the content of the xApp descriptor against the xApp schema to ensure the descriptor contains all the required parameters for deploying the given xApp. The AppMgr comes preloaded with JSON schemas to verify most of the required and optional sections of the xApp descriptor, e.g., *containers*, *rmr*, and *messaging* sections. The only exception is the *controls* section, which contains customized optional control parameters. If the xApp descriptor has an empty *controls* section, the xApp schema is entirely optional. However, if the xApp descriptor contains a non-empty *controls* section, the xApp developer must provide the AppMgr with a custom schema file to verify and validate this section, as shown in Listing 6, or that will cause the xApp deployment to fail. The xApp schema contains the IETF JSON Schema version, the ID of the xApp descriptor section that this schema will verify (*controls*), a list of the required control parameters that the xApp descriptor must contain, followed by a list of their properties. Should the xApp developer decide to include any required control parameters, they must define their properties, as shown in Listing 7, specifying their ID (the URI from the root of the descriptor), their data type, their default values, descriptive titles, and optionally, examples of possible values. The xApp schema can also contain no required control parameters and properties, making all parameters optional, as shown in Listing 6. For completeness, we refer the reader to our online repository [29], where we include the entire xApp descriptor and schema files used as examples in this section.

V. XAPP MANAGEMENT: CONTROLLING ITS LIFECYCLE

In this section, we detail the xApp lifecycle, how to create and publish Docker Images from the xApp's source code, and how to interact with the Near-RT RIC to manage xApps, teaching xApp developers to onboard, install, query, and uninstall xApps in their O-RAN development environment.

A. The xApp Lifecycle

From an implementation perspective, xApps are specialized applications leveraging xApp Framework libraries, distributed to MNOs as universal Docker images, and instantiated as Kubernetes pods running on a Near-RT RIC cluster [67]. As such, there are several steps in the xApp lifecycle, from the

```

1   ...
2   "required": [
3     "meid",
4     "ran_function_id",
5     ...
6   ],
7   "properties": {
8     "meid": {
9       "$id": "#/properties/controls/items/
   ↪ properties/meid",
10      "type": "string",
11      "default": "gnb123456",
12      "title": "E2 Node Managed Entity ID",
13      "examples": [
14        "gnbABCDEF", "enbMNOPQR"
15      ]
16    },
17    "ran_function_id": {
18      "$id": "#/properties/controls/items/
   ↪ properties/ran_function_id",
19      "type": "integer",
20      "title": "E2 Node RAN Function ID",
21      "default": "1231"
22    }
23  }
24   ...

```

Listing 7: Schema for defining required control parameters.

initial ideation and software development to container creation and publishing, and finally, to xApp deployment and execution inside a Near-RT RIC [12], as shown in Fig. 8. In theory, the role of the xApp developer would end after publishing their xApp in a Docker Registry and distributing their xApp's descriptor and schema files publicly or to the intended MNOs. However, in a practical setting, the xApp developer will most likely need to test, debug, and validate the xApp in their own O-RAN development environment, which can include, but is not limited to, a testing Near-RT RIC cluster for testing the deployment and operation of xApps, as well as a real or a simulated [71] E2 Node to evaluate the xApp's interaction with E2 Nodes for validating its business logic. Therefore, the xApp developer must know how to interact with the Near-RT RIC to manage xApps. In this tutorial, we cover the entire xApp development process and instruct the xApp developer throughout all steps of the xApp lifecycle, introduced below.

Source Code Developing: The xApp developer writes the source code that implements its intended business logic. We will go into detail about the implementation of xApps and their available APIs further in Section VI.

Docker Image Building: The xApp developer prepares a Dockerfile with instructions to build Docker image(s), specifying the complete environment to run the xApp source code, including directories and dependencies.

Docker Image Publishing: After creating Docker Image(s), the xApp developer pushes them to a (local or remote) Docker Registry so the Near-RT RIC can fetch the image(s) to create container(s) and instantiate the xApp.

Config File Sharing: With the location of the Docker image(s), i.e., the Docker Registry's URL, the image name, and its tag, the xApp developer includes them in the xApp

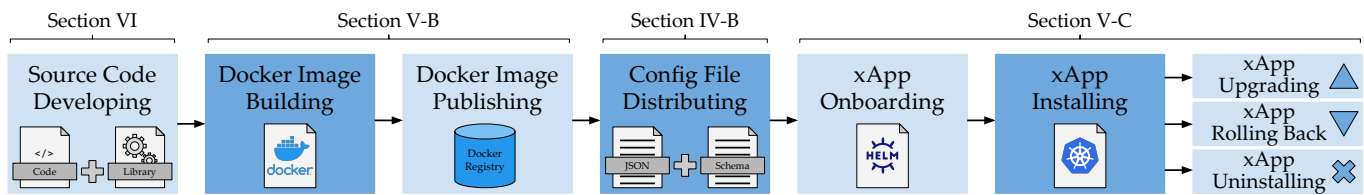


Fig. 8: The stages of the xApp lifecycle, from software development to container generation, configuration file sharing, and pod execution in a testing or production Near-RT RIC cluster. The xApp developer may need to repeat these steps several times during the xApp development cycle to test, debug, and validate their xApp, before their xApp is ready for public release.

descriptor, and shares both the descriptor and schema files with the intended MNOs to distribute the xApp.

xApp Onboarding: In possession of the xApp descriptor and schema files, the xApp developer (or the Near-RT RIC's users) onboards the xApp into the Near-RT RIC, generating Helm Charts stored in the Local Helm Chart Repository [72], which we detail later in this section.

xApp Installing: After the xApp is onboarded and its charts are in the Local Helm Chart Repository, the xApp developer (or the Near-RT RIC's users) can install the xApp, triggering the creation of its Docker container(s) and instantiation of the xApp pod, as well as the registration of the xApp with the *AppMgr* discussed later in Section VI.

xApp Upgrading/Rolling Back: Once the xApp pod is running, the xApp developer (or the Near-RT RIC's users) may upgrade it or roll it back to a different version. After onboarding the different xApp descriptor and schema files, upgrading (or rolling back) the xApp will uninstall its pod and subsequently install the newer (or previous) version of the xApp.

xApp Uninstalling: Once the xApp pod is running, the xApp developer (or the Near-RT RIC's users) may uninstall it, releasing its computational resources and de-registering it with the *AppMgr*, which terminates the xApp's subscriptions and RMR endpoints, discussed later in Section VI.

We discussed the design and distribution of xApp descriptor and schema files in the previous section, and will dive deep into the xApp source code development later in Section VI. In the next subsections, we overview the remaining steps of the xApp lifecycle, detailing the Docker image building and publishing, as well as the xApp management operations to onboard, install, query, and uninstall xApps inside the Near-RT RIC cluster through interactions with the *AppMgr*.

B. Creating and Publishing xApp Docker Containers

The xApp developer prepares a Dockerfile during the xApp development process, a text file that specifies the complete environment for running the xApp source code. The Dockerfile serves to build a Docker image file, a read-only snapshot containing the several layers that constitute a live Docker container [46]. For the xApp developer, creating an xApp Docker image serves two purposes. First, it aggregates all the source code, packages, and directory structures required to run the xApp into a single file, which can be uploaded into an accessible Docker Registry, facilitating the xApp developer to distribute its xApp to interested parties, e.g., MNOs, in

a scalable and stateless manner. Second, it allows the xApp developer to instantiate a container from the said image, either standalone or in an xApp pod inside a Near-RT RIC cluster, and interact with a live instance of their compiled source code for development and debugging. For simplicity, we focus on the latter throughout this tutorial, i.e., instantiating xApp containers in pods inside a Near-RT RIC.

We outline the structure of a Dockerfile in Listing 8. It contains a series of basic commands that the xApp developer can use to create the container environment, detailed below. For the complete list of Dockerfile commands and syntax, we refer the reader to the official Docker documentation on [73].

FROM: Defines the base image that we will modify in this Dockerfile, e.g., the certain release of a Linux distribution or the development environment of a Python version [74].

ARG: Creates a temporary variable for use in the Dockerfile, useful for scripting and controlling parameters used in multiple commands, e.g., file paths and package versions.

RUN: Runs a Linux shell command in the Docker image file system, serves to modify system settings, installs required dependencies, and compiles libraries and binaries.

COPY: Copies files and directories from the host machine of the xApp developer to the container image, useful to copy their repositories, source code files, and datasets.

ENV: Defines a Linux environment variable that will persist when the Docker container is instantiated from the resulting image; it serves to specify configuration file locations and pass parameters to the xApp running in this container.

CMD: The last command in a Dockerfile; it specifies the Linux shell command that will be run when the Docker container starts, we use it to start our xApp binary.

The xApp developer will likely need to customize their Dockerfiles using the aforementioned commands according to the requirements, business logic, and dependencies of their xApps. For completeness, we refer the reader to our online repository [29], where we include the entire Dockerfile used to create the Docker image containers for running the Python xApps used throughout this tutorial.

In possession of a Dockerfile, the xApp developer can use its location as an argument to create a Docker image, as shown in Listing 9. The `docker build` command sequentially executes the instructions in the Dockerfile and, when completed, generates a single file containing the snapshot of the container. In addition to the Dockerfile, the `docker build` command requires (i) the hostname and port of a private Docker Registry, either local or remote [75], (ii) a name for

```

1 # Start by building from a base image
2 FROM python:3.8-alpine
3
4 # Create temporary variable with a path
5 ARG dir=/tmp
6
7 # Run shell command to install dependencies
8 RUN apk update && apk add gcc musl-dev bash
9
10 # Copy files from host machine to the image
11 COPY src/ ${dir}/src
12 COPY init/ ${dir}/init
13 COPY setup.py ${dir}/
14
15 # Install the Python xApp
16 RUN pip3 install ${dir}
17
18 # Set location of xApp configuration file
19 ENV CONFIG_FILE=${dir}/config_file.json
20
21 # Starting the container running our xApp
22 CMD run-xapp

```

Listing 8: Basic structure of a Dockerfile with the steps to build a Docker image for running an xApp written in Python.

the xApp container image, and (iii) an associated tag, i.e., a custom human-readable identifier that typically refers to the version or variant of an image. For the container to gain access to the host network, e.g., to clone repositories or install packages, the xApp developer may need to include the `"--network host"` argument. We refer the reader to the official Docker Build documentation [76] for additional information on the `docker build` command. After the xApp developer builds a Docker image, the next step is to publish it to a Docker Registry, so that the container image can be fetched by the Near-RT RIC and deployed as an xApp. The `docker push` command uploads the local Docker image into the chosen private Docker Registry, either local or remote, using the Docker Registry's hostname and port, as well as the xApp container name and tag used during the build phase.

Should the xApp developer decide to set up their own local Docker Registry inside the Near-RT RIC cluster of their O-RAN development environment for testing and debugging xApps, they can use Docker's official open-source registry, which on itself runs as a Docker container, as shown in Listing 10. The `docker run` command instantiates the Docker Registry image (`registry:2`) as a container, where (i) the `"-d"` flag indicates the registry will run as a daemon in the background, (ii) the `-p` flag maps an internal port from the container (which, in this case, listens to port 5000) to an arbitrary host port, and (iii) the `--restart` flag specifies the conditions in which the container will restart automatically. The xApp developer will likely want their local Docker Registry to restart automatically upon system restarts or failures, hence, the `"unless-stopped"` option. In addition, the xApp developer can specify a name for its new Docker Registry container. By default, this Docker Registry is publicly accessible locally, but we can make it remotely accessible and restrict access using passwords or

```

1 # Build xApp image with a name and tag
2 docker build <DOCKERFILE_PATH> -t \
3 <REGISTRY_HOSTNAME>:<REGISTRY_PORT>/
4     ↪ <XAPP_NAME>:<XAPP_TAG> \
5 --network host
6
7 # Example using a local Docker Registry
8 docker build . -t \
9 localhost:5001/test_xapp:1.0.0 \
10 --network host
11
12 # Push the xApp image to Docker Registry
13 docker push \
14 <REGISTRY_HOSTNAME>:<REGISTRY_PORT>/
15     ↪ <XAPP_NAME>:<XAPP_TAG>
16
17 # Example using a local Docker Registry
18 docker push \
19 localhost:5001/example_xapp:1.0.0

```

Listing 9: Command for building and pushing an xApp image.

```

1 # Run a self-restarting Docker Registry
2 docker run -d -p <REGISTRY_PORT>:5000 \
3 --restart unless-stopped \
4 --name <REGISTRY_NAME> registry:2
5
6 # Example using port 5001
7 docker run -d -p 5001:5000 \
8 --restart unless-stopped \
9 --name registry registry:2

```

Listing 10: Command for creating a local Docker Registry.

certificates. We refer the reader to the official Docker Registry documentation [77] for additional information.

After the xApp developer pushes the Docker image(s) of their xApp to a Docker Registry, and updates the xApp configuration file to include the image location, i.e., the Docker Registry's URL, the image name and its tag, they are ready to onboard the xApp into the Near-RT RIC, which we will discuss in the next subsection. The xApp developer can also use the commands shown in Listing 11 to inspect the Docker images stored locally or available from a Docker Registry.

C. Interfacing with the AppMgr via the `dms_cli`

In possession of an xApp configuration and schema files, including the location of the Docker Image(s), the xApp de-

```

1 # Check Docker images stored locally
2 docker image ls
3
4 # Query the available images in a Registry
5 curl -X GET http://<REGISTRY_HOSTNAME>:
6     ↪ <REGISTRY_PORT>/v2/_catalog
7
8 # Example of query to a local Registry
9 curl -X GET
10     ↪ http://localhost:5001/v2/_catalog

```

Listing 11: Commands for querying available Docker images.

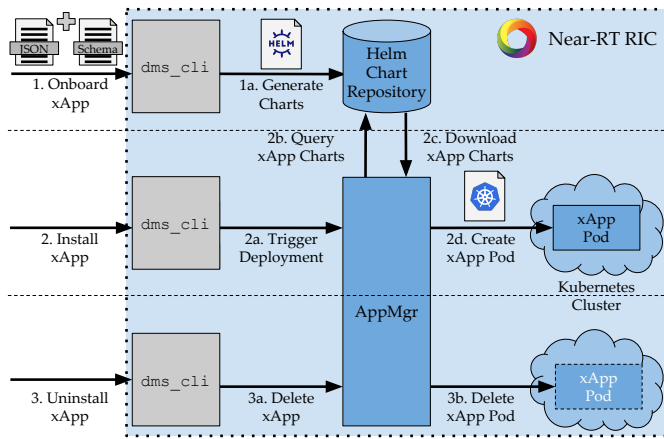


Fig. 9: Interactions between components of the Near-RT RIC to perform operations related to the management of xApps inside the Near-RT RIC, intermediated by the `dms_cli`.

veloper is ready to onboard their xApp, or any of the publicly available first- and third-party xApp listed in Section III-D, into a Near-RT RIC. The onboarding process, as well as the other operations related to the management of xApps by the AppMgr, are intermediated by an application provided by the OSC called `dms_cli` [78], which stands for Deployment Management Service (DMS) Command Line Interface (CLI). The `dms_cli` is a command line tool for deploying xApps and managing their lifecycle in a Near-RT RIC, as illustrated in Fig. 9. We detail below the operations related to xApp management intermediated by the `dms_cli`.

1) *xApp Onboarding*: This process collects the information required to deploy an xApp and stores it locally as Helm Charts for later use. First, the `dms_cli` validates the descriptor file against the schema file, as shown in Listing 12, and throws errors if there are missing parameters or invalid formatting. If the validation succeeds, then the `dms_cli` uses the content of the descriptor file to generate Helm Charts, which define the xApp pod's open ports, computational resources, and environment variables (as discussed in Section III-B), and are stored in a Local Helm Chart Repository available to the AppMgr. After onboarding is complete, the name and version contained in the descriptor file serve as identifiers for the Helm Charts stored in the local repository.

2) *xApp Installing*: This process triggers the AppMgr to deploy an xApp Kubernetes pod using the Helm Charts stored locally during the xApp onboarding. The `dms_cli` passes the name and version of the xApp Helm Chart, alongside the Kubernetes namespace for xApps (defined during the Near-RT RIC installation, defaulting to "ricxapp") to the AppMgr, as shown in Listing 13. Then, the AppMgr queries the Local Helm Chart Repository to download the xApp Helm Charts and use the information therein for creating the xApp pod. The `dms_cli` throws errors if the AppMgr cannot locate the Helm Chart or the correct version. Next, during the instantiation of the xApp pod, Kubernetes uses the location of the xApp Containers in the xApp Helm Chart to fetch the xApp images and instantiate the xApp Kubernetes pod. If the xApp Kubernetes pod is successfully instantiated, there is an

```
1 # Onboard xApp to generate its chart
2 dms_cli onboard <CONFIG_JSON> <SCHEMA_JSON>
3 # Or
4 dms_cli onboard \
5 --config_file_path=<CONFIG_JSON> \
6 --schema_file_path=<SCHEMA_JSON>
7
8 # Example of an onboarding command
9 dms_cli onboard \
10 xapp_path/init/config_file.json \
11 xapp_path/init/schema_file.json
```

Listing 12: `dms_cli` command for onboarding an xApp.

```
1 # Install an xApp on the Near-RT RIC
2 dms_cli install <XAPP_CHART_NAME> \
3 <VERSION> <NAMESPACE>
4 # Or
5 dms_cli install \
6 --xapp_chart_name=<XAPP_CHART_NAME> \
7 --version=<VERSION> \
8 --namespace=<NAMESPACE>
9
10 # Example of an install command
11 dms_cli install example_xapp 1.0.0 ricxapp
```

Listing 13: `dms_cli` command for installing xApps.

additional step where the xApp registers with the AppMgr to avail of the features and capabilities of the Near-RT RIC; we detail this further in Section VI. There might be issues preventing pod instantiation, e.g., Kubernetes cannot fetch the images, reach the Docker Registry location, or the cluster lacks computational resources. However, these errors are not automatically shown to the user as the result of running this command. Instead, the users must debug the Kubernetes deployment themselves to understand the reason for failure, e.g., using the `health_check` command detailed later in this section, or other alternatives described further in Section VIII.

3) *xApp Uninstalling*: This process triggers the AppMgr to stop the execution of a given xApp pod and release all of its resources, as shown in Listing 14. The `dms_cli` throws errors if the AppMgr cannot locate the given xApp or if it is not running. First, the AppMgr instructs Kubernetes to delete the xApp pod, which sends a terminating signal (SIGTERM) to the pod and puts it in a *Terminating* state. Then, Kubernetes grants the xApp pod 30 seconds (by default) to exit gracefully,

```
1 # Uninstall an xApp from the Near-RT RIC
2 dms_cli uninstall <XAPP_CHART_NAME> \
3 <NAMESPACE>
4 # Or
5 dms_cli uninstall \
6 --xapp_chart_name=<XAPP_CHART_NAME> \
7 --namespace=<NAMESPACE>
8
9 # Example of an uninstall command
10 dms_cli uninstall example_xapp ricxapp
```

Listing 14: `dms_cli` command for uninstalling xApps.

```

1 # Upgrade an xApp to a new version
2 dms_cli upgrade \
3 --xapp_chart_name=<XAPP_CHART_NAME> \
4 --old_version=<OLD_VERSION> \
5 --new_version=<NEW_VERSION> \
6 --namespace=<NAMESPACE>
7
8 # Example of an upgrade command
9 dms_cli upgrade \
10 --xapp_chart_name=example_xapp \
11 --old_version=1.0.0 --new_version=1.1.0 \
12 --namespace=ricxapp
13
14 # Roll back an xApp to a previous version
15 dms_cli rollback \
16 --xapp_chart_name=<XAPP_CHART_NAME> \
17 --new_version=<NEW_VERSION> \
18 --old_version=<OLD_VERSION> \
19 --namespace=<NAMESPACE>
20
21 # Example of a rollback command
22 dms_cli rollback \
23 --xapp_chart_name=example_xapp \
24 --old_version=1.1.0 --new_version=1.0.0 \
25 --namespace=ricxapp

```

Listing 15: dms_cli commands to up/downgrade xApps.

after which the pod is forcefully deleted. During this period, the xApp must de-register with the AppMgr, which informs the Near-RT RIC components to remove or release resources associated with it; we detail this further in Section VI. The xApp pod can also use this period to perform additional operations before stopping, e.g., saving cached information to the SDL/STSL. After the grace period, the xApp pod is deleted and its resources are released.

4) *xApp Upgrading and Rolling Back*: This pair of operations, upgrading and rolling back, allow the xApp developer or the user of the Near-RT RIC to change the version of a running xApp. They can be useful for deploying new bug fixes or reverting to a previous stable version of an xApp, respectively. The dms_cli commands for upgrading and rolling back xApps that combine the previous uninstall and install commands. They use the name of the xApp, its old current version, the new intended version, and the xApp namespace, as shown in Listing 15, to trigger AppMgr to carry uninstall and install operations in succession. In that regard, one could use the dms_cli to perform these operations manually, but these commands allow these processes to be partially automated. Similar to the install and uninstall commands, the dms_cli will throw errors if it cannot find the given xApp, if the xApp is not running, or if it cannot locate its name or intended new version in the Local Helm Chart Repository.

In addition to the operations related to the management of xApps listed above, the xApp developer, or Near-RT RIC's users, can leverage the dms_cli to perform a number of other useful operations for querying the status of the onboarded and installed xApps, as well as checking the health of the Local Helm Chart Repository or xApp pods, as shown in Listing 16. We detail these additional operations below.

```

1 # Check health of Helm Chart Repository
2 dms_cli health
3
4 # Query list of onboarded xApps
5 dms_cli get_charts_list
6
7 # Check the health of an xApp pod
8 dms_cli health_check \
9 --xapp_chart_name=<XAPP_CHART_NAME> \
10 --namespace=<NAMESPACE>
11
12 # Download the xApp Helm Charts
13 dms_cli download_values_yaml \
14 --xapp_chart_name=<XAPP_NAME> \
15 --version=<VERSION> \
16 --output_path=<OUTPUT_PATH>
17
18 # Override xApp Helm Chart's values.yaml
19 dms_cli install <XAPP_CHART_NAME> \
20 <VERSION> <NAMESPACE> \
21 --overridefile <VALUES_PATH>

```

Listing 16: dms_cli commands to check useful information.

5) *Checking the Health of the Local Helm Chart Repository*: This operation checks whether the dms_cli can successfully communicate with the Local Helm Chart Repository, whose location is defined by the CHART_REPO_URL environment variable in the Near-RT RIC cluster [67]. This operation is useful to ensure the Near-RT RIC cluster works as it should and that the Local Helm Repository is operational.

6) *Querying Onboarded xApps*: This operation lists all the onboarded xApps whose charts are stored in the Local Helm Chart Repository. The dms_cli lists the xApp charts' names, API versions, creation times, descriptions, hashes for validating their integrity, and the location of their Charts, displayed as JSON strings. This is helpful for identifying missing versions or misspelled names in case xApp installations fail.

7) *Checking the Health of xApp Pods*: This operation checks the deployment status of an xApp, serving as an approach to verify whether the instantiation was successful. The dms_cli uses the xApp's chart name and its namespace to check whether all the containers are ready and initialized, and whether the pod is scheduled and initialized, throwing errors if the pod is not running correctly. We discuss other strategies to assess the deployment of xApps later in Section VIII.

8) *Downloading and Modifying xApps Helm Charts*: This operation allows one to override the Helm Chart used to instantiate the xApp Kubernetes pod before its deployment. This operation is useful for customizing internal parameters according to the MNO's requirements or performing quick tests without the need to modify the xApp's descriptor file and onboarding them again. First, the dms_cli downloads the "values.yaml" file of the Helm Chart, as discussed in Section III-A, using the name and version of the xApp chart, as well as an output path to save the file. Then, the xApp developer or the Near-RT RIC users can modify "values.yaml" file saved locally according to their requirements. Finally, one can use the install command with an optional flag that loads the modified "values.yaml" and overrides the Helm Chart

stored in the Near-RT RIC.

With these commands at their disposal, the xApp developer or the Near-RT RIC's are ready to manage xApps throughout their entire lifecycle. In addition, they can perform a number of operations for testing and debugging the deployment of xApps on a Near-RT RIC cluster, which will be very useful during the xApp development process discussed in the next section.

VI. XAPP IMPLEMENTATION: REALIZING YOUR IDEAS

The Python xApp Framework [63] provides two types of xApp implementations that differ regarding their approach to treating RIC Message Router (RMR) messages: (i) the reactive xApp, known as RMRxApp, is passive and only acts in response to incoming RMR messages, and (ii) the general xApp, known as Xapp, can implement any business logic and act upon any desired criteria. Both xApp implementations import libraries for using the Near-RT RIC interfaces, e.g., RMR, SDL, and REST, provide methods for abstracting interactions with Near-RT RIC components, and automatically register xApps with the AppMgr, simplifying the xApp development.

In this section, we dive deep into the interfaces and functionality available for the xApp implementations, such as messaging, policies, data storage, and external input, accompanied by examples leveraging the Python xApp Framework.

A. Messaging

In the following, we explain how xApps can communicate with one another and the components of the Near-RT RIC. First, we detail the operation of the RMR library, the RMR routing table, and route resolution via the RtMgr. Then, we introduce the two classes of xApps regarding their treatment of RMR messages. Next, we detail the APIs for creating callbacks to receive, reply, and send RMR messages.

1) RMR Library, Routing Table, and Route Resolution:

The Near-RT RIC's RMR messaging infrastructure allows its components and the running xApps to communicate without knowing each other's IP addresses and open ports, which can be subject to changes as their Kubernetes pods are scaled or re-deployed. Each Near-RT RIC component and xApp leverages the RMR messaging library, which abstracts the connection establishment and routing decisions from their business logic. The library operates by forwarding messages to an endpoint (their destination) based on the message type (mtype) and subscription ID (subid) contained in the message; these fields are referred to together as the message key. The mtypes are named values that identify the purpose of the message and must be chosen according to the API of the desired endpoint. For example, for reacting to policies, an xApp must send an A1 policy query (A1_POLICY_QUERY) to the A1 Mediator, and later acknowledge a response with an A1 policy response (A1_POLICY_RESP), as shown earlier in Section IV-C2 in Listing 3. Each mtype has a numeric value, and the full list of supported mtypes and their associated numeric values can be found in the RMR repository [79]. The names of mtypes the xApp can transmit and receive must be specified in its descriptor file, as shown in Listing 4, so RtMgr can create routes for their respective numeric values. Conversely,

```

1 newrt | start | [<table_name>]
2 mse | <mtype>[, <sender_endpoint>] | <subid> |
   ↪ <dest_endpoint> [<[, ]>];] >
   ↪ <dest_endpoint>... ] | %meid]
3 rte | <mtype>[, <sender_endpoint>] |
   ↪ <dest_endpoint> [<[, ]>];] >
   ↪ <dest_endpoint>... ] | %meid]
4 ...
5 newrt | end | [<route_counter>]
```

Listing 17: Structure of an RMR routing table with mse and rte entry record types, showing their mandatory (between chevrons) and optional fields (between brackets and chevrons).

the subid are integers generated by the SubMgr during runtime when subscribing to E2 Nodes, which we detail later in Section VII-E.

The RMR decides how to forward outgoing messages according to the information from the xApp's own RMR routing table, which defines the desired endpoints for each message key. This table can be (i) defined statically, loaded once from a file during the xApp's instantiation, and (ii) updated dynamically, with constant updates from the RtMgr whenever a new xApp or Near-RT RIC component starts [68]. The xApp developer can define their static RMR route table to specify what mtypes their xApp will produce and with whom it will communicate, i.e., which Near-RT RIC components and other xApps. During the xApp instantiation, the RMR library loads a static route table from the path defined by the RMR_SEED_RT environment variable, which can be set in the Dockerfile, as shown in Section V-B.

The RMR routing table file possesses a standard and well-defined structure, as shown in Listing 17. It contains mandatory header and footer lines delimiting its start and end, which can include an optional table name for identification and a counter for the number of route entries used to parse the table's integrity, respectively. In addition, the table can contain any number of entries that specify the routes for each message key, known as entry records. There are two types of entry records, the mse and rte. The mse defines: an mtype, an optional sender application, a subid, and at least one destination endpoint. The subid is only used for RMR messages based on subscriptions, which we will detail later in Section VII-C. For routes unrelated to subscriptions from the SubMgr, one must use the subid -1. The rte is a deprecated type of entry record and may be removed from RMR in future releases [80]. In this context, the OSC advises xApp developers to use only mse entry records for new xApps. However, we can still find several occurrences of rte entries in existing Near-RT RIC components and xApps, so we present it here for completeness. The rte defines: an mtype, an optional sender application, and at least one destination endpoint. It does not support subscriptions, and hence, operates the same way as an mse entry record with the subid -1. Furthermore, we show in Listing 18 a realistic example of a static route table for an xApp that listens to policies from the A1 Mediator and communicates with two other xApps using custom mtypes. We detail how to obtain the endpoints of existing Near-RT

```

1 newrt|start
2 mse|20011|-1|service-ricplt-almediator-rmr.
   ↪ ricplt
3 mse|20012|-1|service-ricplt-almediator-rmr.
   ↪ ricplt
4 mse|30001|-1|service-ricxapp-A-rmr.ricxapp
5 mse|30002|-1|service-ricxapp-B-rmr.ricxapp
6 mse|12040|200| %meid
7 newrt|end

```

Listing 18: Example of an xApp's RMR routing table file, configured to send A1 policy query (20011) and response (20012) messages to the A1 Mediator, messages with custom mtypes (30001 and 30002) to two other xApps, and a subscription control request message (12040) with a subid 200 to the entity that owns the E2 Node, i.e., the E2Term.

```

1 newrt|start
2 mse|mttype_1|subid_1|
   ↪ dest_endpoint_A;dest_endpoint_B
3 mse|mttype_2|subid_2|
   ↪ dest_endpoint_M,dest_endpoint_N
4 mse|mttype_3|subid_3|
   ↪ dest_endpoint_X;dest_endpoint_Y_1,
   ↪ dest_endpoint_Y_2;dest_endpoint_Z
5 newrt|end

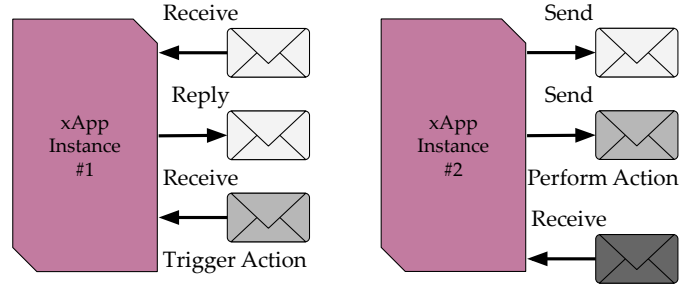
```

Listing 19: Example of the different approaches for sending messages to multiple destination endpoints. Endpoints separated by semicolons receive copies of all messages, while endpoints separated by commas are cycled in round robin.

RIC components and running xApps later in Section VIII.

The RMR library also supports sending messages to a group of multiple destination endpoints using two message distribution approaches: (i) fanout, where each destination endpoint receives a copy of the outgoing message, which is useful to broadcast information to multiple xApps; or (ii) round-robin, where messages are cycled to one endpoint at a time, which is useful for load balancing across multiple xApps. To accomplish this, the RMR introduces the concept of endpoint groups, each of which can contain one or more endpoints. The RMR messages are distributed in fanout to multiple endpoint groups, which are separated by semicolons, and each group will receive copies of all messages. Moreover, the RMR messages are distributed in round-robin to the endpoints comprising an endpoint group, which are separated by commas, and successive messages will be cycled between endpoints. Listing 19 shows examples of the two message distribution approaches and how they can be combined to send messages in more complex manners. For example, the third entry record in Listing 19 will fanout every message to `dest_endpoint_X` and `dest_endpoint_Z`, and round-robin the same messages between `dest_endpoint_Y_1` and `dest_endpoint_Y_2`.

The RMR allows the selection of the destination endpoint based on the Managed Entity ID (MEID) contained in the RMR message instead of selecting the endpoint on the matching entry records [80]. When routing using MEID, the RMR



(a) Reactive xApp.

(b) General xApp.

Fig. 10: Example of differences between reactive and general xApps. The reactive xApps can only perform tasks triggered by receiving RMR messages, whereas the general xApps can support any desired business logic and performing actions in any order, including sending messages to trigger other xApps.

```

1 # Initialize the xApp
2 def __init__(self):
3     # RMRXapp Class constructor
4     self._rmr_xapp = RMRXapp(
5         <default_message_handler>,
6         config_handler=<config_handler>,
7         post_init=<post_init_method>,
8         rmr_port=<RMR_port>
9     )

```

Listing 20: Structure of the RMRXapp class constructor.

message is sent to the endpoint that owns the managed entity. To use MEID routing, one or more route table entry records must contain the special endpoint name `%meid` instead of a list of destination endpoints, as shown in Listing 18. This feature is particularly useful in the context of subscriptions for routing messages to E2 Nodes. In this case, the E2Term owns the E2 Nodes and intermediates all their communications, and the special entry records are created automatically by the SubMgr in conjunction with RtMgr (detailed later in Section VII).

After the xApp initialization and RMR loading the static route table file, the xApp's route table is updated periodically by the RtMgr. These updates happen through the `rmroute` port opened in the xApp descriptor file, as shown in Listing 4. The RtMgr populates the xApps' routing tables with information about the accepted mtypes and existing endpoints of Near-RT RIC components and other running xApps. Every time a new xApp is registered with the AppMgr (detailed in the next section), the AppMgr informs the RtMgr about the new endpoints. Then, the RtMgr propagates this information to existing Near-RT RIC components and xApps. The RMR stashes the additional routes updated during runtime on the same directory where the static table route file is located, with an added `.stash` extension, to facilitate debugging.

2) *Reactive and General xApps*: The Python xApp Framework contains two xApp implementations that only differ regarding their treatment of RMR message, as illustrated in Fig. 10. On the one hand, the RMRXapp provides a more straightforward starting point for xApp developers, leveraging custom callbacks to trigger different actions and reply to RMR messages, e.g., controlling the E2 Nodes based on new policies

```

1 # Called when xApp descriptor file changes
2 def config_handler(self, rmr_xapp, config):
3     # Check for missing parameters
4     if "flag" not in config["controls"]:
5         raise ValueError('Missing parameter')
6
7     # Load the new configuration data
8     rmr_xapp._config_data = config

```

Listing 21: Example of the `config_handler` function, checking for required parameters before starting the xApp.

from the A1 Mediator or storing information on SDL based on messages from other xApps. On the other hand, the Xapp is more versatile and allows the development of more complex xApps, e.g., deciding to send multiple RMR messages to xApps and Near-RT RIC components, promptly interfacing with SDL or the RAN and when to listen to incoming RMR messages, which comes at the cost of being more involved and requiring attention to detail from the xApp developer.

From an implementation standpoint, the RMRXapp operates in a loop, listening to and handling incoming RMR messages with callbacks, and checking for changes in the xApp descriptor file. It also automatically replies to health checks and de-registers itself with the AppMgr and gracefully exits when the xApp process is terminated. The RMRXapp requires the xApp developer to specify: (i) a default RMR message callback to handle incoming messages, (ii) a configuration handler to load and sanitize the xApp configuration file, (iii) a post-initialization function that will be called after the xApp class is initialized, and (iv) the port that the RMR library will listen to (defaults to 4060), as shown in Listing 20. The configuration handler loads the content of the xApp descriptor file into the xApp, as shown in Listing 21. This method is called when the xApp starts running and whenever the configuration file is modified (either by the Near-RT RIC users manually or by editing the xApp pod's ConfigMap). The xApp developer can leverage this method to sanitize its configuration, check for missing parameters, and log information. The post-initialization function serves to instantiate objects and create class attributes available in the RMR message callbacks, e.g., logging objects (detailed later in Section VIII) or data structures shared between callbacks, as shown in Listing 22. Finally, the RMRXapp allows the xApp developer to create a default RMR message handler, serving as a catch-all for all unregistered mtypes, and register specialized RMR message handlers for responding to specific mtypes, as shown in Listing 23. We detail how to create callbacks to receive, reply, and send RMR messages in the next section.

The Xapp implementation provides only the minimal core functionality for the operation of xApps, requiring the xApp developer to implement most of the procedures automated and abstracted by the RMRXapp. Nevertheless, it gives the xApp developer more control to implement any desired business logic. The Xapp requires the xApp developer to specify (i) an `entrypoint` function that will be called after the Xapp class is initialized, and (ii) the port that the RMR library will listen to (defaults to 4060), as shown in Listing 24. The

```

1 # Function called after the constructor
2 def _post_init(self, rmr_xapp):
3     # Create a class attribute
4     rmr_xapp.callback_counter = 0
5
6     # Set the log level of the xApp
7     rmr_xapp.logger.set_level(Level.DEBUG)

```

Listing 22: Example of a `post_init` function, creating class attributes and instantiating objects shared between callbacks.

```

1 # Register custom RMR callback handlers
2 self._rmr_xapp.register_callback(
3     ↪ <custom_message_handler>, <mtype>)
4
5 # Examples of custom handlers
6 self._rmr_xapp.register_callback(
7     self._message_handler, 30002
8 )
9 self._rmr_xapp.register_callback(
10    self._policy_request_handler,
11    A1_POLICY_REQ
12 )

```

Listing 23: Example on how to register RMR message callbacks for handling different mtotypes using the RMRXapp.

`entrypoint` method is the only function that the Xapp will execute, and hence, the xApp developer must use it to implement their business logic. For example, setting the log level (detailed later in Section VIII), opening and loading the xApp configuration file, and creating their own loop with any custom actions, e.g., performing an RMR and SDL health check, sending messages to two other xApp or Near-RT RIC components and then listening to incoming RMR messages, as shown in Listing 25.

It is worth mentioning that both xApp implementations automate the registration of xApp with the AppMgr, a critical step for xApps to work correctly and interface with Near-RT RIC components after being instantiated [67]. In this process, the xApp (i) generates its RMR and HTTP endpoints according to their names, namespace, and interface types, (ii) locates the AppMgr exposed Kubernetes services, and (iii) forwards its name, version, namespace, RMR, and HTTP endpoints, as well as its configuration in JSON format to the AppMgr. In possession of this information, the AppMgr notifies other Near-RT RIC components of the new xApp, provides them

```

1 # Initialize the xApp
2 def __init__(self):
3     # Xapp Class Constructor
4     self._xapp = Xapp(
5         <entrypoint_function>,
6         rmr_port=<RMR_port>
7     )
8     # Potential flag to control xApp shutdown
9     self.shutdown = False

```

Listing 24: Structure of the Xapp class constructor.

```

1 # Function called after the constructor
2 def _entrypoint(self, xapp):
3     # Set log level
4     self._xapp.logger.set_level(Level.DEBUG)
5     # Load configuration file
6     self._xapp._config_data = load(
7         open(self._xapp._config_path))
8
9     # Loop while not set to shutdown
10    while not self.shutdown:
11        # Health check the RMR and SDL
12        if not xapp.healthcheck():
13            # Oops, something is going wrong
14            xapp.logger.error(
15                "Healthcheck failed. Terminating.")
16            # Let us stop the xApp here
17            self.shutdown = True
18
19        # Do anything you like!
20        xapp.rmr_send(<payload_1>, <mtype_1>)
21        xapp.rmr_send(<payload_2>, <mtype_2>)
22
23        # Check for incoming messages
24        for (summary, msg_buf) in
25            ↪ xapp.rmr_get_messages():
26            # Log the received message
27            xapp.logger.info("Msg:" + str(summary))
28
29            # Dispatch mtypes to custom callbacks
30            if summary[rmr.RMR_MS_MSG_TYPE] ==
31            ↪ 30002:
32                self._message_handler(
33                    xapp, summary, msg_buf
34                )
35            elif summary[rmr.RMR_MS_MSG_TYPE] ==
36            ↪ A1_POLICY_REQ:
37                self._policy_request_handler(
38                    xapp, summary, msg_buf
39                )
40
41        # Sleep for a while
42        sleep(1)

```

Listing 25: Example of the entrypoint function, opening and loading the xApp descriptor file, and then checking if RMR and SDL are operational to remain operational, sending RMR messages, and listening to incoming messages in a loop.

with the new endpoints for establishing communication with the xApp, and notifies the xApp that it is ready to work. The RMRXapp handles being terminated and automatically triggers its de-registration process with the AppMgr, which removes references to the given xApp and its endpoints from all Near-RT RIC components. However, the Xapp expects the xApp developer to handle the de-registration themselves. *Failure to de-register the xApp will leave broken references and endpoints on the Near-RT RIC components, leading to undefined behavior and preventing a new instance of that xApp from working correctly until the Near-RT RIC cluster is restarted.* We explain how the xApp developer can automatically trigger the de-registration of their xApps in Section IX.

3) *Communicating using RMR*: The Python xApp Framework provides methods for receiving, replying, and sending

```

1 # Returns the queue of received messages
2 summaries, msg_bufs =
3     ↪ xapp.rmr_get_messages()
4
5 # Reply to received message reusing buffer
6 rmr_xapp.rmr_rts(<msg_buf>
7     ↪ [,new_payload=<payload>]
8     ↪ [,new_mtype=<mtype>]
9     ↪ [,retries=<n_retries>])
10
11 # Send an RMR message w/ custom payload
12 xapp.rmr_send(<payload>, <mtype>
13     ↪ [,retries=<n_retries>])
14
15 # Free memory allocated to message buffer
16 rmr_xapp.rmr_free(<msg_buf>)

```

Listing 26: Structure of the methods available to xApps for sending, receiving and replying to RMR messages.

```

1 # Payload data
2 summary[rmr.RMR_MS_PAYLOAD]
3 # Payload length
4 summary[rmr.RMR_MS_PAYLOAD_LEN]
5 # Subscription ID
6 summary[rmr.RMR_MS_SUB_ID]
7 # Transaction id (send or reply)
8 summary[rmr.RMR_MS_TRN_ID]
9 # Status (ok or not ok)
10 summary[rmr.RMR_MS_MSG_STATUS]
11 # Error if not ok
12 summary[rmr.RMR_MS_ERRNO]
13 # Managed Entity ID
14 summary[rmr.RMR_MS_MEID]

```

Listing 27: Information included in RMR message summary.

RMR messages, as shown in Listing 26. The RMR messages contain a payload in the form of JSON-compatible Python objects, e.g., dictionaries, strings, floats, etc., accompanied by a *mtype*. The RMR library stores the message data as *bytes*, and hence, the sent payloads must be encoded as *UTF-8* strings. Conversely, the received payloads must be decoded from *UTF-8* strings. In possession of the payload and *mtype*, the RMR library: (i) allocates a message buffer to store the RMR message; (ii) generates the message metadata, e.g., length, status, etc., and stores it on the message buffer; and (iii) forwards a copy of the message buffer to its destination RMR endpoint based on the RMR routing table [68].

Both xApp implementations abstract and automate the creation of a threaded RMR server for listening to incoming RMR messages and storing the received RMR messages in a queue. Therefore, the xApp developer only needs to check for the presence of new messages and potentially parse them according to their *mtypes* to select the correct callback for handling them. When received, the RMR messages contain a summary dictionary containing their data and metadata, whose fields are shown in Listing 27, and the raw message buffer where the message was stored. After receiving an RMR message, the xApp developer can either (i) reuse the allocated message buffer to create a reply to the same sender, which may


```

1 # All payloads must be encoded in UTF-8
2 xapp.rmr_send("hi".encode(), 30001,
3             ↪ retries=5)
4 xapp.rmr_send(str(3.14).encode(), 30002)
5 # Let us iterate over the received messages
6 for (summary, msg_buf) in
7     ↪ xapp.rmr_get_messages():
8     # Create a new serializable payload
9     new_payload = dumps({"my_key": "my_val"})
10
11     # Reply to received msg w/ new payload
12     rmr_xapp.rmr_rts(msg_buf, retries=10,
13                    new_payload=new_payload.encode())
14
15     # Clear the msg_buf after we use it
16     rmr_xapp.rmr_free(msg_buf)

```

Listing 28: Example on how to combine RMR methods to create a custom communication protocol between xApps.

or may not contain the same *mtype*, or (ii) free the memory allocated for the message buffer if they have no further use for it, which prevents memory leaks [80].

We can combine the methods for receiving, replying, and sending RMR messages to create communication protocols between xApps, as shown in Listing 28. For example, an xApp developer may create xApps that send information between each other, replying with an acknowledgment confirming the reception of the messages akin to TCP, or xApps that first manipulate the received data in some manner before returning the results to the original sender. For more information on creating chains of xApps that communicate via RMR, we refer the reader to O-RAN's anomaly detection use case, which employs three xApps working together to detect anomalous UEs accessing the RAN [81].

The xApp developer can encapsulate the steps for handling messages and creating communication protocols in callback functions, as shown in Listing 29. The creation of callbacks is a requirement for the RMRXapp, which relies on registered callbacks to operate. However, the creation of callbacks is an optional software design approach for the xApp, which can support the message handling directly inside its `entrypoint` method. For the RMRXapp implementation, these functions are automatically called any time the xApp receives an RMR message with the corresponding registered *mtype*, as shown earlier in Listing 23. For the Xapp implementation, the xApp developer must include a mechanism to parse received messages by their *mtype* and then call the corresponding callback, as shown earlier in Listing 25. These callback functions receive as arguments: (i) a pointer to the class instance where the callback was defined; (ii) a pointer to the xApp implementation being used, which is useful for accessing its internal information and functionality; (iii) the RMR message summary dictionary, which includes the message data and metadata; and (iv) the raw RMR message buffer. In possession of these arguments, the xApp developer can implement any business logic leveraging the interfaces and functionality available to the xApp.

```

1 # Example of a default message callback
2 def _default_message_handler(self, xapp,
3                               ↪ summary, msg_buf):
4     # Logging incoming message types
5     xapp.logger.info(
6         "Handler called for mtype: " +
7         ↪ str(summary[rmr.RMR_MS_MSG_TYPE])
8     )
9     # Logging incoming message contents
10    xapp.logger.debug(
11        "Message content: " +
12        ↪ str(summary[rmr.RMR_MS_PAYLOAD])
13    )
14
15    # Modify internal class parameter
16    rmr_xapp.callback_counter += 1
17
18    # Return an acknowledgement
19    xapp.rmr_rts(msg_buf,
20                new_payload="ack".encode())
21
22    # Free allocated memory
23    xapp.rmr_free(msg_buf)

```

Listing 29: Example of the creation of callback functions for handling RMR messages and executing desired operations.

```

1 {
2     "payload": {
3         <policy_payload>
4     },
5     "policy_type_id": <policy_id>,
6     "policy_instance_id":
7         ↪ <policy_instance_id>,
8     "operation": <operation>
9 }

```

Listing 30: Structure of the RMR message payload from an A1 policy that an xApp will receive from the A1 Mediator.

B. Policies

In the following, we explain how xApps can listen to and take actions based on policies from the Non-RT RIC. We detail the structure of an A1 policy and outline the steps to enable support for policies in an xApp, describing how to create handlers and callbacks for reacting to A1 policies.

1) *A1 Interface between Non- and Near-RT RIC*: The rApps in the Non-RT RIC can generate policies for steering the behavior of the Near-RT RIC and the xApps therein. These policies contain high-level intents, allowing xApps to decide how to interpret and act upon them. In the Non-RT RIC, the A1 policies are expressed in JSON, following a specific syntax validated through a JSON schema [12]. The A1 Mediator serves as a northbound interface toward the Non-RT RIC, translating the A1 policies received via the A1-AP interface in JSON to the RMR format used for internal communication in the Near-RT RIC [82]. After this translation, the A1 Mediator publishes policies in RMR format to the xApps that have registered to receive policies of that given type. Finally, the xApps receive and handle A1 policies in the same way they receive other RMR messages.

The A1 policies received via RMR possess a predefined RMR message payload structure, as shown in Listing 30. These are Python dictionaries that contain the following fields:

- payload:** A Python dictionary containing JSON-compatible objects. It contains the high-level information, parameters, or flags generated by rApp in the Non-RT RIC for controlling the operation of xApps in the Near-RT RIC.
- policy_type_id:** An integer identifying the type of A1 policy that xApps will listen to and defines the template of the policy, i.e., the fields of the payload dictionary and its types and ranges of accepted values.
- policy_instance_id:** A string identifying a concrete realization of a given A1 policy, complete with values. Instances of a given policy type will always contain the same structure but may contain different values.
- operation:** A string defining the operation being performed. It can be either "CREATE" when the xApp starts running or a new policy instance is deployed; "UPDATE" when the policy instance is updated with new values in its payload dictionary; or "DELETE" when the policy instance is removed from the A1 Mediator.

The xApp developer may use the information about the policy instance types, the values in its payload, and the current operation to steer the operation of their xApps as they see fit. For example, the xApp developer can create xApps that react to values from policies to change the signal strength threshold for handovers [83] or switch the scheduling algorithm of base stations on the fly.

2) *Handling A1 Policies:* There are a few steps required to enable support for A1 policies on an xApp. First, the xApp developer must edit the xApp descriptor file and include the following `mtypes` in the RMR configuration section: (i) The `A1_POLICY_REQ` on the list of received messages to obtain RMR messages with A1 policies, (ii) the `A1_POLICY_RESP` on the list of transmitted messages to reply to the A1 policy with an acknowledgment, and optionally (iii) the `A1_POLICY_QUERY` also on the list of transmitted messages to query all existing instances of a given policy type, as shown earlier in Listing 3. Then, the xApp developer must list the policy type identifiers for all A1 policies of interest for the xApp pod on the RMR configuration section, and the specific policy type identifiers on the ports and services section for the containers that will handle each A1 policy, as shown in Listing 4. Such separation allows the xApp developer to use different containers to handle distinct A1 policies. Next, the xApp developer must edit the static route table file of their xApp for routing the `A1_POLICY_RESP` (and optionally the `A1_POLICY_QUERY`) messages to the A1 Mediator, as shown in Listing 18. Finally, the xApp is ready to receive RMR messages containing A1 policies from the A1 Mediator.

To handle A1 policies, the xApp developer must create a policy callback and register it according to its xApp implementation (either directly with the `RMRXapp` or manually in the `Xapp`, as shown in Listing 25). We show an example of such a policy callback function in Listing 31, where we first check the validity of the JSON data structure and the integrity of the A1 policy (whether it contains the required dictionary keys). Then, we can make any decisions according

```

1 def _policy_request_handler(self, xapp,
2     ↪ summary, msg_buf):
3     # Clear message buffer
4     self._rmr_xapp.rmr_free(msg_buf)
5
6     try:
7         # Get JSON string encoded as bytes
8         req = json.loads(
9             summary[rmr.RMR_MS_PAYLOAD])
10
11     except (json.decoder.JSONDecodeError,
12           ↪ KeyError):
13         self.logger.error("Invalid JSON")
14         return
15
16     # Check mandatory policy keys
17     policy_keys = ["policy_type_id",
18                   ↪ "operation", "policy_instance_id"]
19     if not all(key in policy_keys for key in
20               ↪ req.keys()):
21         self.logger.error("Invalid policy")
22         return
23
24     # Do anything you like!
25
26     # Construct response
27     req["handler_id"] =
28         ↪ self._rmr_xapp._config_data["name"]
29     req["status"] = "OK"
30     del req["operation"]
31
32     # Convert dict. to JSON string in UTF-8
33     self._xapp.rmr_send(json.dumps(resp) .
34                       ↪ encode(), A1_POLICY_RESP)

```

Listing 31: Example of an A1 Policy Handler.

to the consent of the A1 policy, as detailed in the previous section. Next, we must send an acknowledgment to the A1 Mediator in the form of an `A1_POLICY_RESP` message. The A1 Mediator expects a response with the same policy type and instance identifiers from the `A1_POLICY_REQ`, as well as the name of the xApp that consumed the A1 policy and the return status of this operation, which can be an OK to indicate success or ERROR to indicate failure in consuming the A1 policy. Therefore, we can reuse part of the RMR payload from the `A1_POLICY_REQ` message and adapt it accordingly.

C. Storage

In the following, we explain the Shared Layer functionality that xApps can leverage to store data within the Near-RT RIC, detail the APIs available for xApps to read, write, modify, and delete information from persistent storage, and describe the Network Information Base (NIB) Databases.

1) *Share Layer Abstraction:* Storing data in the Near-RT RIC can be useful for (i) saving and retrieving the application state, which will persist if the xApp gets updated, rolled back, crashes, or reboots, (ii) performing data analytics over long-term metrics, and (iii) transferring large amounts of data between xApps. However, each Near-RT RIC instance and Kubernetes deployment can have different configurations for their data storage backends, e.g., different credentials and

```

1 # Reads value for a given key
2 xapp.sdl.get(ns, key)
3
4 # Writes a key value entry
5 xapp.sdl.set(ns, key, val)
6
7 # Deletes key and value entry
8 xapp.sdl.delete(ns, key)
9
10 # Writes key value if it does not exist
11 xapp.sdl.set_if_not_exists(ns, key, val)
12
13 # Updates value if old value matches search
14 xapp.sdl.set_if(ns, key, old_val, new_val)
15
16 # Removes entry if value matches search
17 xapp.sdl.delete_if(ns, key, val)
18
19 # Find keys starting with prefix
20 xapp.sdl.find_keys(ns, prefix)
21
22 # Find keys starting w/ prefix and get
23 # their associated values
24 xapp.sdl.find_and_get(ns, prefix)

```

Listing 32: The SDL API calls available for xApps in Python to leverage the Near-RT RIC's internal relational database.

authorization mechanisms, database software from various vendors (incurring in different APIs), and distinct database architectures, e.g., distributed, redundant, or load-balancing databases. Thus, the OSC created the Shared Layers to handle the actual data storage while providing a unified, flexible interface to xApp, abstracting the specific implementation from the current database backend, which allows xApps to be stateless and portable across different Near-RT RICs [69]. There are two types of Shared Layers: the (i) SDL, supporting structured databases, which organize data using keys and namespaces, and the (ii) STSL, supporting time-series databases, which organize data sequentially with associated time stamps. At the time of writing, STSL support is limited and only available for xApps implemented in Go [84]. While this is expected to change in the future, we will refrain from detailing its API as it has yet to become available in the Python xApp Framework.

2) *Storing Data using SDL*: The SDL structures data according to keys, values, and namespaces. Keys and values operate similarly to Python dictionaries or JSON key-value pairs, where each data entry has a human-readable tag associated with a value. Namespaces encapsulate the data, attributing an identifier for a group of keys and their values [69]. Each xApp can use one or more namespaces to identify its persistent data and store as many keys and values as the underlying database backend capacity allows. By using distinct namespaces, we can isolate data between different xApps and Near-RT RIC components. Conversely, using the same namespaces enables us to share data across xApps and Near-RT RIC components.

An xApp can leverage the SDL Library, part of the Python xApp Framework, to avail from the capabilities of this Shared Layer to read/write persistent data on the Near-RT RIC. The SDL Library offers xApps with an API for manipulating data

```

1 # On the First xApp:
2 # -----
3 # Writes an entry on its own namespace
4 xapp_1.sdl.set("xapp_1_ns", "gnb_meid",
5               ↪ "gnbABCDEF")
6
7 # Writes on shared namespace if new entry
8 xapp_1.sdl.set_if_not_exists("shared_ns",
9                               ↪ "ue_list_240101_123836", ue_list)
10
11 # On the Second xApp:
12 # -----
13 # Tries to access key not in this namespace
14 xapp_2.sdl.get("xapp_2_ns", "gnb_meid")
15 # This API call will return None
16
17 # Finds key used in the shared namespace
18 ue_key = xapp_2.sdl.find("shared_ns",
19                           ↪ "ue_list")
20
21 # Reads value from the shared namespace
22 ue_list = xapp_2.sdl.get("shared_ns",
23                           ↪ ue_key)
24
25 # Deletes entry from shared namespace
26 xapp.sdl.delete("shared_ns", ue_key)

```

Listing 33: Example of how xApps can use multiple SDL namespaces to isolate and share data with each other, also showing how xApps can manipulate data across namespaces.

on a given namespace, as shown in Listing 32. Through this API, xApps can perform (i) traditional data storage operations, e.g., reading values associated with keys, writing new keys and values, and deleting keys and their associated values; (ii) conditional operations, e.g., writing new keys and values if no entries with these keys exist, updating keys and values according to their existing values, and deleting keys-value pairs if their keys and values match a search; and (iii) search operations, e.g., retrieving existing keys (or keys and their associated values) that start with a prefix (which can be an empty prefix for obtaining all the existing keys). Moreover, we can see examples of how xApps can leverage SDL namespaces to isolate and share data between each other in Listing 33.

3) *User Equipment and Radio NIBs*: The Near-RT RIC contains two databases for storing information about the RAN: The (i) Radio-NIB (R-NIB) database contains information about the E2 nodes, their supported SMs, and RAN functions (detailed later in Section VII-B), and the (ii) User Equipment-NIB (UE-NIB) contains information about the associated UEs, their identity and reported metrics [85]. The R-NIB is populated by the E2Mgr whenever new base stations set up an E2 connection, serving as an inventory of RAN elements connected to the Near-RT RIC. Meanwhile, the UE-NIB contains identifying tags for associated UEs, enabling the Near-RT RIC and its xApps to make user-centric decisions at the cost of storing potentially sensitive information about users [12]. At the time of writing, the UE-NIB exists in the O-RAN specifications, but there are no current efforts to develop it in the OSC or enable it in the xApp frameworks. For more information about the current implementation of the UE-NIB,

```

1 # Gets list of all base stations
2 xapp.GetListNodeIds()
3
4 # Gets list of all gNodeBs
5 xapp.get_list_enb_ids()
6
7 # Gets list of all eNodeBs
8 xapp.get_list_gnb_ids()
9
10 # Get detailed info about base station
11 xapp.GetNodeB(<inventory_name>)
12
13 # Gets definition of RAN functions
14 xapp.GetRanFunctionDefinition(
    ↳ <inventory_name>,
    ↳ <ran_function_oid>)

```

Listing 34: The R-NIB API calls available for xApps in Python to find information about the currently connected base stations.

```

1 # Returns list gNodeBs
2 for gnb in xapp.get_list_gnb_ids():
3     print("gNodeB:", gnb)
4
5 # Get the name of the last gNodeB
6 gnb_name = gnb.inventory_name
7
8 # Get detailed info about that gNodeB
9 print(xapp.GetNodeB(gnb_name))
10
11 # Get its RAN Function definition
12 print(xapp.GetRanFunctionDefinition(
    ↳ "gnb_734_733_16b8cef1", "OID123"))

```

Listing 35: Example of how to find a base stations's inventory name from the R-NIB, and use it to find detailed information about the base station and its supported RAN functions.

we refer the reader to the SD-RAN documentation [86].

The Python xApp Framework offers an API for accessing information stored in the R-NIB, allowing xApps to find the list of base stations currently connected to the Near-RT RIC (either eNodeBs, gNodeBs, or both), as shown in Listing 34. The base stations are stored in the R-NIB using an inventory name generated by the E2Mgr, which serves to identify the E2 Nodes in O-RAN. We can use the inventory name to obtain detailed information about a particular base station, such as (i) its type and connection status, (ii) the Public Land Mobile Network (PLMN) ID and gNodeB ID, which identify the MNO and the base station [87], respectively, (iii) certain base station configurations, e.g., its associated AMF, and (iv) its supported RAN functions (detailed later in Section VII-B). We provide an example of how to find the inventory name of a base station and use it to obtain detailed information about its RAN Functions in Listing 35 (detailed in Section VII).

D. External Input

In the following, we explain how xApps can respond to external input using their REST interface. First, we detail how to enable support for REST and handle HTTP requests. Then, we describe how to respond to probes and user interactions.

```

1 # Create HTTP server to listen to requests
2 self.server =
    ↳ xapp_rest.ThreadedHTTPServer(
    ↳ <address>, <port>)
3
4 # Example of a server listening to
5 # requests from any host on port 8080
6 self.server =
    ↳ xapp_rest.ThreadedHTTPServer(
    ↳ "0.0.0.0", 8080)

```

Listing 36: Example of the creation and configuration of a threaded HTTP server for listening to requests inside an xApp.

```

1 # Create handler for requests on a URI
2 self.server.handler.add_handler(
    ↳ self.server.handler,
    ↳ <HTTP_request_type>,
    ↳ <REST_call_name>, <URI>,
    ↳ <callback_method>)
3
4 # Example of a REST method to get config
5 self.server.handler.add_handler(
    ↳ self.server.handler, "GET",
    ↳ "config", "/ric/v1/config",
    ↳ self.configGetHandler)

```

Listing 37: Example of the creation of a handler to serve incoming HTTP requests and implement a REST call.

1) REST Interface, HTTP Server, Handlers, and Callbacks: The xApps have an optional REST interface, allowing them to respond to external input aside from the pods of Near-RT RIC components. It serves two purposes: (i) allows xApps to react to Kubernetes' readiness and liveness probes, indicating their operational status; and (ii) allows xApps to support interactions from the users of the Near-RT RIC, which can be useful for obtaining information about the internal state of the xApps and passing control parameters [67]. REST is a widely popular interface for web-based applications, which maps HTTP requests acting on exposed URI endpoints onto internal RPC calls, allowing remote hosts to query information, execute functions, and pass parameters to a local server over HTTP. For brevity, we refer the reader to [88] for additional information on the RESTful paradigm and the operation of REST calls running on top of HTTP requests.

The Python xApp Framework contains the `xapp_rest` library, which simplifies the process of handling HTTP requests and creating REST callbacks. However, there are a few steps required to enable support for the REST interface on an xApp: First, the xApp developer must edit the xApp descriptor to open port 8080 for Kubernetes to expose the HTTP service on the desired containers, as shown earlier in Listing 4. Then, the xApp developer must create an HTTP server in the xApp to listen to incoming HTTP requests. The `xapp_rest` provides a threaded HTTP server that can listen to incoming HTTP requests without blocking the xApp's main loop, as shown in Listing 36. For the HTTP server to work correctly, the xApp developer must instantiate it inside the `post_init` method


```

1 # Structure of a generic REST handler
2 def example_rest_handler(self, name, path,
   ↪ data, ctype):
3     # Method to initiate an HTTP response
4     response = xapp_rest.initResponse()
5
6     # Decode data if there was any in request
7     python_data = data.decode("utf-8")
8
9     # Create resp. w/ a status code and payload
10    response['status'] = <HTTP_status_code>
11    response['payload'] = <desired_response>
12    # Return new HTTP response
13    return response
14
15 # Example of a readiness probe handler
16 def readiness_handler(self, name, path,
   ↪ data, ctype):
17     # Initiate a new HTTP response
18     response = xapp_rest.initResponse()
19
20     # Check if a key was populated in SDL
21     if self.xapp.sdl.get("xapp_1_ns",
   ↪ "gnb_meid"):
22         # We are ready to start working
23         response['status'] = 200
24     else:
25         # We are not ready yet
26         response['status'] = 500
27
28     return response

```

Listing 38: The structure of REST handlers and examples of how to respond to Kubernetes probes.

for reactive xApps, or inside the `entrypoint` method for general xApps. At this point, the xApp is ready to receive and listen to incoming HTTP requests.

The next step is to register URI endpoints in the HTTP server, specify their supported HTTP request types, e.g., GET, POST, PUT, DELETE, etc., and map which internal REST callback will reply to a certain HTTP request type on given URI. This step is crucial for exposing any internal information and functionality from the xApp through the REST interface. The `xapp_rest` HTTP server allows us to create handlers for registering URI endpoints and mapping an HTTP request type to an internal function that will be called every time the server receives an HTTP request of that type on that given URI endpoint, as shown in Listing 37. The xApp developer should consider which type of HTTP request to use when exposing internal information and functionality via REST, as they behave in different manners [88]. For example, POST and PUT requests are accompanied by new resources, e.g., JSON data structures, which can serve as input for xApps, whereas GET and DELETE requests only contain identifiers for the resources they are operating on. Finally, we can create REST callbacks to implement any logic for reacting to incoming HTTP requests from a remote host. We detail how to create REST callbacks for reacting to Kubernetes's probes and user interactions in the next subsection.

2) *Probes and Custom User Interaction:* To verify the availability and health of pods in a cluster, Kubernetes employs

```

1 # Example of a GET handler
2 def get_config_handler(self, name, path,
   ↪ data, ctype):
3     # Initiate a new HTTP response
4     response = xapp_rest.initResponse()
5     # Attribute its the OK status code
6     response['status'] = 200
7     # Return a JSON w/ the xApp configuration
8     response['payload'] = dumps(
   ↪ self._xapp._config_data)
9
10    return response
11
12 # Example of a POST handler
13 def set_new_parameters(self, name, path,
   ↪ data, ctype):
14     # Initiate a new HTTP response
15     response = xapp_rest.initResponse()
16     # Decode new information and save it
17     self.upload = data.decode("utf-8")
18     # Create response w/ JSON success message
19     response['payload'] = ('[{"uploaded" :
   ↪ "complete"}]')
20
21    return response

```

Listing 39: Examples of REST handlers, showing how xApps can respond to different types of external input, e.g., retrieving the internal xApp state and passing new parameters.

two probes on each container: (i) the readiness probe checks if the container carried out all required initialization tasks and ensures it is ready to serve incoming traffic, and (ii) the liveness probe serves as a periodic check of the operation of the container and ensures it remains alive. During the instantiation of pods, Kubernetes periodically probes the readiness of their containers until they return a positive response, issuing GET requests on the `/ric/v1/health/ready` URI, and only then will Kubernetes allow them to communicate with other pods. After the pods start running, Kubernetes periodically probes the liveness of their containers, issuing GET requests on the `/ric/v1/health/alive` URI. In case the liveness probe fails, Kubernetes considers the container unhealthy (due to a crash or bug) and then tries to restart the containers as a recovery measure [47].

The business logic between xApps can differ vastly, and so do their conditions for readiness and liveness. Therefore, the xApp developer must define their own handlers for responding to Kubernetes probes, for example, waiting to create entries in SDL before the xApp is ready to work or checking if the xApp has the necessary variables to continue working. We detail how to create custom REST handlers in Listing 38, which also shows how we can create HTTP responses with custom HTTP status codes and payloads. Any data received as an argument in the REST handler must be decoded as *UTF-8* strings before we can process it in Python, and any data we want to return in the HTTP response must be encoded as a valid JSON string. In addition, Kubernetes considers any response with a 2xx HTTP status code a positive response that the probe is successful, while any response with HTTP status codes 3xx, 4xx, or 5xx

indicates a negative response that the probe failed.

The xApp developer can leverage these custom handlers to expose internal information and functionality via REST, for example, creating custom URIs and handlers to provide easy access to the current xApp configuration via a GET request or accepting additional parameters via a POST request, as shown in Listing 39. In Section VIII, we detail how users of the Near-RT RIC can find the IP addresses of xApps that enabled support for REST and explain how to interact with them via the terminal. Moreover, for additional information on how to trigger HTTP requests from within the xApp itself, e.g., interfacing with Near-RT RIC components or other xApps via their REST interface, we refer the reader to the documentation of the Python Requests module [89]. For completeness, we refer the reader to our online repository [29], where we include the entire source code used on the examples in this section.

VII. XAPP CONTROL: MANAGING RANs

In this section, we describe how xApps can manage RANs by interacting with E2 Nodes through subscriptions. First, we discuss the E2 Nodes and their interaction with the Near-RT RIC, which is useful for xApp developers creating end-to-end development environments. Then, we detail the SMs, the subscription procedure, and the interaction between xApps and E2 Nodes. Finally, we show how xApps can subscribe to E2 Nodes, trigger events, set up actions, and react to indication messages with information from E2 Nodes.

A. E2 Nodes, Termination, and Setup

The E2 Nodes, whether the disaggregated O-CU, O-DU, and O-RU, or the monolithic O-gNodeB and O-eNodeB, interact with the Near-RT RIC via the E2 Interface, which exposes information and control over their internal state, enabling near-real-time control loops to manage the RAN. The communication over the E2 interface occurs through the E2AP, a protocol running on top of the SCTP that specifies a number of well-defined message types with different purposes and goals [12]. Each E2 Node can expose a number of RAN Functions related to the features and capabilities it supports, e.g., beamforming, power control, and RAN slicing [90]. Each RAN Function may have widely distinct APIs involving different actions, required parameters, and data structures. To this end, the interaction with the RAN Functions is structured in the form of a SMs [91], which combines the basic RIC Services provided by E2AP as building blocks to define more complex APIs for interacting with the E2 Nodes and leveraging their functionality (detailed in Section VII-C).

At the Near-RT RIC, the communication with E2 Nodes is intermediated through the E2Term, which serves as a translation component between the southbound SCTP protocol and the internal RMR messaging infrastructure, forwarding messages between E2 Nodes and the E2Mgr. Conversely, the E2Mgr is responsible for establishing, maintaining, and terminating connections with E2 Nodes, as well as updating the R-NIB inventory with information about existing E2 nodes and their available SMs. The E2Term and the E2Mgr play different roles in monitoring the E2 interface: the

E2Term monitors the status of the SCTP connection to the E2 Nodes for identifying sudden disconnections (and notifying the E2Mgr), and the E2Mgr monitors the status of the E2Term, sending periodic probes to for identifying errors. When an E2 Node starts, it performs an E2 Setup procedure, where it tries to register itself with a Near-RT RIC. The E2 Setup procedure creates an entry in the R-NIB using a unique identifier for the E2 Node, known as the inventory name. Only after the E2 Node is set up with the Near-RT RIC and registered in the R-NIB, the xApps can subscribe to and communicate with it, which we detail in the Section VII-C. For additional information about registration of E2 Nodes with the Near-RT RIC, we refer the reader to [92].

B. Service Models

The O-RAN Alliance provides several first-party SMs in their specifications, e.g., the RAN Control (RC), the Cell Configuration and Control (CCC), and the Key Performance Measurement (KPM) SMs [93]. The specification documents for each SM include: (i) an overview of the SM and the corresponding RAN Function, their services and capabilities; (ii) the formal description of the RAN Function and its supported actions (the RIC Services detailed in Section VII-C); (iii) the formal description of the RAN parameters, known as Information Elements (IEs), i.e., the data structures and data formats for each variable and arguments for the actions supported by the RAN Function; (iv) the structure of how the different actions are combined to form a standard interface descriptor, and the definition of the SM in the form of an ASN.1 document; and (v) their approach for handling unknown, unforeseen, and erroneous interactions and protocol data [94]. These specifications are useful for vendors and system integrators creating or testing E2 Nodes to ensure they abide by the standard interfaces in the SMs.

It is worth mentioning the importance of the ASN.1 document, which can be manually excerpted from the specifications or automatically extracted using scripts. It provides a practical definition of the SM, which can be used both for understanding the operation of the RAN Function and for compiling the code bindings to support the SM on an E2 Node. The ASN.1 document is essential for xApp developers interested in controlling the RAN, as xApps only interact with E2 Nodes through the standard interface defined in the SMs. It is possible to develop third-party SMs to enable custom functionality on E2 Nodes [95], but this is outside the scope of this tutorial. For information about custom SMs, we refer the reader to [96].

Depending on the vendor, model, and version, an E2 Node may possess multiple RAN Functions and support their corresponding SMs to expose different capabilities and services to the Near-RT RIC. For example, an E2 Node may support the KPM and the CCC SMs at the same time, exposing KPMs and adjusting the base stations' transmit power in near-real-time, respectively. Conversely, the Near-RT RIC is agnostic to SMs, a design choice that ensures the Near-RT RIC architecture and components remain general and futureproof as new SMs are developed over time. As part of this paradigm, only the

E2 Nodes and the xApps interacting with them should be aware of the SM's capabilities and parameters. To achieve this, the data exchanged between xApps and E2 Nodes is en/decoded according to the ASN.1 definition, and it is up to the xApp developer to en/decode data accordingly. We discuss the ASN.1 en/decoding in Python later in Section VII-E.

The E2AP protocol has been updated regularly with bug fixes and the inclusion of new features. However, some updates required a significant redesign, leading to breaking changes. For example, with the E2AP update to version 2.0, which included improved encoding and handling racing conditions, all SMs had to be updated based on the new E2AP to remain operational [90]. Accordingly, the SMs themselves have been updated over the years to expose new capabilities and improve interoperability with RAN Functions from different vendors. These changes have led to differences in supported attributes or message formats between SM versions, which can impact compatibility with older xApps and RAN Functions. Therefore, it is fundamental for the xApp developer to ensure they are using the correct version of the SM to interface with the RAN Functions of the intended base stations done by checking the Object Identifier (OID) of the SM [94], i.e., a universally unique string that identifies all SMs and includes their version (detailed later in Section VII-E).

C. E2 Subscriptions

In an O-RAN deployment, multiple xApps may consume data, take control decisions, or respond to events of different RAN Functions on several E2 Nodes. To handle this many-to-many relationship, the interaction between xApps and E2 Nodes follows a publish-subscribe communication pattern intermediated by the SubMgr [12], as illustrated in Fig. 11. In the following, we detail how the SubMgr facilitates the communication between xApps and E2 Nodes, how xApps handle subscriptions, and the different RIC Services.

1) *Communication between xApps and E2 Nodes:* The xApps interact with the SubMgr via REST to create subscriptions to specific E2 Nodes. A subscription is created based on (i) the inventory name of the specific E2 Node, referred to as the *meid* in Section VI-A, (ii) the *RANFunctionID* that identifies the RAN Function the xApp intends to interface and the corresponding SM, and (iii) the desired RIC Service that defines the intended action to be set up at the E2 Node (detail later in Section VII-C3). Moreover, we examine the structure of the Subscription Request, its required fields, and how to create and delete subscriptions using the Python xApp Framework in the next subsection.

Upon receiving a Subscription Request, the SubMgr interacts with the RtMgr to create an RMR route between the xApp and the E2 Node. In turn, the RtMgr generates a new RMR routing entry for this subscription and distributes it to the E2Term and the new subscribed xApp. The RMR routes related to the subscription use the special *subid* and *meid* fields, introduced earlier in Section VI-A. The *subid* is generated by the SubMgr to identify that particular subscription between an xApp and E2 Node, and in the context of subscriptions, the *meid* is the inventory name of

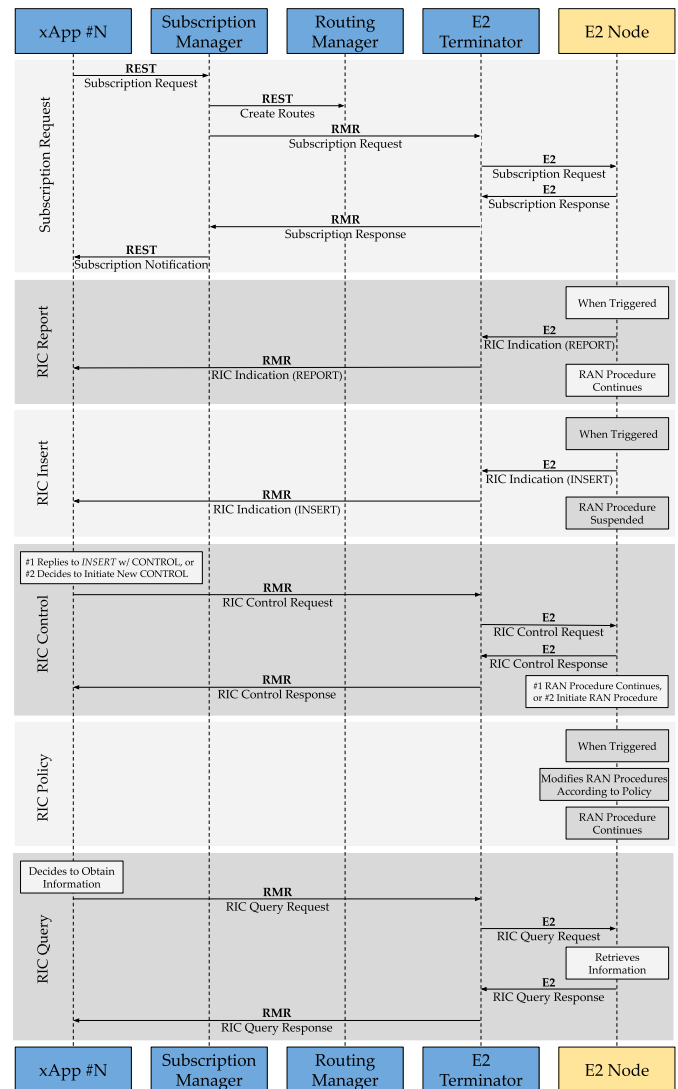


Fig. 11: Communication between xApps and E2 Nodes intermediated by the SubMgr, showing the interactions with Near-RT RIC components to create subscriptions (top). Each type of subscription behaves differently (detailed in Section VII-C3), be it reporting information or requesting control decisions when events trigger, waiting for control decisions or autonomously handling it, or reacting to queries on demand.

the intended E2 Node [97]. On the one hand, the routing of messages from xApps to E2 Nodes uses the *meid*: the xApp sets the *meid* in the message that is used by RMR to identify the correct endpoint of the E2Term that is interacting with the corresponding E2 Node. Upon receiving the RMR message, the E2Term translates it to the E2AP protocol and forward it to the respective E2 Node [98]. On the other hand, the routing of messages from E2 Node to xApps uses the *subid*: upon receiving an E2 message from an E2 Node, the E2Term translates it to RMR and forwards it to the xApp with the corresponding *subid*. The RMR identifies the endpoint of the xApp based on the *subid* populated in the message by the E2Term. If the Subscription Request fails at any stage, the SubMgr deletes any routes created and returns a message to

the xApp indicating the reason for failure, e.g., invalid meid or unsupported RANFunctionID, to the xApp.

2) *Handling Multiple Subscriptions*: The xApps can use the subid field to operate over their active subscriptions, such as listing active subscriptions, obtaining information about them, updating subscription parameters, or deleting them. If multiple xApps subscribe to the same information from the same E2 Node, e.g., using the KPM SM to obtain KPMs from the same E2 Node, the SubMgr merges the multiple subscriptions and only appends new subids to the existing RMR routes [98]. In this case, operations such as updating or deleting a particular subscription only affect the specific xApp, not the entire routes, e.g., a delete operation removes the subid related to the given xApp instead of deleting the RMR routes entirely. It is important to note that existing subscriptions remain active and persist even if the xApp pod is restarted (due to crashes, updates or rollbacks). Therefore, it is expected that the xApp either (i) gracefully handles errors and signals to delete existing subscriptions before stopping, or (ii) stores the subid of its active subscriptions on SDL and re-sends Subscription Requests to update its subscriptions upon booting again. The subids are useful because xApps can have multiple subscriptions to (i) the same E2 Nodes for interfacing with different SMs, and (ii) different E2 nodes for interfacing with multiple base stations. In addition, xApps are required to generate an integer to identify each of their multiple subscriptions locally, known as the XappEventInstanceId [98].

3) *RIC Services*: Each RAN Function may support different actions, i.e., the RIC Services defined in the E2AP specification, that allow xApps to instruct the E2 Nodes how to report information and/or control RAN procedures via subscriptions [94]. Each RIC Service, i.e., REPORT, INSERT, CONTROL, POLICY, and QUERY, operates in different manners and is better suited to cater to different use cases and applications. Each RIC Service contains RAN Function-specific data structures, i.e., the IEs encoded according to the ASN.1 of the corresponding SM. Therefore, xApps managing the RAN must understand the particularities of the SM of each intended RAN Function and en/decode data to/from ASN.1 accordingly. In the following, we detail the RICServices illustrated in Fig. 11 and how they operate.

REPORT: The xApp sends a subscription message instructing the E2 Node to report particular information according to a specified condition, e.g., trigger condition or periodic interval, using the REPORT message. This RIC Service is asynchronous and does not require a response.

INSERT: The xApp sends a subscription message instructing the E2 Node to suspend a particular RAN procedure, e.g., handover or attachment, according to a specified trigger condition, and requests control guidance from the Near-RT RIC using an INSERT message. This RIC Service is synchronous and requires a response within a predefined time limit, or it executes a specific subsequent action if an xApp does not respond in time.

CONTROL: The xApp sends a CONTROL message instructing the E2 Node to initiate or resume an associated RAN procedure, e.g., power control or RAN slicing. This

RIC Service is synchronous and requires a CONTROL acknowledgment or failure message from the E2 Node.

POLICY: The xApp sends a subscription message instructing the E2 Node on a policy with specific procedures for reacting autonomously to a particular trigger condition, e.g., scheduling directives. This RIC Service is asynchronous and does not require a response from the xApp.

QUERY: The xApp sends a QUERY message to the E2 Node to request information about the RAN or the associated UEs. After each request, the E2 Node issues a single QUERY Response with the information or an error message.

Such diversity in how the RIC Services operate enables xApps to create distinct types of subscriptions for controlling E2 Nodes in widely different manners, for example, requesting a periodic reporting of certain KPMs, consulting the xApp on how to react to particular events, or creating rules for reacting to particular events autonomously. For additional information about the RIC Services and the steps associated with their E2AP procedures in the Near-RT RIC and E2 Nodes, we refer the reader to the E2AP documentation [90].

D. End-to-end Testing & Development Environment in Python

In the following, we detail how the xApp developer can leverage simulated E2 Nodes to create an end-to-end testing and development environment. Next, we discuss the current limitations to perform subscriptions with the xApp Python Framework, as well as workarounds to perform subscriptions directly with the SubMgr through its REST interface.

1) *Simulating E2 Nodes*: The OSC provides a simulated E2 Node, known as the E2Sim, as a tool for testing the operation of the E2Term and E2Mgr and facilitating the xApp development process [71]. The E2Sim is an SCTP client that implements the E2AP protocol, which allows the testing of the E2 Setup procedure with the E2Term and E2Mgr, the creation of the RAN inventory in the R-NIB, and the subscription to E2 Nodes by xApps [99]. By using the E2Sim, xApp developers can create an end-to-end development environment in software, including xApps, Near-RT RIC components, and simulated E2 Nodes, for testing and validating all the capabilities and interfaces used by their xApps, including RMR messaging, A1 Policies, SDL storage, and E2 subscriptions.

The xApp developer can manually run the E2Sim as a Docker container or as a Kubernetes pod part of its Near-RT RIC cluster using Helm. For instructions on installing and setting up the E2Sim, we refer the reader to its official documentation [100]. The upstream E2Sim provided by the OSC supports the KPM SM, which exposes different metrics about the base station and its UEs using the REPORT type of subscription. In this case, the E2Sim streams metrics based on a trace file, the reports.json, which can be edited by the xApp developer before creating the xApp's Dockerfile to use custom or artificial data. The E2Sim uses the KPM SM to encode the metrics to ASN.1 and stream RAN telemetry to the subscribed xApps [99]. In the remainder of this section, we use the E2Sim and its supported Key Performance Measurement to demonstrate how xApps can perform subscriptions to control E2 nodes in Python. For

additional information about the E2Sim, we refer the reader to its official documentation and repository [71].

2) *Subscriptions in Python*: The different xApp Frameworks provided by the OSC for developing xApps in different programming languages abstract a number of interfaces and automate many interactions with Near-RT RIC components to simplify the xApp development process. However, due to the open-source nature of the OSC, the xApps Frameworks receive different levels of attention from the community and, hence, possess distinct subsets of features or API versions (discussed later in Section X). For example, the STSL is currently only supported by the Go xApp Framework. The API for subscribing to E2 Nodes has undergone significant changes on the F Release of the Near-RT RIC, with the migration of the subscription management operations, e.g., creation, query, update, and delete, from RMR to REST (detailed in Section VII-E). These breaking changes across Near-RT RIC releases were propagated to the C++, Go, and Rust xApp Frameworks so they could continue subscribing to E2 Nodes and managing the RAN. While the Python xApp Framework has received initial support for subscriptions via REST, at the time of writing, we observe that (i) it does not provide an approach for en/decoding ASN.1 from/to Python data objects, and (ii) its HTTP methods for interacting with the SubMgr employ the snake case convention, whereas the SubMgr expects HTTP requests in camel case. Consequently, despite receiving several updates since the F release, the subscription API of the Python xApp Framework remains incompatible with the SubMgr and incapable of subscribing to E2 Nodes.

Without loss of generality, we can leverage some of the existing functionality provided by the Python xApp Framework and the lessons learned in Section VI to develop xApps in Python that can subscribe to E2 Nodes and control the RAN by (i) en/decoding ASN.1 data structures from/to Python objects using external libraries, and (ii) interacting with the SubMgr via REST directly by creating HTTP requests. In the event that the Python xApp Framework is updated and its subscription API is fixed, the principles therein will remain useful to inform the reader how the subscription-related data structures and procedures are handled under the hood.

We leverage the Python PyCrate module to en/decode ASN.1 data structures from/to Python objects. Based on the ASN.1 documents from the E2AP and E2SM protocols, and from the intended SM, PyCrate can generate a Python representation of the SM. This representation contains Python methods for en/decoding data from/to ASN.1 according to the SM's standard interface description. For additional information about the utilization of the PyCrate module, we refer the reader to its official documentation [101]. In the next subsection, we detail how we can leverage PyCrate and manually create HTTP requests to complement the functionality of the Python xApp Framework to subscribe to E2 Nodes and control the RAN.

E. Controlling E2 Nodes using the Python xApp Framework

In the following, we discuss how the xApp developer can effectively interact with E2 Nodes using the Python xApp Framework. First, we demonstrate how to set up an xApp to

```

1  # Function called after the constructor
2  def _post_init(self, rmr_xapp):
3      ...
4
5      # Create Subscriber Object
6      self._submgr = NewSubscriber(
7          uri=<SubMgr_URL>,
8          local_port=<xApp_HTTP_Port>,
9          rmr_port=<xApp_RMR_Route_Port>
10     )
11
12     # Register Notification Callback Handler
13     self._submgr.ResponseHandler(
14         responseCB=self._subscription_notif)
15
16     # Hold active subscriptions
17     self._subscriptions = []
18     # Counter to identify subscriptions
19     self._event_instance = 0
20
21     # Iterate list of registered gNodeBs
22     for gnb in xapp.get_list_gnb_ids():
23         gnb_info = rmr_xapp.GetNodeB(
24             gnb.inventory_name)
25
26     # Iterate list of RAN Functions
27     for ran_function in
28         ↪ gnb_info.ran_functions:
29         # Check for matching OID of the KPM
30         if ran_function.oid == \
31             "1.3.6.1.4.1.53148.1.2.2.2":
32
33             # Subscribe to gNodeB
34             self._send_subscription_request(
35                 gnb.inventory_name)

```

Listing 40: Example of a `post_init` method where we register a callback for handling Subscription Notifications, iterate over the list of registered E2 Nodes, and subscribe to one of them according to their available RAN Functions.

subscribe to E2 Nodes. Next, we show how to create Subscription Requests with the SubMgr via REST and encode data to ASN.1. Then, we detail how to react to subscription indication messages and how to decode data from ASN.1. Finally, we show how xApps can operate over their subscriptions.

1) *Setting Up the Subscriptions*: The xApp developer can leverage the SubMgr's REST interface to interact with it directly to send Subscription Requests for creating, modifying, and deleting subscriptions to E2 Nodes. In this case, the SubMgr also interacts with the xApp via REST to send Subscription Notification messages containing the `subid` if the Subscription Request was successful or the type and reason for errors otherwise. To facilitate the handling of Subscription Notification messages, we can avail from the `NewSubscriber` object from the Python xApp Framework, which uses the SubMgr's URL, as well as the xApp's HTTP and RMR route port, to create an HTTP server configured to receive requests from the SubMgr, as shown in Listing 40. The `NewSubscriber` object allows us to register a callback to handle Subscription Notifications, which we detail later in Section VII-E3. It is also strongly recommended that xApps (i)

```

1  # Custom method for creating subscriptions
2  def _send_subscription_request(self, meid):
3
4      # Create trigger condition ASN.1 encoded
5      encoded_trigger = <Detailed in Lst. 43>
6      # Create action definition ASN.1 encoded
7      encoded_action = <Detailed in Lst. 44>
8
9      # Increment counter
10     self._event_instance += 1
11
12     # Prepare Subscription Request Payload
13     sub_payload = <Detailed in Lst. 42>
14
15     # Send POST request to the SubMgr
16     response = requests.post(
17         <SubMgr_URL> + "/ric/v1/subscriptions",
18         json=sub_payload
19     )
20
21     # Handle HTTP Response
22     if response.status_code == 201:
23         self.logger.debug("Subscription
24             ↳ Request Success!")
25
26     else:
27         self.logger.debug("Subscription
28             ↳ Request Failure!")

```

Listing 41: Example of a custom method for creating Subscription Requests via the SubMgr's REST interface.

store the list of subids of their active subscriptions, keeping them in persistent storage via SDL and/or in memory, e.g., using a global `self._subscriptions` variable; and (ii) create a monotonic counter to identify each of their multiple subscriptions locally, the `XappEventInstanceId`.

An xApp can subscribe to any E2 Node registered on the Near-RT RIC. However, the xApp may decide to filter the pool of E2 Nodes to identify the subset that supports the RAN Functions it intends to control. The xApp developer can accomplish this by inspecting the R-NIB to iterate over the list of E2 Nodes, parsing their RAN Functions, and checking their vendor-specific `RANFunctionIDs` and/or their `OIDS`, as shown in Listing 40. Finally, we can subscribe to the matching E2 Node(s) that support the functionality we want to control. For information about the structure of the `OID`, the meaning of each field, and the matching list between `OIDS` and `SMs`, we refer the reader to Table 5-2 of the official documentation about the E2SM [94].

2) *Creating Subscription Requests*: We can subscribe to a given E2 Node by issuing a POST request to the SubMgr's REST interface, containing a JSON payload that defines the Subscription Request, as shown in Listing 41. This JSON payload contains two fields encoded in ASN.1 according to the corresponding SM: the trigger condition and the action definition. In the following, we first discuss the structure and content of the JSON payload, detailed in Listing 42. Then, we overview the details of the trigger condition and action definition, and how to encode them in ASN.1.

The Subscription Request can create or modify an existing

```

1  {
2      "SubscriptionId": "",
3      "ClientEndpoint": {
4          "Host": <xApp_URL>,
5          "HTTPPort": 8080,
6          "RMRPort": 4560
7      },
8      "Meid": <inventory_name>,
9      "RANFunctionID": <RANFunctionID>,
10     "E2SubscriptionDirectives": { # Optional
11         "E2TimeoutTimerValue": 2,
12         "E2RetryCount": 2,
13         "RMRRoutingNeeded": True
14     },
15     "SubscriptionDetails": [
16         {
17             "XappEventInstanceId":
18                 ↳ self._event_instance
19             "EventTriggers": [
20                 <ASN.1 Event Definition> ],
21             "ActionToBeSetupList": [
22                 {
23                     "ActionID": 1,
24                     "ActionType": <RIC Service>,
25                     "ActionDefinition": [
26                         <ASN.1 Action Definition> ],
27                     "SubsequentAction": {
28                         "SubsequentActionType":
29                             ↳ "continue",
30                         "TimeToWait": "w10ms"
31                     }
32                 }
33             ]
34         }
35     ]
36 }

```

Listing 42: Subscription Request payload structure.

subscription, as specified by the `SubscriptionId` field: when creating a new subscription, it is an empty string, whereas when modifying an existing subscription, it uses the subid of the target subscription. The SubMgr also needs information about the xApp's RMR and HTTP endpoints, i.e., the RMR service's data port and the HTTP service's URL and port, to (i) instruct the RtMgr to create or update RMR routes related to the subscription and (ii) reply to the xApp with a Subscription Notification via REST about the subscription result. Next, the xApp must specify which E2 Node the xApp wants to subscribe to, based on its inventory name, and which RAN Function it wants to control, according to its `RANFunctionID`. The xApp developer can also configure optional directives related to this subscription, e.g., the duration of a timer to wait until the SubMgr receives a Subscription Response from the E2 Node, the number of Subscription Request retries from the SubMgr to the E2 Node, and whether the RtMgr needs to create or update RMR routes.

The subscription details describe in what manner that given subscription controls the E2 Node. It contains (i) the `XappEventInstanceId` to identify that given subscription locally at the xApp; (ii) a list of trigger conditions in ASN.1, which specifies when the given actions occur, e.g., periodically

```

1 event_definition = {
2     "eventDefinition-formats":
3     ("eventDefinition-Format1",
4     ↪ {"reportingPeriod": 1000})
5 }
6 trigger = E2SM_KPM_IEs.
7     ↪ E2SM_KPM_EventTriggerDefinition
8 trigger.set_val(event_definition)
9 encoded_trigger = trigger.to_aper()

```

Listing 43: Creating an event trigger condition for the Key Performance Measurement SM (setting up a periodic report) and its encoding to ASN.1 using PyCrate.

```

1 action_definition = {
2     "actionDefinition-formats": (
3     "actionDefinition-Format1", {
4     "measInfoList": [
5     { "measType":
6     ↪ ("measName",
7     ↪ "DRB.PerDataVolumeDLDist.Bin"),
8     "labelInfoList":
9     ↪ [{"measLabel": {"noLabel": "true"}}],
10     },
11     ...
12     ],
13     "granulPeriod": 1000 },
14 ),
15 "ric-Style-Type": 1,
16 }
17 action = E2SM_KPM_IEs.
18     ↪ E2SM_KPM_ActionDefinition
19 action.set_val(action_definition)
20 encoded_action = action.to_aper()

```

Listing 44: Creating an action definition for the Key Performance Measurement SM (specifying KPMs to report) and its encoding to ASN.1 using PyCrate.

or when a given variable reaches a threshold; and (iii) a list of actions to be set up at the E2 Node, including an ID for each action (used to notify the xApp about the status of each action set up), the type of RIC Service, e.g., REPORT or INSERT, a list of action definitions in ASN.1, which represent what is executed at the E2 Node, e.g., the metrics to be reported or the parameters to be changed. In the case of INSERT and CONTROL actions, the xApp can specify how the E2 Node will handle the RAN procedures (continuing or halting) if the xApp does not reply within a given time to wait.

Most parameters in the Subscription Request JSON payload are in plaintext and independent of the SM. However, the trigger conditions and action definitions are ASN.1 data objects that depend on the RAN Function and its corresponding SM. To find information about the supported trigger conditions and action definitions, as well as their formats and IEs, the xApp developer must refer to the SM's specifications. The PyCrate representation of the SM provides methods for encoding Python data structures to ASN.1 format, as long as they abide by the strict structure of the IEs in the ASN.1 document. We

```

1 # Custom method to handle Notifications
2 def _subscription_notif(self, name, path,
3     ↪ data, ctype):
4     # Convert the JSON string to Python
5     python_data = json.loads(data)
6
7     # Extract the subid from the Notification
8     subid = python_data["SubscriptionId"]
9     # Store the new subscription
10    self._subscriptions.append(subid)
11
12    # Extract useful information
13    sub_inst= python_data[
14    ↪ "SubscriptionInstances"][0]
15    xapp_event_instance =
16    ↪ sub_inst["XappEventInstanceId"]
17    e2_event_instance =
18    ↪ sub_inst["E2EventInstanceId"]
19    error_cause = sub_inst["ErrorCause"]
20    error_source = sub_inst["ErrorSource"]
21
22    # Respond to the POST request
23    response = initResponse()
24    return response

```

Listing 45: Example of a custom callback for handling Subscription Notification messages from the SubMgr.

```

1 # Register callback to handle Indications
2 self._rmr_xapp.register_callback(
3     ↪ self._indication_handler,
4     ↪ RIC_INDICATION)

```

Listing 46: Example on how to register a RMR message callback to handle RIC Indication messages from the E2 Node.

show an example of the KPM SM in Listings 43 and 44. The former shows how to instantiate an event definition format and create an event trigger condition to report measurements every 1000 ms. The latter shows how to instantiate an action definition format, create an action to report a list of measurements based on the names of the KPMs name and how to label them, define a measurement granularity period of 1000 ms, and configure a report style that defines how to collect KPMs, e.g., per UE, per group of UEs, or per base station. Both listings show the PyCrate methods for setting the values to the ASN.1 encoding and representing it using the APER format used by the KPM SM specification from O-RAN.

After issuing the POST request with the Subscription Request JSON payload, the SubMgr responds to the xApp with an HTTP status code 201 if the subscription was successful or with an error status code alongside the reason for failure.

3) *Handling Subscription Notifications:* After handling a Subscription Request, the SubMgr returns a Subscription Notification message to the xApp. This response message contains the subid generated by the SubMgr to identify the subscription, which the xApp should store in memory and/or persistent storage. In addition, the Subscription Notification message contains information about the result of the Subscription Request that is useful for debugging, in-

```

1 # Callback to Handle Indication Messages
2 def _indication_handler(self, rmxapp,
   ↳ summary, msg_buf):
3     # Get Message Payload
4     raw_data = summary[rmr.RMR_MS_PAYLOAD]
5
6     # Populate E2AP ASN.1 Data Structure
7     e2ap_pdu.from_aper(raw_data)
8     # Decode it from ASN.1 to Python
9     pdu = e2ap_pdu.get_val()
10
11    # Parse contents of the message
12    if pdu[0] == 'initiatingMessage':
13        # Traverse dicts to obtain protocol IEs
14        ies = e2ap_pdu.get_val_at(
15            ['initiatingMessage', 'value',
16             'RICIndication', 'protocolIEs'])
17        # Iterate over protocol IEs
18        for ie in ies:
19            # If it is the KPM SM message header
20            if ie['value'][0] ==
   ↳ 'RICIndicationHeader':
21                # Populate KPM ASN.1 Data Structure
22                header = E2SM_KPM_IEs.
   ↳ E2SM_KPM_IndicationHeader
23                header.from_aper(ie['value'][1])
24                data = header.get_val_at(
25                    ['indicationHeader-formats',
26                     'indicationHeader-Format1'])
27                self.logger.info(f"KPM Hdr {data}")
28
29            # If it is the KPM SM message payload
30            elif ie['value'][0] ==
   ↳ 'RICIndicationMessage':
31                # Populate KPM ASN.1 Data Structure
32                message = E2SM_KPM_IEs.
   ↳ E2SM_KPM_IndicationMessage
33                message.from_aper(ie['value'][1])
34                data = message.get_val_at(
35                    ['indicationMessage-formats',
36                     'indicationMessage-Format1'])
37                self.logger.info(f"KPM Msg {data}")

```

Listing 47: Example of a custom RMR callback for handling RIC Indication messages from an E2 Node using the KPM SM.

cluding (i) the xAppInstanceEvenID, so that the xApp knows which Subscription Notification this refers to, (ii) an E2EventInstanceId, to identify the particular subscription procedure at the E2 Node, (iii) the error cause, to explain the reason for failure, and (iv) the error source, i.e., the Near-RT RIC component or E2 Node that raised the error. An xApp can handle a Subscription Notification message by creating a POST request handler, as shown in Listing 45.

4) *Reacting to RIC Indications:* Depending on the type of subscription, i.e., REPORT or INSERT, the E2 Node may send a RIC Indication message to the xApp via RMR to report information or request a control decision, respectively. The E2 Node generates a RIC Indication message when an event condition in the subscription is triggered, e.g., periodically or when a variable reaches a threshold. To handle RIC Indication messages, the xApp can register an RMR callback to handle the RIC Indication mtype (12050), as shown in Listing 46.

```

1 # Custom method for querying subscriptions
2 def _query_subscriptions(self):
3     # Send GET request to the SubMgr
4     response = requests.get(
5         <SubMgr_URL> + "/ric/v1/
   ↳ get_xapp_rest_restsubscriptions/" +
   ↳ <xApp_URL>,
6     )
7
8     # If the query request was successful
9     if response.status_code == 200:
10        # List active subscriptions
11        for subid in response.json():
12            self.logger.info(f"Active
   ↳ Subscription ID: {subid}")

```

Listing 48: Example of a custom method for querying the SubMgr about the all active subscriptions of an xApp.

```

1 # Method to Unsubscribe from all E2 Nodes
2 def unsubscribe(self):
3     # Iterate over the active subscriptions
4     for subid in self._subscriptions:
5         # Unsubscribe to each E2 Node
6         data, reason, status =
   ↳ self._submgr.UnSubscribe(subid)
7
8     # Handle Unsubscribe Response
9     if status == 204:
10        self.logger.debug("Subscription
   ↳ Delete Successful!")
11    else:
12        self.logger.debug(f"Subscription
   ↳ Delete Failure! {status} {reason}")

```

Listing 49: Example of a method to delete all active subscriptions and release resources before stopping the xApp.

A RIC Indication message is encapsulated inside the RMR payload of an E2AP message, both of which are encoded in ASN.1. The outer E2AP message has a generic format, encoded in ASN.1 according to the E2AP specification, whereas the inner RIC Indication message has a very particular format, encoded in ASN.1 according to the SM and the type of subscription. The PyCrate representation of ASN.1 documents also provides methods for decoding ASN.1 data structures to Python objects, which we can leverage to decode the E2AP Message, extract the RIC Indication message, and decode its content. In Listing 47, we show an example of a tailored RMR message handler for RIC Indication messages from the KPM SM using the REPORT subscription type. This handler decodes the E2AP message using PyCrate, traverses through its payload, extracts and decodes the RIC Indication message, and logs its content. For additional examples of how to respond to RIC Indication messages of the INSERT subscription type and issue control messages to the E2 Nodes, we refer the reader to the SubMgr's official documentation [70].

5) *Operating over Active Subscriptions:* After the SubMgr returns to the xApp with a successful Subscription Notification, the subscription is active, and the xApp can perform operations on it, e.g., querying information about it, modifying

its parameters, or deleting it. The active subscriptions of an xApp remain active and persist across reboots due to updates, rollbacks, or crashes. More importantly, the actions set up on an E2 Node through a subscription stay in effect until (i) the E2 Node is removed from the Near-RT RIC or (ii) the xApp deletes the subscription to the E2 Node. To query the list of active subscriptions, e.g., for recovering the previous operational state after a crash, an xApp can issue a GET request to the SubMgr's REST interface, as shown in Listing 48. In this case, the SubMgr will respond with a list of subids of the active connections, if any. To modify a subscription, the xApp can send a new subscription request to the SubMgr, as shown earlier in Listing 41, using the subid of the active subscription in the SubscriptionId field and updated subscription details, e.g., different action definitions or event triggers. To delete a given subscription, e.g., for canceling actions no longer required as part of the xApp's business logic or releasing resources before gracefully exiting, an xApp can leverage the UnSubscribe method from the Python xApp Framework, as illustrated in Listing 49.

VIII. XAPP DEBUGGING: INSPECTING YOUR APPLICATION

In this section, we discuss debugging strategies to assist the xApp developer in identifying and fixing errors as part of the xApp development cycle. First, we examine approaches to debug the deployment of xApps. Then, we discuss debugging exposed services, open ports, and REST communications. Next, we show how to log information from xApps to debug their operation during runtime. Finally, we detail how to debug issues with RMR communications and SDL data storage.

A. Debugging xApp Deployment

During the xApp deployment, the AppMgr performs several steps to instantiate xApps: fetching Docker images from a Docker Registry, spawning their containers, configuring their resources, opening ports, and exposing services. Each step is prone to errors that can prevent the xApp from being deployed or working correctly. In addition, there can be issues when installing or restarting the Near-RT RIC Kubernetes cluster, which can prevent some of its components from starting and, consequently, impair the operation of the xApps. To debug the aforementioned errors, the xApp developer can use the commands in Listing 50 to identify potential issues. We detail these commands below:

List All Pods: This command lists all Kubernetes pods in the Near-RT RIC cluster, showing their namespaces, names, number of ready containers, number of restarts, and the time elapsed since their creation. It is useful for finding the names of the pods and getting a global view of the Near-RT RIC, as an operational cluster should have all its ricinfra and ricplt pods (detailed in Section III-B) in the *Running* state (except for the tiller-secret-generator pod as *Completed*).

List Pods in a Given Namespace: This command lists all pods that belong to a given namespace, which is useful to focus and inspect a particular aspect of the Near-RT RIC, such as the deployed xApps in the ricxapp namespace.

```

1 # List Kubernetes pods in all namespaces
2 kubectl get pods -A
3
4 # List Kubernetes pods in a given namespace
5 kubectl get pods -n <namespace>
6
7 # Example to list all running xApp pods
8 kubectl get pods -n ricxapp
9
10 # Describe information about a given pod
11 kubectl describe \
12 pod <pod_name> -n <namespace>
13
14 # Example to describe info a given xApp
15 kubectl describe pod
    ↪ ricxapp-examplexapp-6867f6c785-9pvc5
    ↪ -n ricxapp
16
17 # Print the log/stdout of a given pod
18 kubectl logs <pod_name> -n <namespace>
19
20 # Example of the log command for an xApp
21 kubectl logs
    ↪ ricxapp-examplexapp-6867f6c785-9pvc5
    ↪ -n ricxapp

```

Listing 50: Commands for interacting with the Near-RT RIC cluster to obtain information about the xApp Kubernetes pods.

Describe a Given Pod: This command provides in-depth information about a particular pod, including its container ID, Docker Registry's location, open ports, state, the ConfigMaps used to create its environment variables, the results from Kubernetes probes, the mounted volumes, and events that occurred during the pod's lifecycle. The events provide detailed information about the pods' creation, including when Docker images were fetched and when containers were created and started. The events also record information about failures, including why and when they occurred, if and why the pod is back-off restarting, and if it was evicted due to the lack of resources. For the complete list of events, we refer the reader to their comprehensive documentation in [73].

Print Logs of a Given Pod: This command displays the standard output generated by the given pod. It is useful to see the steps taken by the Python xApp Framework to start the xApp, including setting up the RMR library, loading the RMR route table, the content of the xApp configuration file loaded by the xApp, the registration with the AppMgr, the IP addresses of HTTP and RMR endpoints, and information about RMR messages exchanged with components of the Near-RT RIC and other xApps. In addition, this command displays any information logged by the xApp developer (detailed later in this section), e.g., debug information from different functions, data received from RMR and A1 from callbacks, or warnings and errors regarding the xApp's business logic.

These commands are helpful to debug issues related to the implementation of new xApps, including potential errors associated with accessing the Docker Registry, typos in the image

```

1 # List exposed services and open ports
2 kubectl get services -A
3
4 # Send HTTP/REST request
5 curl -X GET <xapp_IP>:<xapp_port>/<path>
6
7 # Example of HTTP/REST request
8 curl -X GET 10.107.57.43:8080/ric/v1/health

```

Listing 51: Commands for debugging exposed services and interacting with xApps or Near-RT RIC components via REST.

name and tag, which prevent the xApp from being instantiated, as well as the wrong content in the xApp descriptor, crashes in the xApp business logic or failures to register with the AppMgr, which will prevent the xApp from working correctly.

B. Debugging Ports, Services, and REST Communications

The xApps and Near-RT RIC components can have an optional REST interface for obtaining debug information about their internal state and passing control parameters during runtime. To communicate with an xApp via REST, the xApp developer must first ensure it has exposed the HTTP service and located the associated IP address and open port, which can be accomplished using the commands listed in Listing 51. The `get services` command lists all exposed services in the Near-RT RIC cluster, showing their namespaces, names, types, IP addresses inside the cluster, optionally an external IP, open ports and supported protocols, and the time elapsed since their creation. With this information, the xApp developer can use the `curl` command to issue custom HTTP requests to the desired xApp, following the HTTP endpoint structure detailed in Section VI-D. We refer the reader to the official `curl` documentation [102] for more information about the `curl` command, including instructions on transferring JSON objects and files.

These commands are helpful to debug issues related to connectivity and communication with xApps, including missing the HTTP service in the xApp descriptor, using an old IP address due to the pods' containers restarting, or using an incorrect port number, which would prevent the xApp developer (or users of the Near-RT RIC) to interact with the xApp via REST. There are also some circumstances in which the xApp developer may want to interact with the AppMgr to debug the onboarded xApps and their parameters. The AppMgr has a REST interface that the xApp developer (or the Near-RT RIC's users) can leverage to debug its operation during runtime. For information about their REST interfaces and supported calls, we refer the reader to the AppMgr's documentation in [103].

C. Logging xApp Data

The Python xApp Framework provides a streamlined logging API, ensuring that log entries adhere to a standardized format and are handled uniformly, which helps the xApp developer to debug and track the execution of their xApp's business logic and control loops [67]. To leverage the logging

```

1 # Create a logger object in a RMRXapp
2 def _post_init(self, rmr_xapp):
3     # Set log level
4     rmr_xapp.logger.set_level(<log_level>)
5     ...
6
7 # Create a logger object in an Xapp
8 def _entrypoint(self, xapp):
9     # Set log level
10    xapp.logger.set_level(<log_level>)
11    ...
12
13 # Example creation of a logger w/ DEBUG
14 xapp.logger.set_level(level.DEBUG)
15
16 # Logging messages w/ different severities
17 xapp.logger.debug(<msg>)
18 xapp.logger.info(<msg>)
19 xapp.logger.warning(<msg>)
20 xapp.logger.error(<msg>)
21
22 # Example of a log message
23 xapp.logger.error("Missing input
    ↪ parameter:" + str(my_parameters))

```

Listing 52: Commands for creating logger objects inside xApps, and logging messages with different severities.

API, the xApp developer must initialize a logger object with a default log level in their xApp's post-initialization function (for RMRXApps) or entrypoint function (for XApps), as shown in Listing 52. The log levels range from DEBUG, INFO, WARNING, and ERROR, which correspond to the growing severity levels of the logged messages. Then, the xApp developer can start logging messages in different parts of their xApp's business logic to log debug information, raise warnings, and throw errors. The log entries are displayed on the standard output of the xApp pod, and they contain the following fields:

- Timestamp:** When the log entry was created, in milliseconds.
- Criticality:** The severity level of the log entry.
- ID:** The name of the process that called the logging library.
- Message:** A custom message defined by the xApp developer.

The logging API is useful to debug the internal state of the xApp and track its control flow, allowing the xApp developer to easily display information about input parameters, results from conditional expressions, calling different methods, errors, exceptions, or any other tests and verifications. By generating comprehensive log messages as part of their business logic and inspecting them during execution using Kubernetes commands, the xApp developer can find valuable information about errors and crashes during development, as well as the users of the Near-RT RIC when in production.

D. Debugging RMR Communications

While the RMR library offers a high-speed, low-latency communication interface between xApps and components of the Near-RT RIC, availing of it requires a number of steps that can be error-prone and prevent xApps from communicating. Namely, some of those steps include: (i) adding the `mtypes` on the xApp file descriptor, both to configure the RMR library

```

1 # List services to search RtMgr's REST info
2 kubectl get svc -A
3
4 # Or obtain the RtMgr's HTTP endpoint IP
5 kubectl get svc -n ricplt \
6 service-ricplt-rtmgr-http \
7 -o=jsonpath='{.spec.clusterIP}'
8
9 # And the RtMgr's HTTP endpoint port
10 kubectl get svc -n ricplt \
11 service-ricplt-rtmgr-http \
12 -o=jsonpath='{.spec.ports[0].port}'
13
14 # Send request to RtMgr's REST interface
15 curl -X GET <RTMGR_IP>:<RTMGR_PORT>
16     ↪ /ric/v1/getdebuginfo
17
18 # Example request to obtain the current
19 # RMR routes displayed as a formatted JSON
20 curl -X GET \
21 10.110.179.180:3800/ric/v1/getdebuginfo |
22     ↪ jq .

```

Listing 53: Commands for interacting with the RtMgr to obtain debug information related to RMR communication.

and to route messages to the different containers that comprise the pod; (ii) creating a static route table file with the desired mtypes and RMR endpoints; and (iii) creating methods to send, receive, and reply to messages using the mtypes used in the previous steps. In the face of any errors, the xApp developer has a couple of strategies for debugging the RMR communication:

1) *Inspecting RMR Logs*: The RMR library also displays debug information on the standard output of the xApp pods, which include exchanged messages and error messages that indicate potential issues. For example, the error message (i) "Name does not resolve" is displayed when the RMR destination endpoint cannot be resolved, which occurs when the destination xApp is not running, has yet to be registered with the AppMgr, or has crashed; and (ii) "No route table entry for mtype=<given_mtype>" is displayed when the RMR library cannot find entry record to route the given mtype, which occurs when the sent message used a mtype that has not been registered on the static RMR route table file, or when the table itself cannot be found.

To obtain more information from the RMR logs, e.g., the current RMR data port, the location where the RMR library expects to find a static route table file, and the name of the RMR endpoint, the xApp developer can either: (i) set the RMR_LOG_LEVEL environment variable to 4 on the xApp's Dockerfile and re-deploy the xApp, which makes the RMR library log additional debug information [68]; or (ii) use kubectl commands to open a shell to the xApp pod (detailed in the next subsection) and parse its environment variables starting with the "RMR_" prefix.

2) *Querying the RtMgr*: Another approach to debugging the RMR communication between xApps and Near-RT RIC components is to query the RtMgr through REST to obtain the current RMR routes distributed and used inside the Near-

```

1 # List the configured storage classes
2 kubectl get storageclass -A
3
4 # List the configured persistent volumes
5 kubectl get pv -A

```

Listing 54: Commands for debugging the storage class and persistent storage configuration on the Near-RT RIC cluster.

```

1 # Open shell to a given pod in the cluster
2 kubectl exec -it <pod_name> \
3 -n <namespace> -- /bin/bash
4
5 # Example to open a shell to the DBaaS pod
6 kubectl exec -it \
7 statefulset-ricplt-dbaas-server-0 \
8 -n ricplt -- /bin/bash

```

Listing 55: Command for opening a shell to a Kubernetes pod and example on how to use it to access the DBaaS pod.

RT RIC, which is useful to debug the entire flow of messages from different sources. To interact with the RtMgr via its REST interface, the xApp developer must first find its IP address and open port (discussed in Section VIII-B), as shown in Listing 53. In possession of this information, the xApp developer can issue a GET request to the RtMgr's URI endpoint /ric/v1/getdebuginfo, for obtaining the current RMR routes distributed and used in the Near-RT RIC (expressed as a JSON string). For more information about the RtMgr's REST interface and other supported REST methods, we refer the reader to its documentation [104].

E. Debugging Persistent Storage

The Near-RT RIC provides persistent storage for the xApp pods, which they can leverage to store datasets, transfer data between each other, and save their internal states between reboots and upgrades. However, issues with the database infrastructure or data manipulation can prevent the xApps from performing read/write operations, as detailed below.

1) *Optional Features*: The InfluxDB component and the persistent volume are optional features of a Near-RT RIC cluster that can easily be overlooked during the Near-RT RIC installation. However, the lack of an InfluxDB component to store relational data prevents xApps from using the SDL API. Moreover, the lack of a persistent volume prevents the installation of the InfluxDB during the Near-RT RIC cluster installation. While InfluxDB and persistent volume are likely available in most production environments, the xApp developer may accidentally set up their O-RAN development environment with a Near-RT RIC without one or both.

To verify whether the InfluxDB component is installed and running, the xApp developer can use the commands in Listing 50 to list running pods on the ricplt namespace and confirm the existence of a pod with "influxdb" on its name. To verify whether the persistent volume is enabled, the xApp developer can use the commands on Listing 54, which should return an "nfs" storage class and a persistent volume

claim with the "influxdb" name. If the InfluxDB is not installed and/or the persistent volume is not present, the xApp developer needs to re-install their Near-RT RIC, making sure they perform the additional, one-time setup for the persistent volume and include the InfluxDB in the list of Near-RT RIC components to be installed [78].

2) *Inspecting the Data:* To debug the data storage during runtime and verify whether xApps are manipulating data as expected, the xApp developer can directly access the SDL database abstraction layer and inspect the data stored therein. To do so, the xApp developer must open a shell to the Near-RT RIC's DBaaS pod, using the commands in Listing 55. Once the xApp developer has shell access to the DBaaS pod, they can use the `sdlcli` command to obtain statistics about the database backend, check the health of the database backend, list database keys, and get and set values into the database. For more information about the `sdlcli` command and functionality available on the DBaaS abstraction layer, we refer the reader to their official documentation on [105].

IX. GOOD PRACTICES: LESSONS LEARNED

In this section, we share some good practices related to the initialization and registration of xApps, as well as their teardown and gracefully exiting, to facilitate their development and ensure correct operation in the Near-RT RIC.

A. Initialization and Registration

A critical step in the xApp lifecycle is the xApp registration for notifying the AppMgr and other Near-RT RIC components about the existence of a new xApp and the endpoints to communicate with it. This process requires the xApp to locate and load its configuration file to obtain essential information for the registration, i.e., its name, namespace, and interfaces. If the xApp cannot locate the configuration file or if there is any missing information, it prevents the registration with the AppMgr and results in errors on the xApp's standard output, e.g., "Cannot Read config file for xapp Registration". However, a registration failure leaves the xApp in an undefined state where the xApp pod remains running, but it cannot work correctly or interact with Near-RT RIC components.

To prevent an xApp from reaching this undefined state, the xApp developer has a few options at their disposal. For example, the RMRxApp implementation contains the `self._config_data` class parameter, which stores the configuration file data (this can be replicated in the Xapp implementation, as shown earlier in Listing 25). The xApp developer can check whether this parameter is an empty dictionary, which indicates that the Python xApp Framework could not find or load the configuration file. Moreover, the RMRxApp implementation contains the `self._keep_registration` boolean flag, which is automatically set to `False` if any issues prevented the registration process from starting. Furthermore, the xApp developer can also log the content of the `self._config_path` class parameter, which contains the configuration file path the Python xApp framework tries to open. After inspecting these variables, the xApp developer

```

1 # Called after the RMRxApp constructor
2 def _post_init(self, rmr_xapp):
3     # Set the log level of the xApp
4     rmr_xapp.logger.set_level(Level.DEBUG)
5
6     # Wait while the xApp is registered
7     sleep(5)
8
9     # Check for empty dict or False flag
10    if not bool(self._config_data) or not
11        ↪ self._keep_registration:
12        # Log config file path
13        rmr_xapp.logger.error(
14            "Could not load config file" + str(
15                self._config_path))
16        # Stop the xApp
17        rmr_xapp.stop()

```

Listing 56: Example of a `post_init` function, where we wait a few seconds and check whether the Python xApp framework was able to successfully load the configuration file.

may decide to debug and/or stop the xApp and avoid unintended behavior, as shown in Listing 56.

B. Signals and Teardown

The xApp pod can be terminated by (i) the users of the Near-RT RIC through the `dms_cli` to uninstall a given xApp; or (ii) Kubernetes itself due to the lack of resources or when the cluster reboots (in this case, the pods are automatically scheduled to restart after the cluster boots up). As part of this process, Kubernetes issues a `SIGTERM` signal to inform the pods of their impending termination so they can cease operations and gracefully exit. However, the xApp must listen to the `SIGTERM` signal to react to it and then manually perform procedures to exit gracefully, e.g., saving their internal state on persistent storage and, importantly, triggering the unsubscription and de-registration processes with the SubMgr and AppMgr, respectively. If the pod is still running after the grace period, Kubernetes issues a `SIGKILL` signal to terminate the pod forcefully. As mentioned in Section VI-A2, failure to de-register the xApp with the AppMgr leaves unresolved references and communication endpoints on the Near-RT RIC components, leading to undefined behavior on the system and preventing the xApp from being installed again until the entire Near-RT RIC cluster reboots, which can disrupt service and affect a large number of users.

Both the RMRxApp and Xapp implementations provide the `stop()` method for abstracting the xApp de-registration, as well as stopping its RMR message receiving loop and any other running threads. However, the xApp developer must ensure their xApp can catch these signals and react accordingly to call `stop()` method, which can be accomplished using signal handlers, as shown in Listing 57 for the Xapp. We can leverage the Python Signal module to register callbacks that will be invoked whenever the xApp receives the respective type of signal. In addition to the `SIGTERM` issued by Kubernetes, we also registered a callback for the `SIGINT`, which can be useful for xApp developers using an open shell to


```

1  # Register callbacks and initialize xApp
2  def __init__(self):
3      # Catch and react to the SIGTERM
4      signal.signal(signal.SIGTERM,
5                    self.signal_handler)
6      # Catch and react to the SIGINT
7      signal.signal(signal.SIGINT,
8                    self.signal_handler)
9
10     # Either instantiate the Xapp class
11     self._xapp = Xapp(
12         self._entrypoint,
13         rmr_port=4560
14     )

```

Listing 57: Example of the Xapp constructor that registers signal handler callbacks for reacting to different signals.

```

1  # Callback that catches registered signals
2  def signal_handler(self, signal, frame):
3      self._xapp.logger.info("signal handler
4      ↪ called")
5
6      # Let's first stop the entrypoint loop
7      self.shutdown = True
8
9      # Next, let's stop and de-register xApp
10     self._xapp.stop()

```

Listing 58: Example of a signal handler callback for catching and reacting to Linux signals and gracefully exiting xApps.

their xApps for testing and debugging in real-time. After these additional preliminary steps, the xApp developer can proceed with the initialization of their xApp as usual. The last step to catch and react to signals is to create callbacks for handling signals, as shown in Listing 58. These methods receive the signal type (in the form of an integer) and the stack frame (which helps to identify which thread was interrupted) as arguments, and can be used to toggle flags to shutdown control loops and call the stop method to exit the xApp gracefully.

As an xApp ceases to operate and gracefully exits, it is also beneficial to delete all active subscriptions to E2 Nodes. The benefits of this preemptive action are twofold: (i) it releases resources from the SubMgr and RtMgr, saving time spent resolving routing decisions, and (ii) it does not introduce unnecessary latency on the near-RT control loops, as lingering CONTROL and INSERT subscriptions cause RAN procedures to wait for a timeout until these subscriptions are deleted, even if the corresponding xApps were uninstalled.

At the time of writing, these additional considerations and procedures are not covered in the documentation provided by the OSC, despite being essential for ensuring the correct operation of both the xApps and the Near-RT RIC. Instead, we observe that existing xApp developers learn these lessons through trial and error throughout their xApp development cycles. Therefore, these lessons serve as invaluable information and good practices to facilitate newcomers in starting to prototype their xApps and accelerate their development.

X. OUTLOOK AND OPEN CHALLENGES

In this section, we discuss the current landscape of xApp development using the OSC O-RAN flavor and resources, new feature capabilities and standardization efforts, as well as open challenges for evaluating xApps in end-to-end scenarios.

A. Embedding of AI/ML to Manage RANs

One of the main aspects that attracted attention to O-RAN is its standardized platforms for (i) deploying custom control logic via third-party applications and (ii) embedding intelligence in mobile networks through AI/ML-based control loops. The OSC provided a reference implementation of the Near-RT RIC with support for third-party applications since its first A Release in 2019, which led to the community developing several xApps for accomplishing different tasks over the years, as discussed in Section III-D. However, the OSC's support for AI/ML has moved at a much slower pace. As a result, many researchers attempted to expand on the Near-RT RIC with AI/ML capabilities using a number of homebrew patches and custom implementations [106]–[108]. The OSC only started supporting AI/ML capabilities as part of its H Release in late 2022, with the initial inclusion of the AIMLFW [66]. This optional entity complements the Non- and Near-RT RICs and provides them with a complete pipeline for data preparation, AI/ML model training, AI/ML inference, and model management. Consequently, we expect future xApps leveraging AI/ML inference to rely on the AIMLFW instead of developing their own homebrewed solutions. Due to a considerable change in focus, we will detail how to leverage the AIMLFW for performing standard-compliant, AI/ML-based control loops to manage the RAN in a follow-up tutorial.

B. Features Across xApp Frameworks

The OSC offers different xApp Frameworks in distinct programming languages to assist developers in creating their xApps, as discussed in Section III-C. However, not all xApp Frameworks receive equal attention, maintenance, and updates from the community due to their open-source nature and volunteer contributions to the OSC. Consequently, certain xApp Frameworks may contain features not present in others, lack support for new capabilities, or suffer from breaking API changes that render some of their features unusable. For example, the Python xApp Framework lacks the Alarm API present in other xApp Frameworks [109], while the STSL API is currently only available in the Go xApp Framework. Furthermore, since the F Release, the OSC transitioned its xApp subscription API from RMR to REST, a change that has yet to be ported to the Python xApp Framework, as discussed in Section VII. Such inconsistency between features across xApp Frameworks poses significant challenges for xApp developers, who may need to adapt their workflows and adopt an xApp Framework based on its available features rather than their preferred programming language. This challenge also poses barriers to new xApp developers, who may need to learn a new programming language to avail from particular capabilities. We strongly believe that new features and breaking API

changes should be addressed consistently throughout all xApp Frameworks before rolling out new OSC releases.

C. New xApps and RMR Message Types

The xApps and Near-RT RIC components communicate with one another using different RMR *mtypes*, as discussed earlier in Section VI-A. Through the development of xApps, their distinct business logic, and different control loops, new xApps will likely require new *mtypes* to establish communication protocols between one another while avoiding message routing conflicts with existing xApps and their *mtypes*. However, the supported *mtypes* in the Near-RT RIC are hardcoded in the RMR source code as a lookup table of known *mtypes* [79]. Consequently, we identified that including a new *mtype* requires modifying the RMR source code and updating (or recompiling) the Near-RT RIC components and the xApps therein using the patched RMR library to recognize and avail from the new *mtypes*. The OSC performs these exact steps whenever a new xApp becomes a first-party, supported application on a new release, as can be seen by the inclusion of *mtypes* 30001, 30002, 30003, and 30010 to support the *Anomaly Detection*, *QoE Predictor*, *Traffic Steering*, and *Measurement Campaign* xApps, respectively. However, this process imposes significant challenges for third-party xApps, as their developers may not have access or permission to modify the RMR source code used by components of a Near-RT RIC cluster, or their installation may incur significant overheads to the Near-RT RIC users, which can include system administrators and network operators, but may not necessarily possess the required development skills. We believe that the OSC should provide an API for xApps to dynamically register new *mtypes* as part of their deployment process, which would considerably facilitate the deployment of new third-party xApps and lower the barrier to entry for new developers.

D. Security of the O-RAN Ecosystem, Near-RT RIC, and xApps

While current O-RAN components are operational and capable of working in unison for managing RANs, several critical security considerations remain to be addressed. For example, there is a lack of safeguards against misbehaving or malicious xApps, which can cause conflicts and degrade network performance [110]–[112], and protections against resource depletion and denial of service attacks, which can disrupt RAN control loops [97]. More fundamentally, there is an urgent need for Authentication, Authorization, and Accountability (AAA) capabilities for verifying xApp identities, controlling shared resource access, and tracing their activities [113]. Addressing these issues is essential to ensure network integrity in realistic settings, prevent unauthorized actions, and maintain accountability, especially in an open, multi-vendor, and multi-party environment [53]. There is a strong regulatory and industry interest in addressing these challenges, and future O-RAN releases are expected to introduce new security mechanisms, e.g., encrypted communication protocols, certificates, and key management systems. These updates will likely affect the existing xApp deployment process and development cycle,

as well as their associated xApp interfaces and APIs for conforming to the more strict security standards.

E. End-to-end Testing and Validation

For xApp developers looking into performing end-to-end testing and validation of their xApps to manage RANs, there are a few options to interact with E2 Nodes in different environments. Depending on their needs and expertise, xApps developers can avail from (i) actual software radio stacks, e.g., srsRAN [114] or OAI [1] and their recently introduced E2 interfaces to create softwarized O-RAN-compatible base stations running either with an emulated air interface or over the air; (ii) emulated E2 Nodes, e.g., the VIAVI's RIC Test [115] product or the ns-3 O-RAN E2 module [61], for emulating an arbitrary number of base stations on a parameterizable radio environment; and (iii) simulating E2 Nodes, e.g., the E2Sim mentioned earlier in Section VII, a simple simulator for testing the communication between the Near-RT RIC with a mock E2 Node, enabling the test and validation of xApps, SMs, and their interactions with the E2Term and E2Mgr. On the one hand, the more comprehensive software radio stacks and emulated E2 Nodes come from third-party software suppliers, requiring some learning curve and integration effort with the Near-RT RIC, and may have licensing considerations. On the other hand, the E2Sim is provided by the OSC and should work out of the box as part of the platform.

While useful for small-scale validations and sanity checks, the E2Sim possesses significant limitations that restrict the types and scope of end-to-end testing and validation, namely: (i) the E2Sim can only simulate a single base station at a time, limiting the scale of experiments and types of xApps would benefit from the E2Sim, e.g., handover operations; and (ii) the E2Sim only supports the RAN function ID 200, which serves to report KPMs back to the Near-RT RIC and hence makes the KPM the only supported SM. These limitations led to research efforts developing their own homebrewed extensions to the E2Sim to support other types of messages and SMs and expand its use cases [61]. However, these works tend to cater to particular scenarios and O-RAN releases, often missing documentation or not releasing their source code. We believe it would be beneficial to the community if the OSC provided extensive documentation for the utilization of the E2Sim to perform end-to-end testing and validation, as well as extended it with other RAN Functions and SMs, e.g., the RC and CCC.

XI. CONCLUSIONS

In this tutorial, we provided the first comprehensive guide on the development of xApps for managing RANs, from theory to practice. This paper addresses a significant gap in the literature and provides extensive material for the community to expedite and accelerate the development of xApps by academia and industry alike. First, we presented a theoretical foundation about the O-RAN ecosystem and its entities, as well as the Near-RT RIC components and its enabling technologies. Then, we introduced the APIs available to xApps, described how to design them through xApp descriptors, overviewed their lifecycle, and demonstrated how to control

their deployment. Next, we addressed the functionality available to xApps and explored how to communicate via RMR, leverage persistent storage via SDL, and react to external input via REST. In addition, we detailed how xApps can interface with E2 Nodes and their RAN functions via SMs and subscriptions. Moreover, we discussed debugging strategies to verify and validate the operation of xApps, as well as good practices to ensure their correct functioning. Finally, we discussed the current landscape of xApp developments, accompanied by new features, open challenges for xApp development, and suggestions for future improvements. It is worth mentioning that the supporting material used throughout the tutorial, i.e., the xApp descriptor and schema files, and source codes, can be found in our public online repository [29].

ACRONYMS

AAA	Authentication, Authorization, and Accountability
AC	Admission Control
AD	Anomaly Detection
AIMLFW	AI/ML Framework
BBU	Base Band Unit
C-RAN	Cloud RAN
CCC	Cell Configuration and Control
CLI	Command Line Interface
CP	Control Plane
DMS	Deployment Management Service
DRL	Deep Reinforcement Learning
DSP	Digital Signal Processor
FCAPS	Fault, Configuration, Accounting, Performance, and Security
FFT	Fast Fourier Transform
FPGA	Field Programmable Gate Array
GNN	Graph Neural Network
IE	Information Element
IFFT	Inverse FFT
KPM	Key Performance Measurement
MEID	Managed Entity ID
MHO	Mobile Handover
ML	Machine Learning
MLB	Mobility Load Balancing
MNO	Mobile Network Operator
Near-RT RIC	Near Real-Time RAN Intelligent Controller
NIB	Network Information Base
Non-RT RIC	Non Real-Time RAN Intelligent Controller
O-RAN	Open Radio Access Network
OAI	OpenAirInterface
O-Cloud	Open Cloud
OID	Object Identifier
ONAP	Open Networking Automation Platform
OSC	O-RAN Software Community
OSM	Open Source MANO
PDCCP	Packet Data Convergence Protocol
PLMN	Public Land Mobile Network
RAN	Radio Access Network
RC	RAN Control

REST	Representational State Transfer
RF	Radio Frequency
RIC	RAN Intelligent Controller
RMR	RIC Message Router
R-NIB	Radio-NIB
RRC	Radio Resource Control
RRU	Remote Radio Unit
RSM	RAN Slice Manager
SDAP	Service Data Adaptation Protocol
SDL	Shared Data Layer
SDN	Software-Defined Networking
SM	Service Model
SMO	Service Management and Orchestration
STSL	Shared Time Series Layer
TS	Traffic Steering
UE	User Equipment
UE-NIB	User Equipment-NIB
UP	User Plane
VES	Virtual Event Streaming

REFERENCES

- [1] F. Kaltenberger *et al.*, "OpenAirInterface: Democratizing innovation in the 5G Era," *Elsevier Computer Networks (ComNet)*, vol. 176, p. 107284, 2020.
- [2] G. O. Pérez *et al.*, "5G New Radio Fronthaul Network Design for eCPRI-IEEE 802.1 CM and Extreme Latency Percentiles," *IEEE Access*, vol. 7, pp. 82 218–82 230, 2019.
- [3] J. F. Santos *et al.*, "Virtual Radios, Real Services: Enabling rANaaS Through Radio Virtualisation," *IEEE Transactions on Network and Service Management (TNSM)*, vol. 17, no. 4, pp. 2610–2619, 2020.
- [4] O-RAN Alliance, "O-RAN Alliance," 2022, accessed: 24/07/23. [Online]. Available: <https://www.o-ran.org/>
- [5] D. Johnson *et al.*, "Open source RAN slicing on POWDER: A top-to-bottom O-RAN use case," in *ACM International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2021, pp. 507–508.
- [6] M. Polese *et al.*, "CoO-RAN: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms," *IEEE Transactions on Mobile Computing (TMC)*, 2022.
- [7] A. Kliks *et al.*, "Towards Autonomous Open Radio Access Networks," *ITU Journal on Future and Evolving Technologies (FET)*, vol. 4, Jun. 2023.
- [8] A. Garcia-Saavedra and X. Costa-Perez, "O-RAN: Disrupting the virtualized RAN ecosystem," *IEEE Communications Standards Magazine (COMSTD)*, vol. 5, no. 4, pp. 96–103, 2021.
- [9] S. D'Oro *et al.*, "OrchestRAN: Network Automation through Orchestrated Intelligence in the Open RAN," in *IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 2022, pp. 270–279.
- [10] S. Niknam *et al.*, "Intelligent O-RAN for beyond 5G and 6G wireless networks," in *IEEE Globecom Workshops (GC Wkshps)*, 2022, pp. 215–220.
- [11] L. Bonati *et al.*, "Intelligent Closed-loop RAN Control with xApps in OpenRAN Gym," *arXiv preprint arXiv:2208.14877*, 2022.
- [12] M. Polese *et al.*, "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges," *arXiv preprint arXiv:2202.01032*, 2022.
- [13] O-RAN Software Community. (Accessed 2024, Jun.) O-RAN Software Community. [Online]. Available: <https://o-ran-sc.org/>
- [14] F. A. Bimo *et al.*, "OSC Community Lab: The Integration Test Bed for O-RAN Software Community," *arXiv preprint arXiv:2208.14885*, 2022.
- [15] O-RAN Software Community. (Accessed 2024, Jun.) Current xApps. [Online]. Available: <https://wiki.o-ran-sc.org/display/RICA/Current+x+App>
- [16] M. S. Wani *et al.*, "Open RAN: A Concise Overview," *IEEE Open Journal of the Communications Society (OJ-COMS)*, 2024.
- [17] M. Polese *et al.*, "Empowering the 6G Cellular Architecture with Open RAN," *IEEE Journal on Selected Areas in Communications (JSAC)*, 2023.

- [18] L. M. Larsen *et al.*, "The Evolution of Mobile Network Operations: A Comprehensive Analysis of Open RAN Adoption," *Elsevier Computer Networks (ComNet)*, p. 110292, 2024.
- [19] J. Groen *et al.*, "Implementing and Evaluating Security in O-RAN: Interfaces, Intelligence, and Platforms," *IEEE Network*, 2024.
- [20] M. Hoffmann *et al.*, "Open RAN xApps Design and Evaluation: Lessons Learnt and Identified Challenges," *IEEE Journal on Selected Areas in Communications (JSAC)*, 2023.
- [21] M. Kouchaki and V. Marojevic, "Actor-Critic Network for O-RAN Resource Allocation: xApp Design, Deployment, and Analysis," in *IEEE Globecom Workshops (GC Wkshps)*, 2022, pp. 968–973.
- [22] O-RAN Working Group 2, "O-RAN AI/ML Workflow Description and Requirements," O-RAN Alliance, Tech. Rep., Oct. 2021.
- [23] O-RAN Working Group 3, "O-RAN Near-RT RIC Architecture," O-RAN Alliance, Tech. Rep., Mar. 2023.
- [24] Learn And Grow Community. (2024, Mar.) Series 1: Intro to Open RAN: A Beginner's Playlist. [Online]. Available: <https://www.youtube.com/playlist?list=PLClhZBCtSx9Zvt2uVpd2xhg0QogxNH>
- [25] IEEE. (2024, Dec.) IEEE 5G/6G Innovation Testbed. [Online]. Available: <https://testbed.ieee.org/our-platform/how-it-works/>
- [26] TIP Academy. (Accessed 2024, Jun.) Open RAN Curriculum. [Online]. Available: <https://www.tip.academy/open-ran-curriculum>
- [27] Intelify. (Accessed 2024, Jun.) O-RAN Hands-On Training. [Online]. Available: <https://courses.intelify.com/courses/o-ran-hands-on-training>
- [28] Rimedo Labs. (Accessed 2024, Jun.) Wireless Technical Courses. [Online]. Available: <https://rimedolabs.com/training>
- [29] J. F. Santos. (Accessed 2024, Sep.) xApp Development from Zero to Hero: Online Resources. [Online]. Available: https://github.com/santos-j/xapp_development_zero_to_hero
- [30] M. Kist *et al.*, "AIRTIME: End-to-end Virtualization Layer for RAN-as-a-service in Future Multi-service Mobile Networks," *IEEE Transactions on Mobile Computing (TMC)*, 2020.
- [31] L. Bonati *et al.*, "Intelligence and Learning in O-RAN for Data-driven NextG Cellular Networks," *IEEE Communications Magazine (ComMag)*, vol. 59, no. 10, pp. 21–27, 2021.
- [32] A. Checko *et al.*, "Cloud RAN for Mobile networks: A Technology Overview," *IEEE Communications Surveys & Tutorials (COMST)*, vol. 17, no. 1, pp. 405–426, 2014.
- [33] European Telecommunications Standards Institute. (2020, Dec.) Open Source MANO. [Online]. Available: <https://osm.etsi.org/>
- [34] The Linux Foundation. (2023, Nov.) Open Network Automation Platform. [Online]. Available: <https://www.onap.org/>
- [35] J. F. Santos *et al.*, "Breaking Down Network Slicing: Hierarchical Orchestration of End-to-End Networks," *IEEE Communications Magazine (ComMag)*, vol. 58, no. 10, pp. 16–22, 2020.
- [36] P. Berde *et al.*, "ONOS: Towards an Open, Distributed SDN OS," in *ACM Workshop on Hot Topics in Software Defined Networking (HotSDN)*, 2014, pp. 1–6.
- [37] R. Schmidt *et al.*, "FlexRIC: an SDK for next-generation SD-RANs," in *ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2021, pp. 411–425.
- [38] O. N. Foundation, "SD-RAN Platform," 2020, accessed: 24/07/23. [Online]. Available: https://docs.sd-ran.org/master/release_notes/sd_ran_1.4.html
- [39] J. Dürre *et al.*, "A Disaggregated 5G Testbed for Professional Live Audio Production," in *IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB)*, 2022, pp. 1–6.
- [40] G. Queirós *et al.*, "Autonomous Control and Positioning of a Mobile Radio Access Node Employing the O-RAN Architecture," in *IEEE Wireless On-Demand Network Systems and Services Conference (WONS)*, 2024, pp. 25–28.
- [41] M. V. Ngo *et al.*, "RAN Intelligent Controller (RIC): From Open-source Implementation to Real-world Validation," *Elsevier ICT Express*, 2024.
- [42] P. Liu *et al.*, "NIST Series TN 2311: Blueprint for Deploying 5G O-RAN Testbeds: A Guide to Using Diverse O-RAN Software Stacks," National Institute of Standards and Technology, Tech. Rep., 2024.
- [43] O-RAN Software Community. (Accessed 2024, Sep.) RIC VirtualBox VMs as Installation Hosts. [Online]. Available: <https://docs.o-ran-sc.org/projects/o-ran-sc-it-dep/en/latest/installation-guides.html#virtualbox-vms-as-installation-hosts>
- [44] O-RAN Software Community. (Accessed 2024, Nov.) New Installer. [Online]. Available: <https://github.com/o-ran-sc/ric-plt-ric-dep/tree/master/new-installer>
- [45] O-RAN Software Community. (Accessed 2024, Jul.) O-RAN Application SDK. [Online]. Available: <https://wiki.o-ran-sc.org/display/ORANSdk/Overview>
- [46] D. Merkel *et al.*, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," *Linux Journal*, vol. 239, no. 2, p. 2, 2014.
- [47] B. Burns *et al.*, *Kubernetes: Up and Running*. "O'Reilly Media, Inc.", 2022.
- [48] J. Shah and D. Dubaria, "Building Modern Clouds: Using Docker, Kubernetes & Google Cloud Platform," in *IEEE Computing and Communication Workshop and Conference (CCWC)*, 2019, pp. 0184–0189.
- [49] D. Johnson *et al.*, "NexRAN: Closed-Loop RAN Slicing in POWDER -A Top-to-Bottom Open-Source Open-RAN Use Case," in *ACM Workshop on Wireless Network Testbeds, Experimental Evaluation & Characterization (WiNTECH)*, 2021, p. 17–23.
- [50] L. Bonati *et al.*, "Intelligence and Learning in O-RAN for Data-Driven NextG Cellular Networks," *IEEE Communications Magazine*, vol. 59, no. 10, pp. 21–27, 2021.
- [51] H. Zhang *et al.*, "Federated Deep Reinforcement Learning for Resource Allocation in O-RAN Slicing," in *IEEE Global Communications Conference*, 2022, pp. 958–963.
- [52] A. Huff *et al.*, "RFT: Scalable and Fault-Tolerant Microservices for the O-RAN Control Plane," in *IFIP/IEEE International Symposium on Integrated Network Management (IM)*, 2021, pp. 402–409.
- [53] H. Wen *et al.*, "A Fine-Grained Telemetry Stream for Security Services in 5G Open Radio Access Networks," in *ACM International Workshop on Emerging Topics in Wireless (EmergingWireless)*, 2022, p. 18–23.
- [54] H. Lee *et al.*, "O-RAN AI/ML Workflow Implementation of Personalized Network Optimization via Reinforcement Learning," in *IEEE Globecom Workshops (GC Wkshps)*, 2021, pp. 1–6.
- [55] P. E. Iturria-Rivera *et al.*, "Multi-Agent Team Learning in Virtualized Open Radio Access Networks (O-RAN)," *Sensors*, vol. 22, no. 14, 2022.
- [56] F. Mungari, "An RL Approach for Radio Resource Management in the O-RAN Architecture," in *IEEE International Conference on Sensing, Communication, and Networking (SECON)*, 2021, pp. 1–2.
- [57] I. Rego *et al.*, "Prototyping near-real time RIC O-RAN xApps for Flexible ML-based Spectrum Sensing," in *IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 2022, pp. 137–142.
- [58] O. Orhan *et al.*, "Connection Management xAPP for O-RAN RIC: A Graph Neural Network and Reinforcement Learning Approach," in *IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2021, pp. 936–941.
- [59] Y.-C. Huang *et al.*, "Universal Vertical Applications Adaptation for Open RAN: A Deep Reinforcement Learning Approach," in *IEEE International Symposium on Wireless Personal Multimedia Communications (WPMC)*, 2022, pp. 92–97.
- [60] B. Agarwal *et al.*, "QoE-Driven Optimization in 5G O-RAN-Enabled HetNets for Enhanced Video Service Quality," *IEEE Communications Magazine (ComMag)*, vol. 61, no. 1, pp. 56–62, 2023.
- [61] A. Lacava *et al.*, "Programmable and Customized Intelligence for Traffic Steering in 5G Networks Using Open RAN Architectures," *IEEE Transactions on Mobile Computing (TMC)*, vol. 23, no. 4, pp. 2882–2897, 2023.
- [62] M. Alavirad *et al.*, "O-RAN Architecture, Interfaces, and Standardization: Study and Application to User intelligent Admission Control," *Frontiers in Communications and Networks*, vol. 4, 2023.
- [63] O-RAN Software Community. (Accessed 2024, Sep.) Python xApp Framework. [Online]. Available: <https://docs.o-ran-sc.org/projects/o-ran-sc-ric-plt-xapp-frame-py/>
- [64] A. Weissberger, "SNS Telecom & IT: Open RAN Intelligent Controller, xApps & rApps to reach \$600 Million by 2025," IEEE Communications Society Blog, Dec 2022. [Online]. Available: <https://techblog.comsoc.org/2022/12/24/sns-telecom-it-open-ran-intelligent-controller-xapps-rapps-to-reach-600-million-by-2025/>
- [65] O. Alliance, "RIC Applications (RICAPP)," 2024, accessed: 04/18/24. [Online]. Available: <https://wiki.o-ran-sc.org/pages/viewpage.action?pageId=1179662>
- [66] O-RAN Software Community. (Accessed 2024, Jul.) G Release. [Online]. Available: <https://wiki.o-ran-sc.org/display/REL/G+Release>
- [67] O-RAN Software Community. (Accessed 2024, Jul.) xApp Writer's Guide. [Online]. Available: https://wiki.o-ran-sc.org/download/attachments/17269011/xApp_Writer_s_Guide_v2.pdf
- [68] O-RAN Software Community. (Accessed 2024, Jul.) RIC Message Router – RMR. [Online]. Available: <https://docs.o-ran-sc.org/projects/o-ran-sc-ric-plt-lib-rmr/en/latest/user-guide.html>

- [69] O-RAN Software Community. (Accessed 2024, Jul.) Shared Data Layer. [Online]. Available: <https://docs.o-ran-sc.org/projects/o-ran-sc-ric-plt-sdl/en/latest/user-guide.html>
- [70] O-RAN Software Community. (Accessed 2024, Jul.) Subscription Manager. [Online]. Available: <https://docs.o-ran-sc.org/projects/o-ran-sc-ric-plt-submgr/en/latest/user-guide.html>
- [71] O-RAN Software Community. (Accessed 2024, Nov.) Sim E2 Interface. [Online]. Available: <https://github.com/o-ran-sc/sim-e2-interface>
- [72] O-RAN Software Community. (Accessed 2024, Sep.) On-boarding and Deploying xApps. [Online]. Available: <https://wiki.o-ran-sc.org/display/RICA/On-boarding+and+Deploying+xApps>
- [73] Docker Inc. (Accessed 2024, Sep.) Dockerfile Reference. [Online]. Available: <https://docs.docker.com/engine/reference/builder/>
- [74] Docker Inc. (Accessed 2024, Sep.) Docker Official Images. [Online]. Available: https://hub.docker.com/search?image_filter=official
- [75] Docker Inc. (Accessed 2024, Sep.) Docker Tag. [Online]. Available: <https://docs.docker.com/engine/reference/commandline/tag/#tag-an-image-for-a-private-registry>
- [76] Docker Inc. (Accessed 2024, Sep.) Docker Build. [Online]. Available: <https://docs.docker.com/engine/reference/commandline/build/>
- [77] Docker Inc. (Accessed 2024, Sep.) Docker Registry. [Online]. Available: <https://docs.docker.com/registry/>
- [78] O-RAN Software Community. (Accessed 2024, Sep.) RIC Installation Guide. [Online]. Available: <https://docs.o-ran-sc.org/projects/o-ran-sc-ric-plt-ric-dep/en/latest/installation-guides.html>
- [79] O-RAN Software Community. (Accessed 2024, Nov.) RIC_message_types.h. [Online]. Available: https://github.com/o-ran-sc/ric-plt-lib-rmr/blob/master/src/rmr/common/include/RIC_message_types.h
- [80] O-RAN Software Community. (Accessed 2024, Jul.) RMR LIBRARY. [Online]. Available: <https://docs.o-ran-sc.org/projects/o-ran-sc-ric-plt-lib-rmr/en/latest/rmr.7.html>
- [81] O-RAN Software Community. (Accessed 2024, Jul.) Anomaly Detection Use Case. [Online]. Available: <https://wiki.o-ran-sc.org/display/RICP/Anomaly+Detection+Use+Case>
- [82] O-RAN Software Community. (Accessed 2024, Nov.) RIC A1 Mediator. [Online]. Available: <https://github.com/o-ran-sc/ric-plt-a1>
- [83] O-RAN Software Community. (Accessed 2024, Jul.) Traffic Steering Use Case. [Online]. Available: <https://wiki.o-ran-sc.org/display/IAT/Traffic+Steering+Use+Case>
- [84] O-RAN Software Community. (Accessed 2024, Dec.) stslgo. [Online]. Available: <https://gerrit.o-ran-sc.org/r/admin/repos/ric-plt/stslgo>
- [85] S. Kukliński *et al.*, "On O-RAN, MEC, SON and Network Slicing Integration," in *IEEE Globecom Workshops (GC Wkshps)*, 2020, pp. 1–6.
- [86] O. N. Foundation, "ONOS-UENIB," 2023, accessed: 08/12/23. [Online]. Available: <https://docs.sd-ran.org/master/onos-uenib/README.html>
- [87] X. Zhao *et al.*, "Multi Independent Logical Cells under 5G Radio Access Network Sharing of Mobile Operators," in *IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB)*, 2022, pp. 1–5.
- [88] A. Rodriguez, "RESTful Web Services: The Basics," *IBM developerWorks*, vol. 33, no. 2008, p. 18, 2008.
- [89] R. V. Chandra and B. S. Varanasi, *Python Requests Essentials*. Packt Publishing Birmingham, UK, 2015.
- [90] O-RAN Working Group 3, "E2 General Aspects and Principles (E2GAP)," O-RAN Alliance, Tech. Rep., Nov. 2023.
- [91] M. Irazabal and N. Nikaein, "TC-RAN: A Programmable Traffic Control Service Model for 5G/6G SD-RAN," *IEEE Journal on Selected Areas in Communications (JSAC)*, 2023.
- [92] C.-F. Hung *et al.*, "Security Threats to xApps Access Control and E2 Interface in O-RAN," *IEEE Open Journal of the Communications Society (OJ-COMS)*, 2024.
- [93] O-RAN Alliance. (Accessed 2024, Jun.) O-RAN Software Specifications. [Online]. Available: <https://specifications.o-ran.org/specifications>
- [94] O-RAN Working Group 3, "E2 Service Model (E2SM)," O-RAN Alliance, Tech. Rep., Nov. 2023.
- [95] D. Johnson *et al.*, "NexRAN: Closed-loop RAN Slicing in POWDER-A top-to-bottom Open-source Open-RAN Use Case," in *ACM Workshop on Wireless Network Testbeds, Experimental evaluation & Characterization*, 2022, pp. 17–23.
- [96] O. N. Foundation, "How to Create Your Own SM?" 2024, accessed: 07/29/24. [Online]. Available: <https://docs.sd-ran.org/sdran-1.0/onos-e2-sm/docs/sm-howto.html>
- [97] V. K. Radhakrishnan, "Detection of Denial of Service Attacks on the Open Radio Access Network Intelligent Controller through the E2 Interface," Ph.D. dissertation, Virginia Tech, 2023.
- [98] O-RAN Software Community. (Accessed 2024, Nov.) Subscription Manager. [Online]. Available: <https://github.com/o-ran-sc/ric-plt-submgr>
- [99] A. Lacava *et al.*, "ns-o-ran: Simulating o-ran 5g systems in ns-3," in *ACM Workshop on ns-3 (WNS3)*, 2023, pp. 35–44.
- [100] O-RAN Software Community. (Accessed 2024, Oct.) E2 Simulator. [Online]. Available: <https://wiki.o-ran-sc.org/display/SIM/E2+Simulator>
- [101] PyCrate-Org. (Accessed 2024, Nov.) PyCrate. [Online]. Available: <https://github.com/pycrate-org/pycrate>
- [102] curl Project. (Accessed 2024, Jun.) curl Man Page. [Online]. Available: <https://curl.se/docs/manpage.html>
- [103] O-RAN Software Community. (Accessed 2024, Nov.) RIC xApp Manager. [Online]. Available: <https://github.com/o-ran-sc/ric-plt-apmng>
- [104] O-RAN Software Community. (Accessed 2024, Nov.) Routing Manager. [Online]. Available: <https://github.com/o-ran-sc/ric-plt-rtmng>
- [105] O-RAN Software Community. (Accessed 2024, Feb.) Database as a Service. [Online]. Available: <https://github.com/o-ran-sc/ric-plt-dbaas>
- [106] D. Overbeck *et al.*, "Data-Driven Proactive Uplink Slicing enabling Real-Time Control within an Open RAN Testbed," in *IEEE INFOCOM Workshops (INFOCOM WKSHPS)*, 2024.
- [107] J. S. Mallu *et al.*, "AI/ML Data-driven Control Loop for Managing O-RAN SDR-based RANs," in *IEEE INFOCOM Workshops (INFOCOM WKSHPS)*, 2023, pp. 1–2.
- [108] M. Tsampazi *et al.*, "A Comparative Analysis of Deep Reinforcement Learning-based xApps in O-RAN," in *IEEE Global Communications Conference (GlobeCom)*, 2023, pp. 1638–1643.
- [109] O. N. Foundation, "O-RAN Alliance," 2023, accessed: 04/18/24. [Online]. Available: <https://docs.o-ran-sc.org/projects/o-ran-sc-ric-plt-alarm-go/en/latest/user-guide.html>
- [110] A. Wadud *et al.*, "Conflict Management in the Near-RT-RIC of Open RAN: A Game Theoretic Approach," in *IEEE International Conferences on Internet of Things (iThings) and IEEE Green Computing & Communications (GreenCom) and IEEE Cyber, Physical & Social Computing (CPSCom) and IEEE Smart Data (SmartData) and IEEE Congress on Cybermatics (Cybermatics)*, 2023, pp. 479–486.
- [111] A. Zolghadr *et al.*, "Learning and Reconstructing Conflicts in O-RAN: A Graph Neural Network Approach," *arXiv preprint arXiv:2412.14119*, 2024.
- [112] P. B. del Prever *et al.*, "PACIFISTA: Conflict Evaluation and Management in Open RAN," *arXiv preprint arXiv:2405.04395*, 2024.
- [113] O-RAN Software Community. (2023, Sep.) Implement Authentication and Authorization in Internal xApp-facing and Operator-facing Interfaces. O-RAN Software Community. Accessed 2024. [Online]. Available: <https://lf-o-ran-sc.atlassian.net/issues/RIC-1009>
- [114] I. Gomez-Miguel *et al.*, "srsLTE: An Open-source Platform for LTE Evolution and Experimentation," in *ACM International Workshop on Wireless Network Testbeds, Experimental Evaluation, and Characterization (WiNTECH)*, 2016, pp. 25–32.
- [115] VIAVI. (Accessed 2024, May) TeraVM RIC Test. [Online]. Available: <https://www.viavisolutions.com/en-us/products/teravm-ric-test>